CSE4131: Artificial Intelligence

Class Teacher:

Professor Dr. A K M Akhtar Hossain

Dept. of CSE,

University of Rajshahi

Reference Books:

1. Dan W. Patterson, Introduction to Artificial Intelligence & Expert Systems, Professor, University of Texas, USA.
2. Elaine Rich, Kevin Knight, S. B Nair, Artificial Intelligence, 3rd Edition, Tata McGraw Hill, USA.
3. Patrick Henry Winston, Artificial Intelligence , Professor, Massachusetts Institute of Technology, USA.

© Prof. A. K. M. Akhtar Hossain, Dept of CSE, University of Rajshahi

Artificial Intelligence(AI)

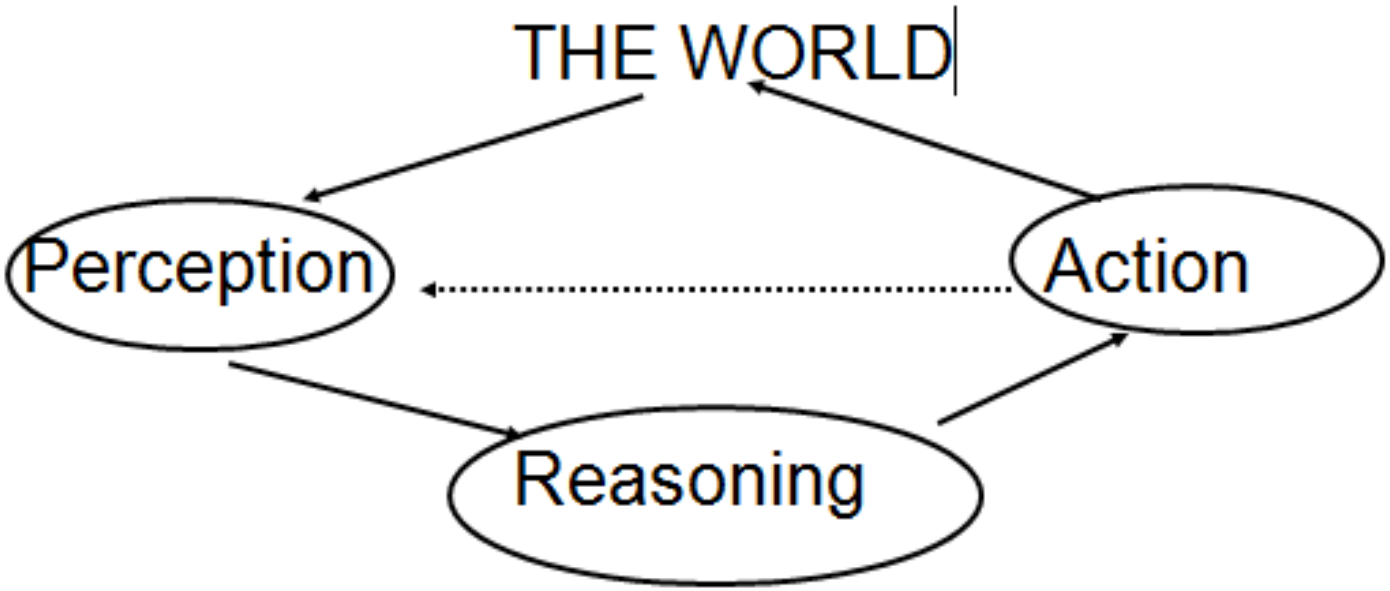
- Definition-1:
- Artificial Intelligence (AI) is the study of computations that make it possible to **perceive**, **reason** and **act**.

- Definition-2:
- Artificial Intelligence (AI) is a branch of computer science concerned with the study and creation of computer systems that exhibit some form of intelligence: such as,
  - ❖ systems that learn new concepts and tasks,
  - ❖ systems that can reason and draw useful conclusions about the world around us,
  - ❖ systems that can understand a natural language or perceive and comprehend a visual **scene**, and
  - ❖ systems that perform other types of feats that require human types of intelligence.

© Prof. A. K. M. Akhtar Hossain, Dept of CSE, University of Rajshahi

Latest Perception of AI

- Three typical components of AI Systems

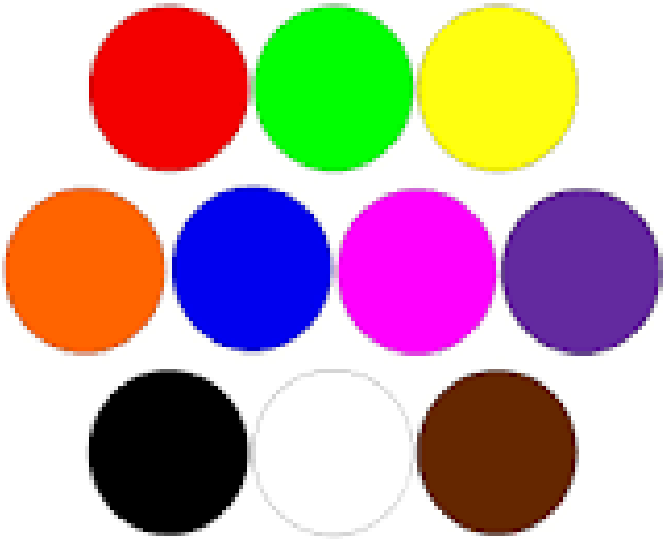


© Prof. A. K. M. Akhtar Hossain, Dept of CSE, University of Rajshahi

Example of AI



Red Rose



Colours



White Rose

© Prof. A. K. M. Akhtar Hossain, Dept of CSE, University of Rajshahi

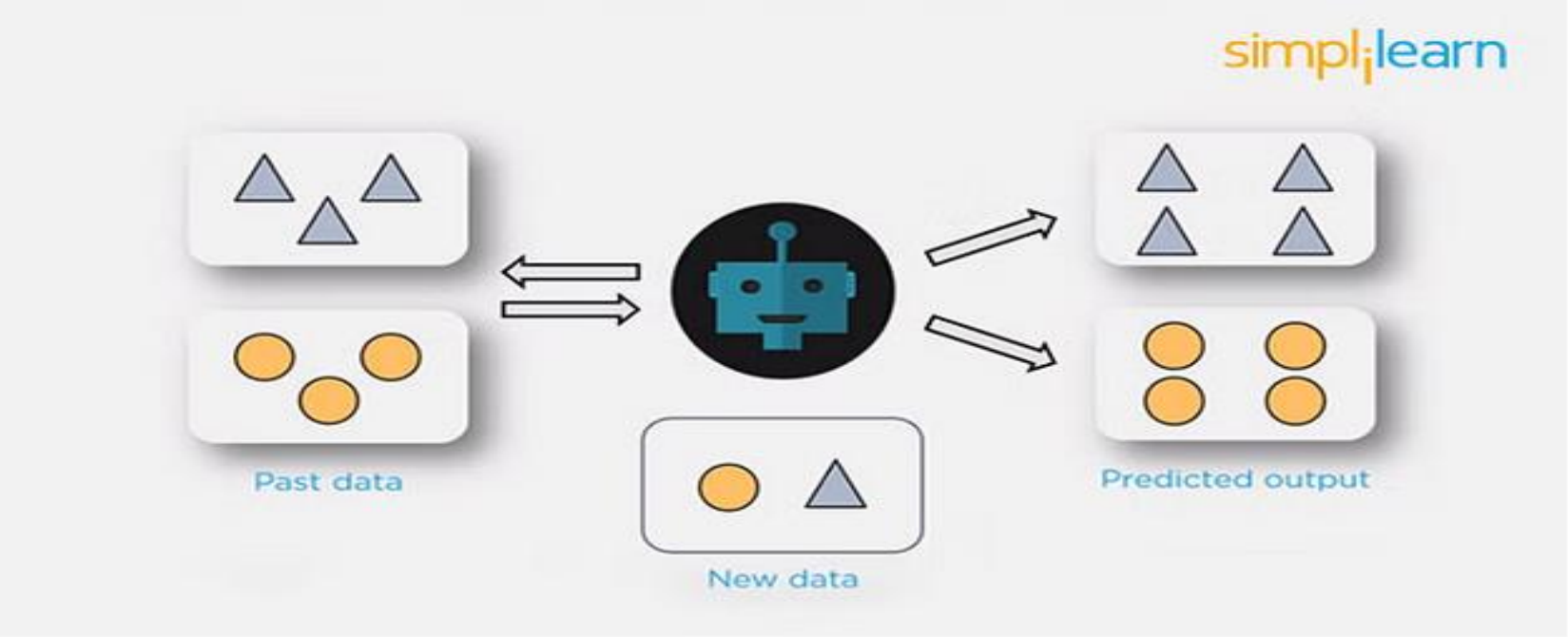
Learning Algorithms

- Basic Types:
- **Supervised Learning**
- **Unsupervised Learning**
- **Reinforcement Learning**

© Prof. A. K. M. Akhtar Hossain, Dept of CSE, University of Rajshahi

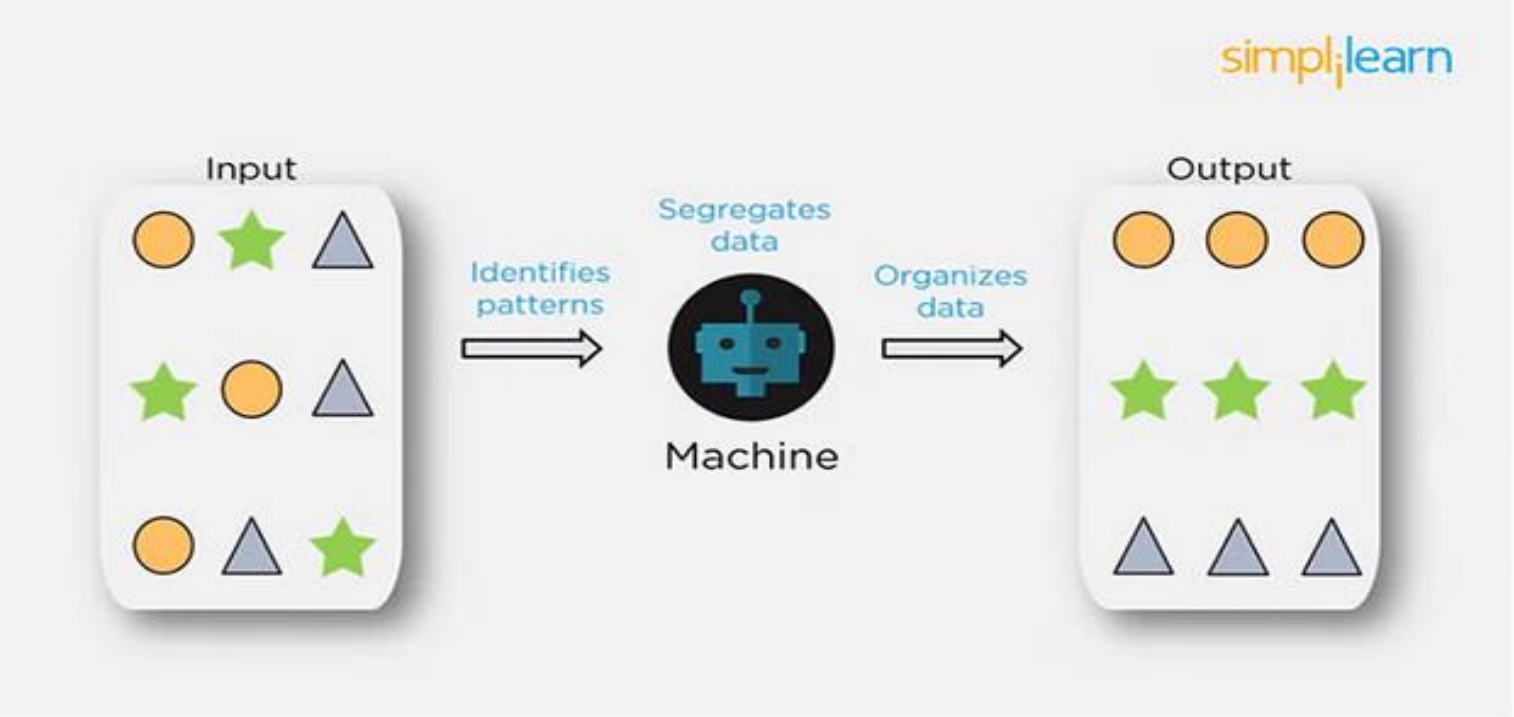
Supervised Learning

- In supervised machine learning, a model makes predictions or decisions based on past or labeled data or learning. Labeled data refers to sets of data that are given tags or labels, and thus made more meaningful.



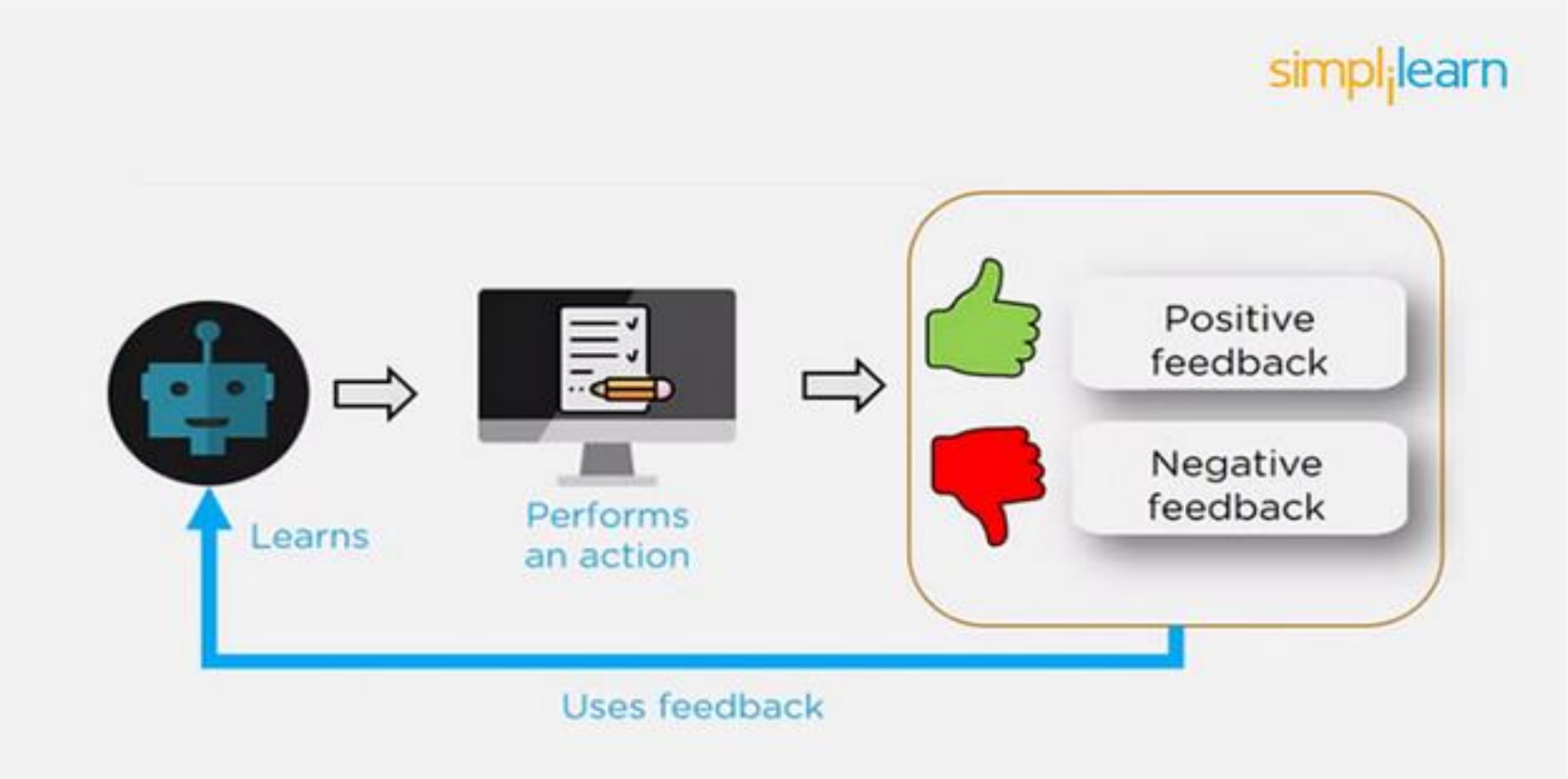
Unsupervised Learning

- In unsupervised learning, we don't have labeled data. A model can identify patterns, anomalies, and relationships in the input data.



Reinforcement Learning

- Using reinforcement learning, the model can learn based on the rewards it received for its previous action.



© Prof. A. K. M. Akhtar Hossain, Dept of CSE, University of Rajshahi

Reinforcement Learning

Consider an environment where an agent is working. The agent is given a target to achieve.

- Every time the agent takes some action toward the target, it is given **positive feedback**.
- If the action taken is going away from the goal, the agent is given **negative feedback**.

© Prof. A. K. M. Akhtar Hossain, Dept of CSE, University of Rajshahi

- **Artificial:**
  - ❖ Produced by human art or effort, rather than originating naturally.
- **Intelligence:**

❖ is the ability to acquire knowledge and use it" [Pigford and Baur]

- **So AI is defined as:**
  - ➢ AI is the study of ideas that enable computers to be intelligent.
  - ➢ AI is the part of computer science concerned with design of computer systems that exhibit human intelligence(From the Concise Oxford Dictionary)

© Prof. A. K. M. Akhtar Hossain, Dept of CSE, University of Rajshahi

Modern Applications of Artificial Intelligence:

Examples:

- Manufacturing robots.
- Self-driving cars.
- Smart assistants.
- Healthcare management.
- Automated financial investing.
- Virtual travel booking agent.
- Social media monitoring.

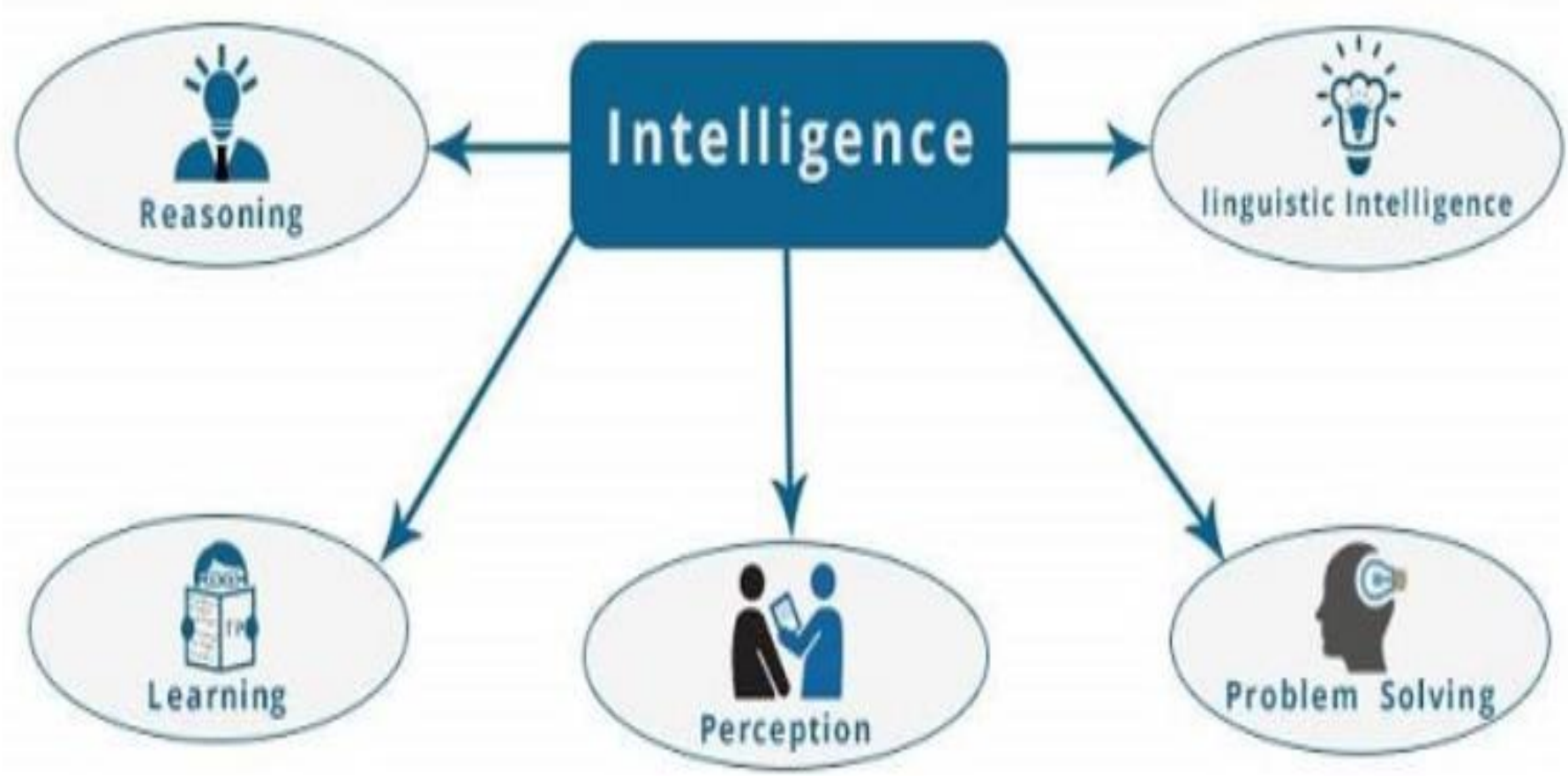
© Prof. A. K. M. Akhtar Hossain, Dept of CSE, University of Rajshahi

What is Intelligence?

- Intelligence is a property of mind that encompasses many related mental abilities, such as the capabilities to
  - Reasoning
  - Learning
  - Problem Solving
  - Perception
  - Linguistic Intelligence

© Prof. A. K. M. Akhtar Hossain, Dept of CSE, University of Rajshahi

What is Intelligence?



© Prof. A. K. M. Akhtar Hossain, Dept of CSE, University of Rajshahi

Difference between Human and Machine Intelligence

- Humans perceive by patterns whereas the machines perceive by set of rules and data.
- Humans store and recall information by patterns, machines do it by searching algorithms.
- Humans can figure out the complete object even if some part of it is missing or distorted; whereas the machines cannot do it correctly.

© Prof. A. K. M. Akhtar Hossain, Dept of CSE, University of Rajshahi

Observations	Human Intelligence	Machine Intelligence
Origin	Human beings are created with the natural ability to think, reason, recall, etc.	Machine intelligence is an innovation which is created by human (scientists).
Speed	Process information slower.	Process information faster.
Decision making	Humans' decisions may be influenced by subjective elements which are not based on figures alone.	Machine intelligence is highly objective in decision making as it analyzes based on purely gathered data.
Accuracy	May be less accurate.	More accurate.
Energy	Uses 25 watts.	Uses 2 watts.
Adaptation	Can easily adapt to changes.	Cannot adapt to changes well.
Multitasking	Can multitask easily.	Cannot multitask easily.
Self-Awareness	Has self-awareness	Still working towards self-awareness.
Social Interaction	Excellent social skills.	Low social skills.
General Function	Innovation	Optimization

© Prof. A. K. M. Akhtar Hossain, Dept of CSE, University of Rajshahi

Areas of AI

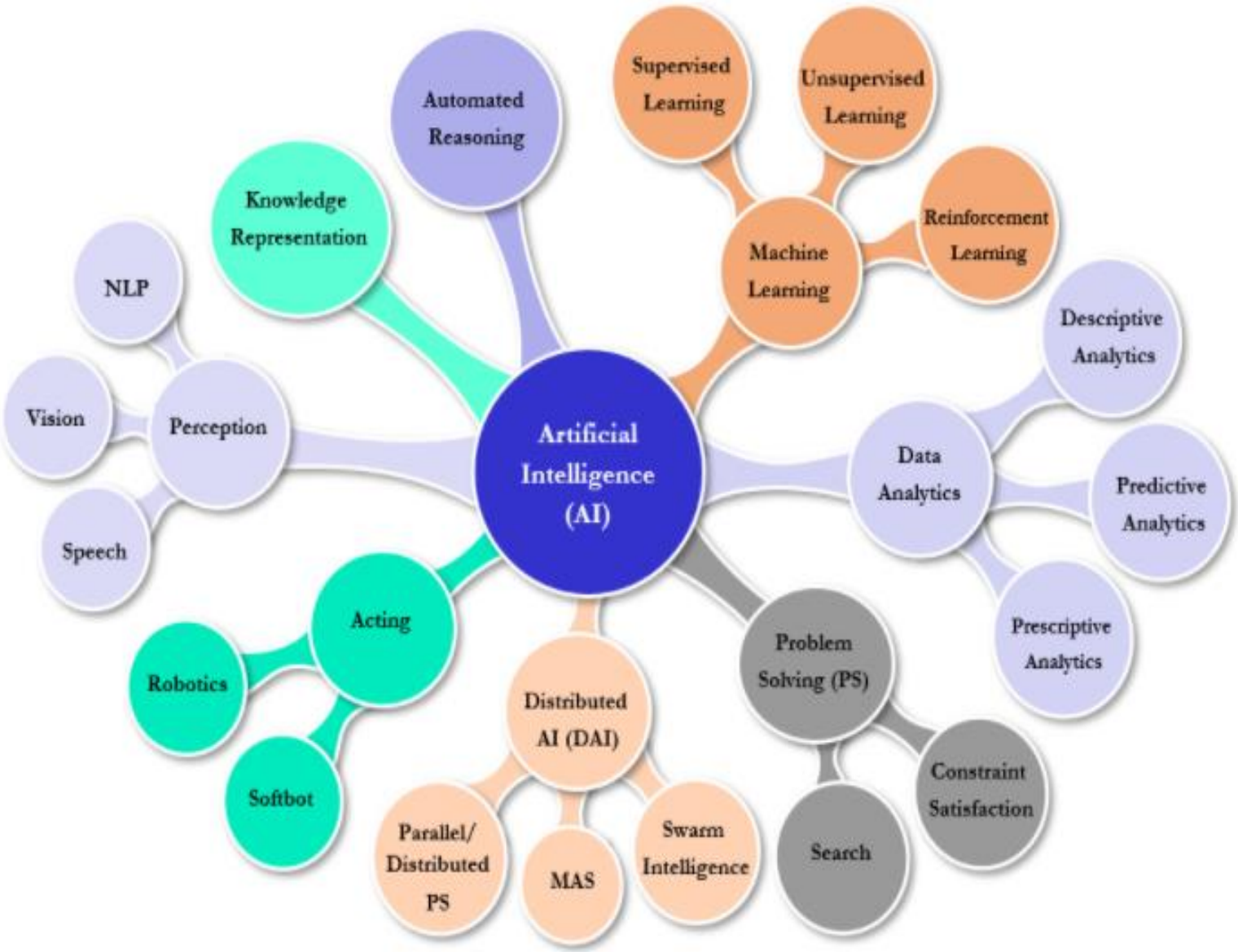
- Areas of AI are:
  - Artificial Neural Networks
  - Machine Learning
  - Deep Learning
  - Robotics
  - Knowledge representation
  - Expert systems
  - Natural language Processing
  - Computer vision
  - Theorem proving
  - Game playing
  - Reasoning dealing with uncertainty and decision making

© Prof. A. K. M. Akhtar Hossain, Dept of CSE, University of Rajshahi

Areas of AI

- Learning models, inference techniques
- Pattern recognition
- Search and matching algorithms
- Logic (fuzzy, temporal, modal) in AI
- Planning and scheduling
- Understanding spoken expressions
- Intelligent tutoring systems
- Machine translation systems

© Prof. A. K. M. Akhtar Hossain, Dept of CSE, University of Rajshahi



© Prof. A. K. M. Akhtar Hossain, Dept of CSE, University of Rajshahi

Applications

- **Business :** Financial strategies, give advice
- **Engineering:** check design, offer suggestions to create new product
- **Manufacturing:** Robotics, Assembly, inspection & maintenance, military weapons, satellites, Space rocket, Sky rockets, Missiles, Ballistic missiles
- **Mining:** used when conditions are dangerous
- **Hospital :** monitoring, diagnosing & prescribing
- **Education :** In teaching and research
- **Household :** AI based Devices for cooking, washing & storing, etc.
- **Farming :** prune trees & selectively harvest mixed crops, biotechnology, genetics engineering.

© Prof. A. K. M. Akhtar Hossain, Dept of CSE, University of Rajshahi

History of AI

- 1943: early beginnings
  - McCulloch & Pitts: Boolean circuit model of brain
- 1950: Turing
  - Turing's "Computing Machinery and Intelligence“
- 1956: birth of AI
  - Dartmouth meeting: "Artificial Intelligence“ name adopted
- 1950s: initial promise
  - Early AI programs, including
  - Samuel's checkers program
  - Newell & Simon's Logic Theorist
- 1955-65: “great enthusiasm”
  - Newell and Simon: GPS, general problem solver
  - Gelertner: Geometry Theorem Prover
  - McCarthy: invention of LISP

© Prof. A. K. M. Akhtar Hossain, Dept of CSE, University of Rajshahi

History of AI

- **1966—73: Reality dawns**
  - Realization that many AI problems are intractable
  - Limitations of existing neural network methods identified
    - Neural network research almost disappears
- **1969—85: Adding domain knowledge**
  - Development of knowledge-based systems
  - Success of rule-based expert systems,
    - E.g., DENDRAL, MYCIN
    - But were brittle and did not scale well in practice
- **1986-- Rise of machine learning**
  - Neural networks return to popularity
  - Major advances in machine learning algorithms and applications
- **1990-- Role of uncertainty**
  - Bayesian networks as a knowledge representation framework
- **1995– 2021 AI as Science**
  - Integration of learning, reasoning, knowledge representation, Deeping learning
  - AI methods used in vision, language, data mining, Robotics etc

© Prof. A. K. M. Akhtar Hossain, Dept of CSE, University of Rajshahi

What is knowledge?

- Knowledge can be defined as the body of facts and principles accumulated by human-kind or the fact, or state of knowing.
- Knowledge should not be confused with data.
- In biological organisms, knowledge is likely stored as complex structures of interconnected neurons.
- In Computer Science, knowledge is also stored as complex structures, but in the form of collections of magnetic spots and voltage states.

© Prof. A. K. M. Akhtar Hossain, Dept of CSE, University of Rajshahi

Five types of knowledge

- **Meta Knowledge** – It's a knowledge about knowledge and how to gain them
- **Heuristic – Knowledge** – Representing knowledge of some expert in a field or subject.
- **Procedural Knowledge** – Gives information/ knowledge about how to achieve something.
- **Declarative Knowledge** – Its about statements that describe a particular object and its attributes , including some behavior in relation with it.
- **Structural Knowledge** – Describes what relationship exists between concepts/ objects.

© Prof. A. K. M. Akhtar Hossain, Dept of CSE, University of Rajshahi

Types of Knowledge

- Procedural knowledge is compiled knowledge related to the performance of some task.
- Example: To solve the algebraic equations.

- Declarative knowledge is the passive knowledge expressed as statements of facts about the world.
- Example: Personnel data in a database.

© Prof. A. K. M. Akhtar Hossain, Dept of CSE, University of Rajshahi

Types of Knowledge

- **Heuristic knowledge** is a special type of knowledge, which is used by human to solve complex problems.
- Examples: Good judgments, tricks, rules of thumb.

© Prof. A. K. M. Akhtar Hossain, Dept of CSE, University of Rajshahi

Meta Knowledge

- **Meta Knowledge** is knowledge about knowledge.
- The term is used to describe things such as tags, models and taxonomies that describe knowledge.
- Several academic areas including bibliography, the study of books, the philosophy of knowledge, are also considered meta-knowledge.

© Prof. A. K. M. Akhtar Hossain, Dept of CSE, University of Rajshahi

Structural knowledge

- According to Jonassen (2000), **structural knowledge**, which connects declarative and procedural knowledge.
- Structural Knowledge – It is **a basic problem-solving knowledge that describes the relationship between concepts and objects**.
- **Structural knowledge** is knowledge of how the ideas within a domain are integrated and interrelated.
- Example: **family relations, social relations, logical relations, administrative organizations, military hierarchies**, etc.

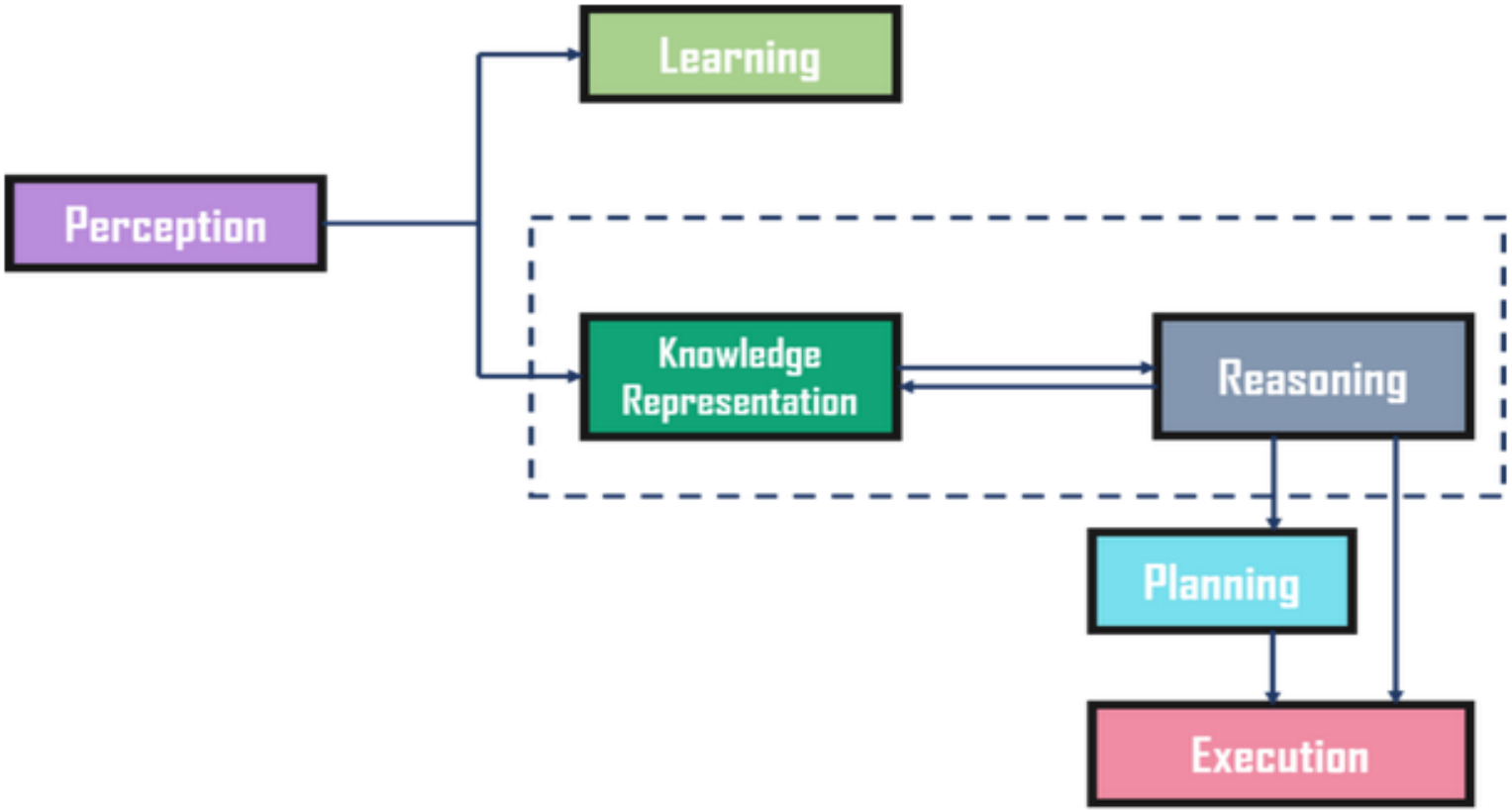
© Prof. A. K. M. Akhtar Hossain, Dept of CSE, University of Rajshahi

Cycle of Knowledge Representation in AI

- Artificial Intelligent Systems usually consist of various components to display their intelligent behavior.
- Some of these components include:
    - **Perception**
    - **Learning**
    - **Knowledge Representation & Reasoning**
    - **Planning**
    - **Execution**

© Prof. A. K. M. Akhtar Hossain, Dept of CSE, University of Rajshahi

Cycle of Knowledge Representation in AI



© Prof. A. K. M. Akhtar Hossain, Dept of CSE, University of Rajshahi

Some Terminology on Knowledge

- **Belief:** We define belief as essentially any meaningful and coherent expression that can be represented. Thus a belief may be true or false.
- **Hypothesis:** We define a hypothesis as a justified belief that is not known to be true.

  Thus , a hypothesis is a belief which is backed up with some supporting evidence, but it may still be false.
- **Finally, we define knowledge as true justified belief (Dan W. Petterson).**

© Prof. A. K. M. Akhtar Hossain, Dept of CSE, University of Rajshahi

Academic Disciplines required to understand AI.

• Philosophy	Logic, methods of reasoning, mind as physical system, foundations of learning, language, rationality.
• Mathematics/ Statistics	Formal representation and proof, algorithms, computation, decision theory, probability.
• Economics	utility, costing
• Neuroscience	neurons as information processing units.
• Psychology/ Cognitive Science	how do people behave, perceive, process information, represent knowledge.
• Computer engineering	building faster computers
• Control theory	design systems that maximize an objective function over time
• Linguistics	knowledge representation, grammar, programming languages

© Prof. A. K. M. Akhtar Hossain, Dept of CSE, University of Rajshahi

END TODAY

THANKS

© Prof. A. K. M. Akhtar Hossain, Dept of CSE, University of Rajshahi

Agents in Artificial Intelligence

Class Teacher:

Prof. A K M Akhtar Hossain

Dept. of CSE

University of Rajshahi, Bangladesh

Agents in AI

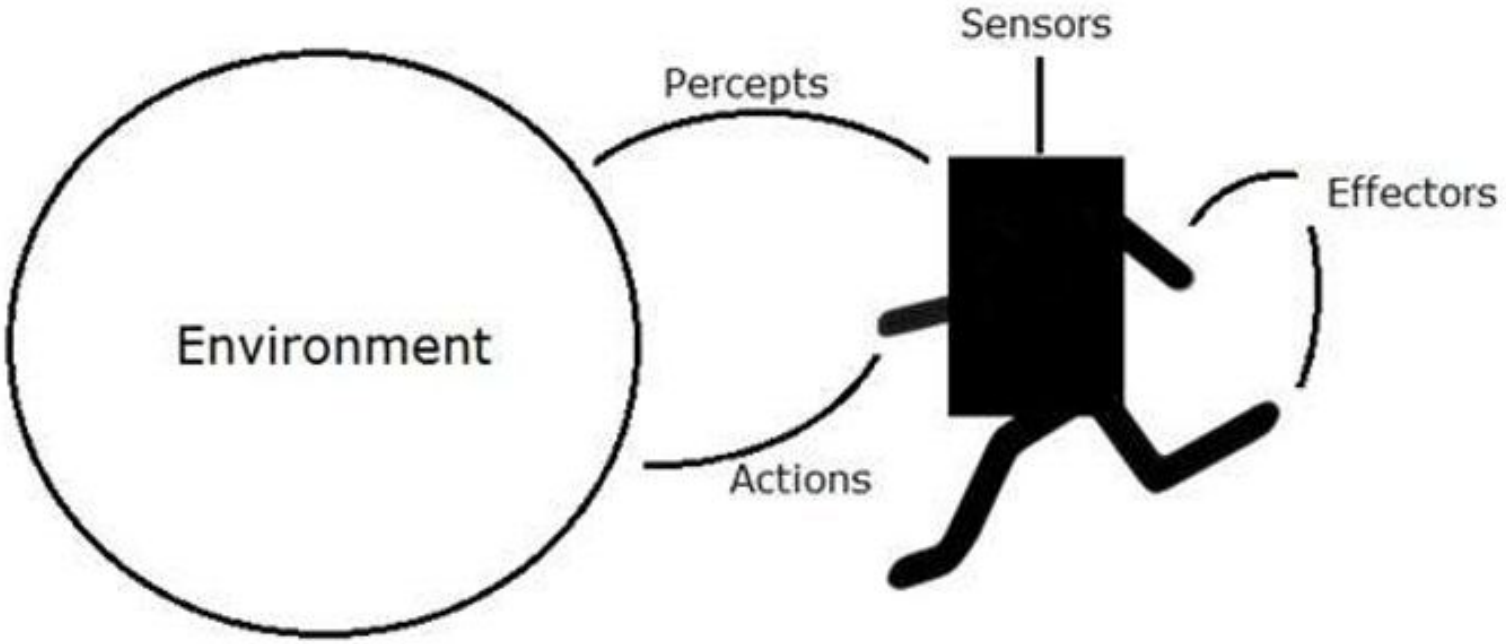
- An **intelligent agent** is an entity that makes a decision, that enables **artificial intelligence** to be put into action.
- An **intelligent agent /agent** is anything that can perceive its environment through **sensors** and acts upon that environment through **effectors.**

- **Basic Types of Agent in Real world Environment:**
  - **A human agent**
  - **An Artificial (robotic) agent**
  - **A software agent**

Agents in AI

- **Basic Types of Agent in Real world Environment:**
- A **human agent** has sensory organs such as eyes, ears, nose, tongue and skin parallel to the sensors, and other organs such as hands, legs, mouth, for effectors.
- An **Artificial (robotic) agent** replaces cameras and infrared range finders for the sensors, and various motors and actuators for effectors.
- A **software agent** has encoded bit strings as its programs and actions.

Agents in AI



Agents in AI

- Agent Terminology:
  - **Performance Measure of Agent** – It is the criteria, which determines how successful an agent is.
  - **Behavior of Agent** – It is the action that agent performs after any given sequence of percepts.
  - **Percept** – It is agent's perceptual inputs at a given instance.
  - **Percept Sequence** – It is the history of all that an agent has perceived till date.
  - **Agent Function** – It is a map from the precept sequence to an action.

Agents in AI

- **Rationality:**
  - Rationality is nothing but status of being reasonable, sensible, and having good sense of judgment.
  - Rationality is concerned with expected actions and results depending upon what the agent has perceived.
  - Performing actions with the aim of obtaining useful information is an important part of rationality.
- **What is Ideal Rational Agent?**
- An ideal rational agent is the one, which is capable of doing expected actions to maximize its performance measure, on the basis of −
  - Its percept sequence
  - Its built-in knowledge base

Agents in AI

- **Rationality of an agent depends on the following:**
  - The **performance measures**, which determine the degree of success.
  - Agent's **Percept Sequence** till now.
  - The agent's **prior knowledge about the environment**.
  - The **actions** that the **agent can carry out**.
- A **rational agent** always performs **right action**, where the **right action means** the action that causes the agent to be **most successful** in the given percept sequence.
- The **problem the agent** solves is characterized by **Performance Measure**, **Environment**, **Actuators,** and **Sensors**.

Agents in AI

- Some Terminologies for Agents in AI:
- **Percept:** It is agent's perceptual inputs at a given instance.
- **Actuator:** An **actuator** is a device that makes something move or operate. An **actuator** is a device that uses a form of power to convert a control signal into mechanical motion.
- **Sensor:** A **sensor** is a device that detects the change in the environment and responds to some output on the other system.
- **Effector:** An **effector** is any device that affects the environment.
  - Robots control their **effectors**, which are also known as end **effectors**.
  - **Effectors** include legs, wheels, arms, fingers, wings and fins.
  - Controllers cause the **effectors** to produce desired effects on the environment.

Agents in AI

- **The Structure of Intelligent Agents:**
  - Agent's structure can be viewed as –
    - **Agent = Architecture + Agent Program**
    - **Architecture** = the machinery that an agent executes on.
    - **Agent Program** = an implementation of an agent function.

Agents in AI

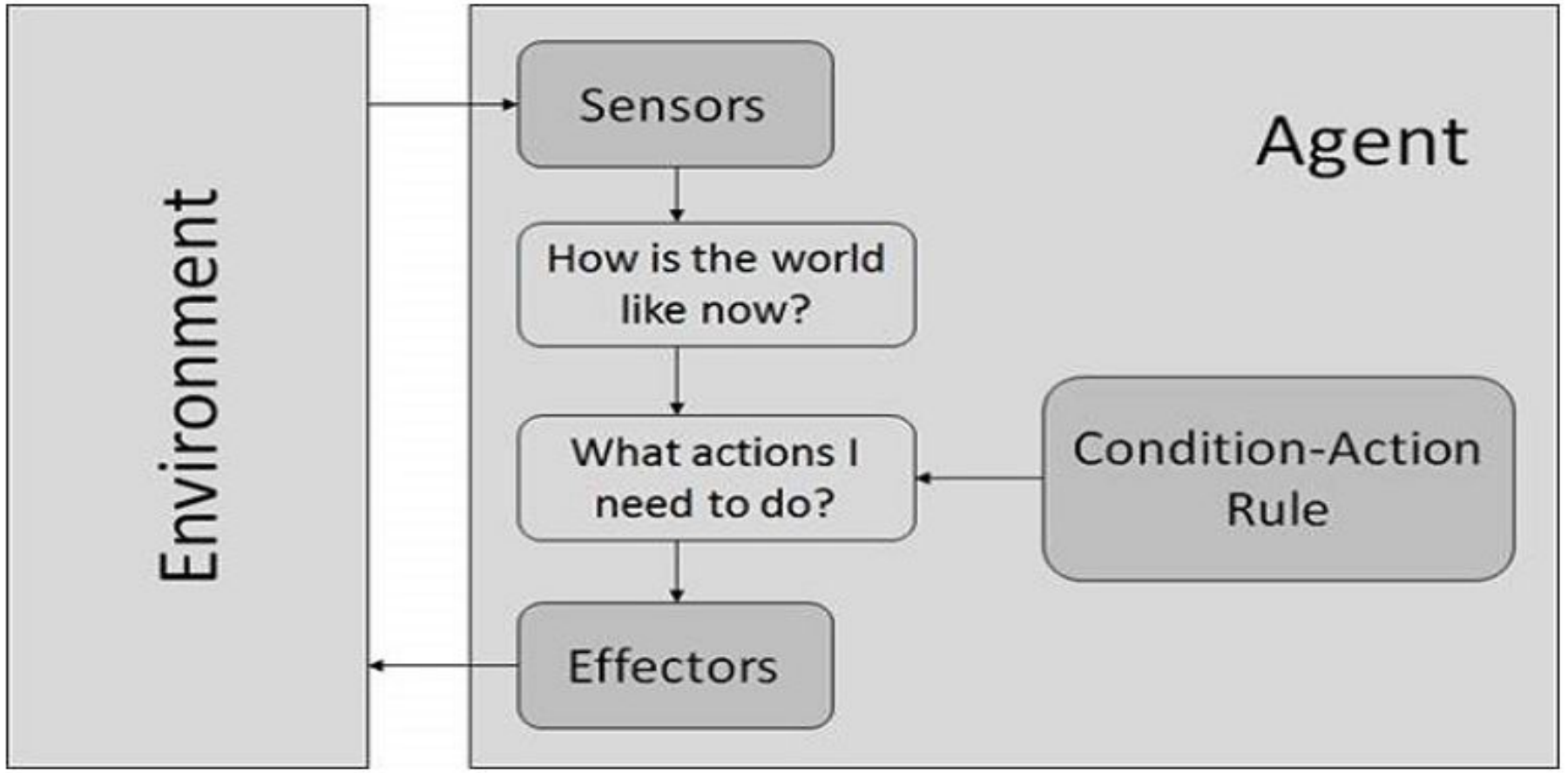
- The Structure of Intelligent Agents:
  - Simple Reflex Agents
  - Model Based Reflex Agents
  - Goal Based Agents
  - Utility Based Agents
  - **Learning Based Agents:**

The Structure of Intelligent Agents:

- **Simple Reflex Agents**
  - They **choose actions** only based on the **current percept**.
  - They are rational only if a **correct decision** is made only on the basis of **current precept**.
  - Their environment is completely **observable**.

Simple Reflex Agents

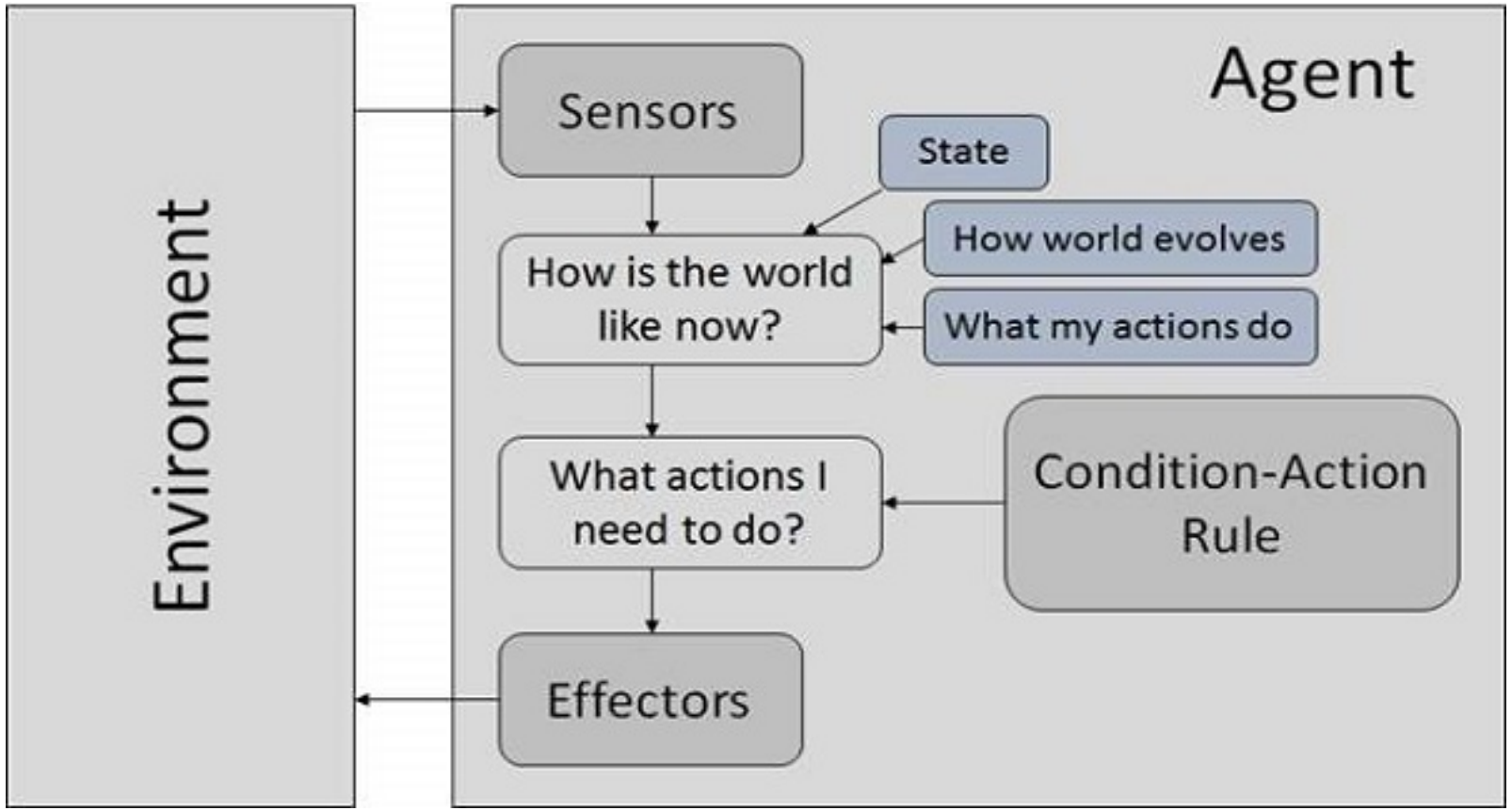
Condition-Action Rule − It is a rule that maps a state (condition) to an action.



The Structure of Intelligent Agents:

- **Model Based Reflex Agents:**
  - They use a model of the world to **choose their actions**. They maintain an **internal state**.
  - **Model** – knowledge about "**how the things happen in the world**".
  - **Internal State** – It is a representation of unobserved aspects of **current state** depending on **percept history**.
- **Updating the state requires the information about –**
  - How the world evolves.
  - How the agent's actions affect the world.

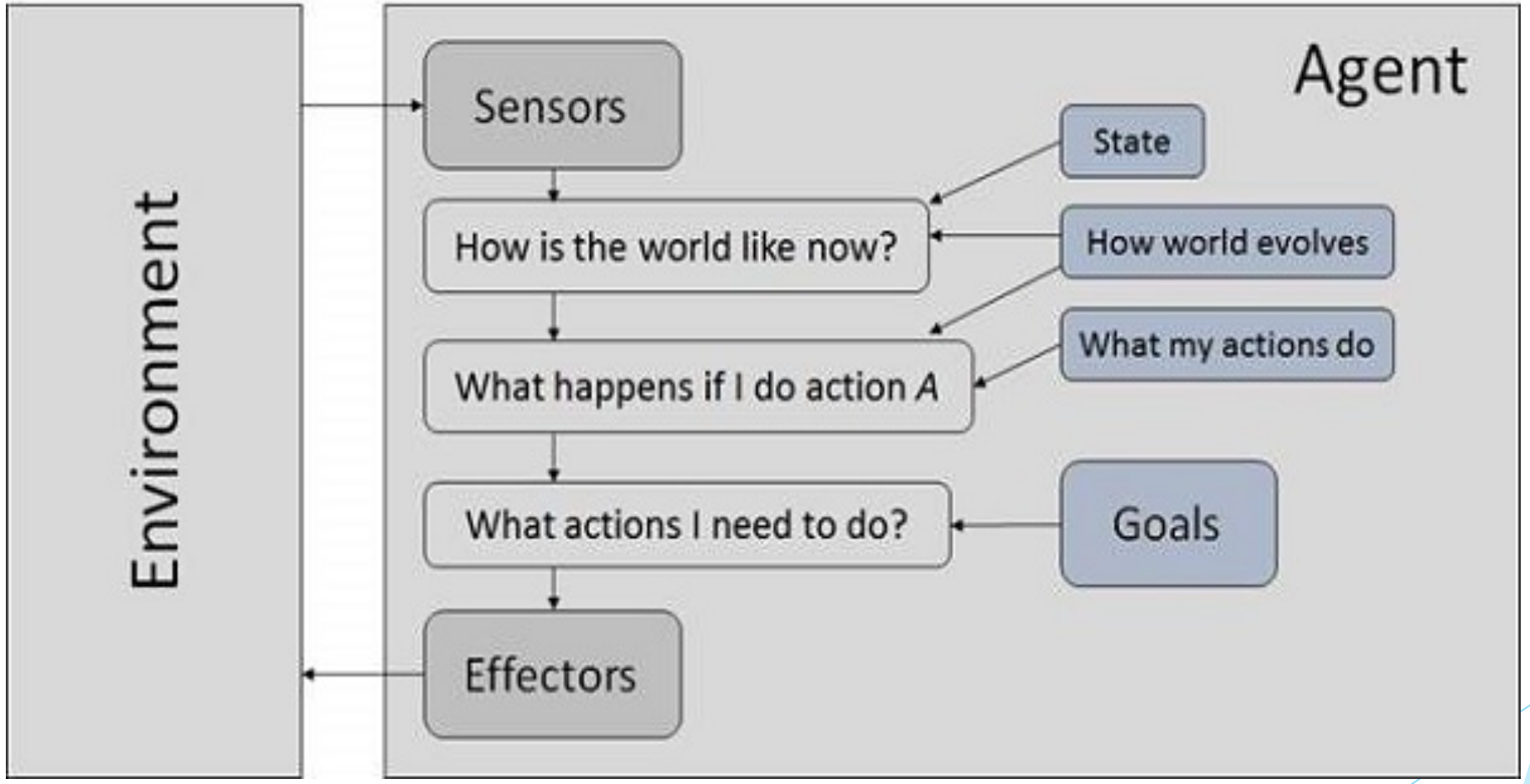
Model Based Reflex Agents:



The Structure of Intelligent Agents:

- **Goal Based Agents:**
  - They **choose their actions** in order to **achieve goals**.
  - **Goal-based approach** is more flexible than reflex agent since the **knowledge supporting a decision** is explicitly modeled, thereby allowing for **modifications**.
  - **Goal** – It is the description of **desirable situations**.

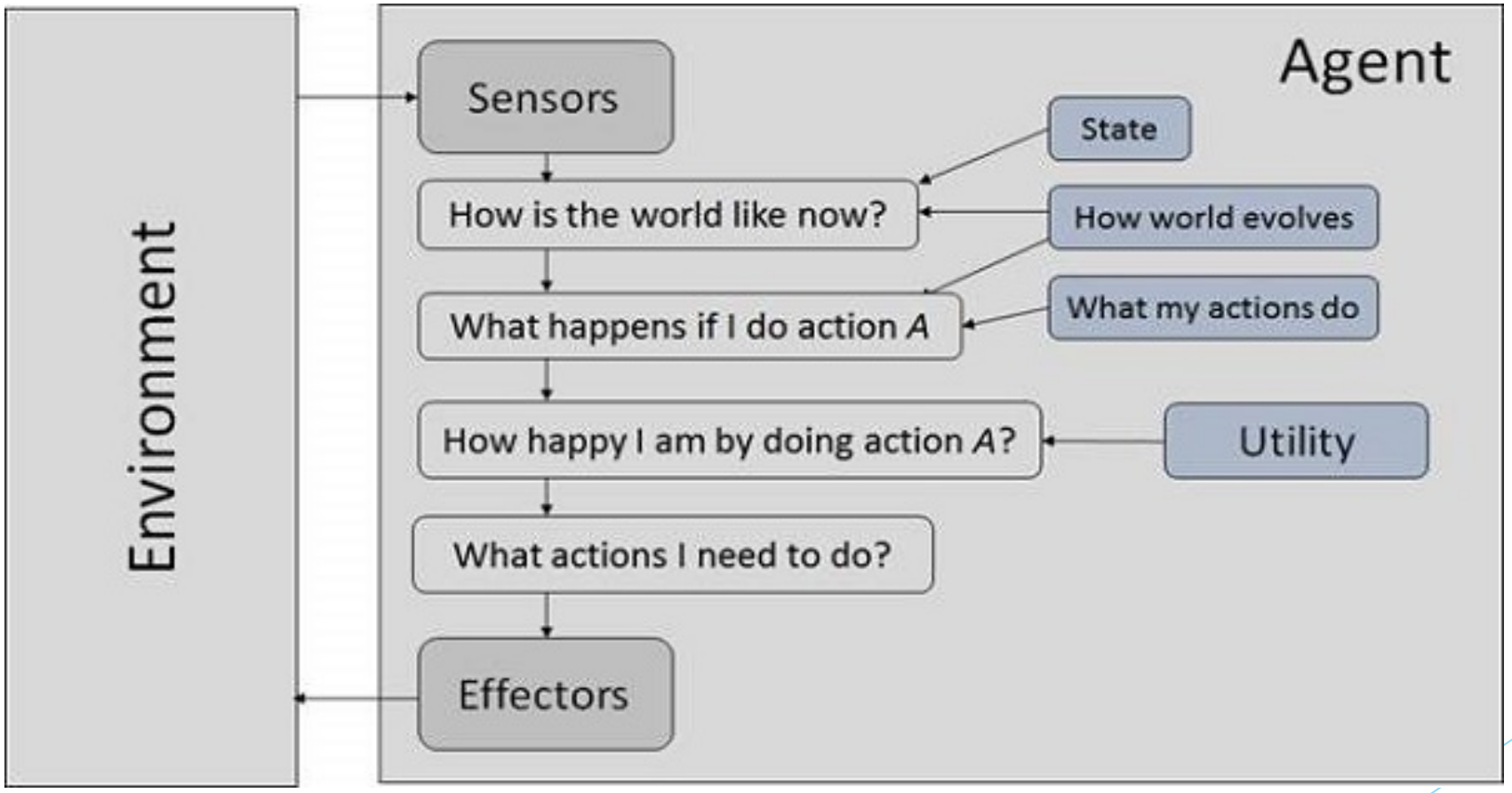
Goal Based Agents



The Structure of Intelligent Agents:

- **Utility Based Agents:**
  - They **choose actions** based on a **preference** (utility) for **each state**.
  - Goals are **inadequate** when –
  - There are **conflicting goals**, out of which only few can be achieved.
  - Goals have some **uncertainty** of being achieved and you need to consider **likelihood of success** against the **importance of a goal**.

Utility Based Agents:



Learning-based Agent

- A **learning-based agent** is an agent capable of learning from experience.
- It has the capability of automatic information acquisition and integration into the system.
- Any agent designed and expected to be successful in an uncertain environment is considered to be learning agent.

Learning Based Agents:

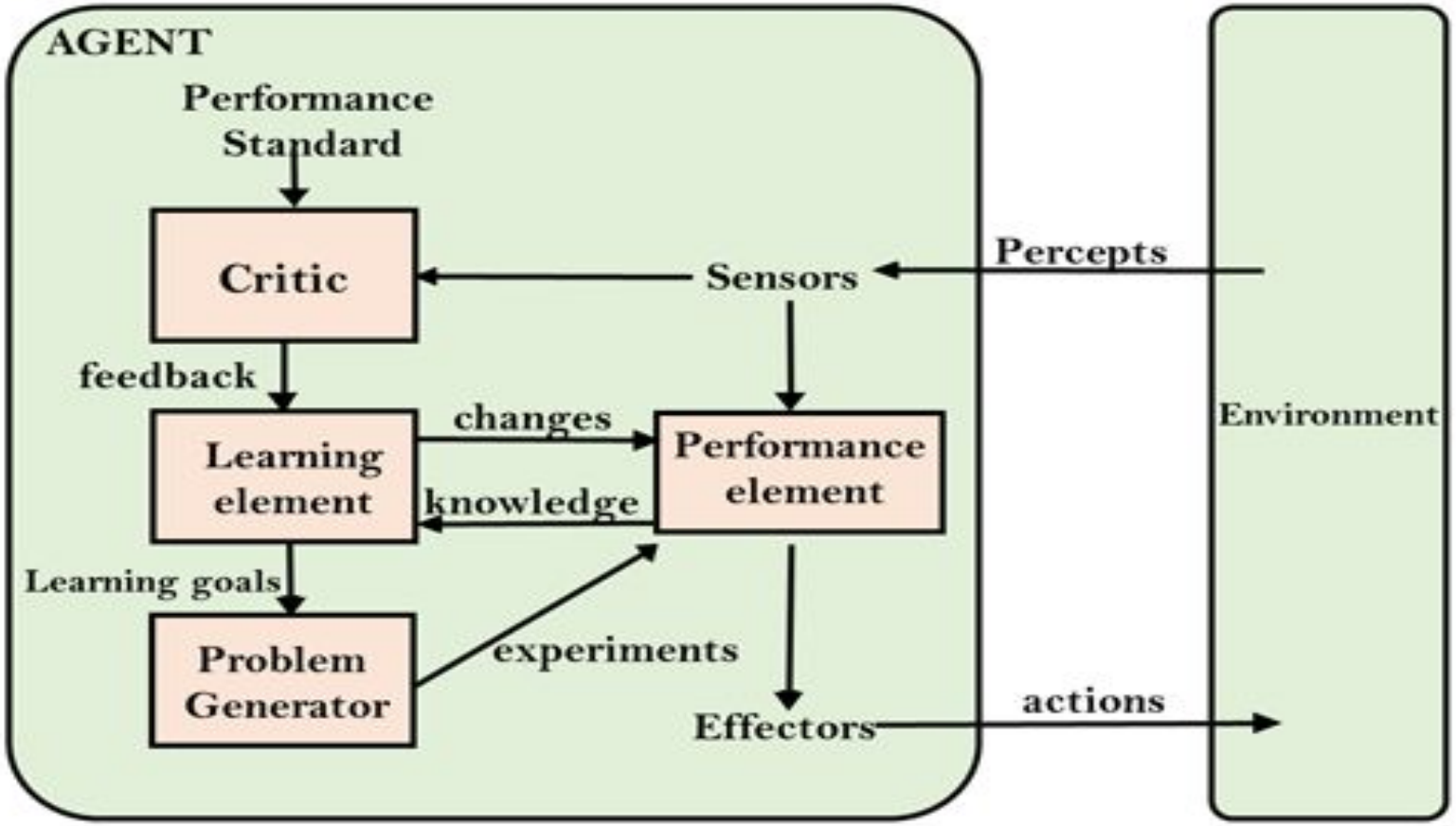
- An AI agent with learning capabilities or the ability to learn from its experiences in the past is referred to as a "learning agent."
- A learning based agent primarily consists of these four conceptual parts:

1. **Element of learning**
2. **Critic**
3. **Element of performance**
4. **Problem Generator**

Learning Based Agents:

- **Element of learning:** It is accountable for producing advancements by absorbing environmental knowledge.
- **Critic:** The learning component uses criticism to gauge how well the agent is performing in relation to a predetermined performance benchmark.
- **Element of performance:** It is in charge of choosing an external action.
- **Problem Generator:** This element is in charge of making recommendations for activities that will result in novel and educational experiences.

Learning Based Agents:



Agents and Environment in AI

- **The Nature of Environments:**
  - Some programs operate in the entirely **artificial environment** confined to keyboard input, database, computer file systems and character output on a screen.
  - In contrast, some **software agents** (**software robots** or **softbots)** exist in rich, unlimited **softbots domains**.
  - The **simulator** has a very **detailed**, **complex environment**.
  - The **software agent** needs to choose from a **long array of actions** in real time.
  - A **softbot designed** to scan the online preferences of the customer and show interesting items to the customer works in the **real** as well as an **artificial environment**.

The Nature of Environments:

- The most famous **artificial environment** is the **Turing Test environment**, in which **one real** and other **artificial agents** are **tested on equal ground**.
- This is a very **challenging environment** as it is **highly difficult** for a **software agent to perform** as well as a human.

What are the challenges arising to design agents for multi-agent environments?

- **1.Non-Stationary Environment:**

- Because multiple agents act simultaneously, the environment keeps changing.
- The agent cannot assume that rules or patterns remain stable, making learning and prediction harder.

The challenges arising to design agents for multi-agent environments:

- **2. Coordination and Cooperation Difficulties**
- Agents must synchronize actions, share tasks, and avoid conflicts.
- Poor coordination can lead to deadlocks, collisions, or inefficiency.

The challenges arising …… …..

- **3. Communication Challenges**
- Agents often need to exchange information. Issues include:
- Limited bandwidth
- Delayed or noisy messages
- High communication cost
- Designing effective protocols

The challenges arising …. …..

- **4. Competition and Conflicting Goals**
- Agents may pursue different or opposing goals. This leads to challenges like:
- Adversarial behavior
- Deception
- Resource contention

The challenges arising …. …..

- **5. Partial Observability**
- Each agent has limited information about the world and other agents' states.
  They must infer missing information or maintain belief states.
- **6. Scalability**
- As the number of agents increases:
- Interactions grow rapidly
- Computation becomes expensive
- Coordination becomes more complex

The challenges arising …. …..

- **7. Emergent and Unpredictable Behavior**
- Unexpected global behavior can arise from simple local interactions.
  Such emergent behavior is hard to analyze, predict, or control.
- **8. Multi-Agent Learning Complexity**
- Learning becomes difficult because:
- Rewards depend on many agents' actions
- Policies must adapt as others change
- Joint action spaces are huge

The challenges arising …. …..

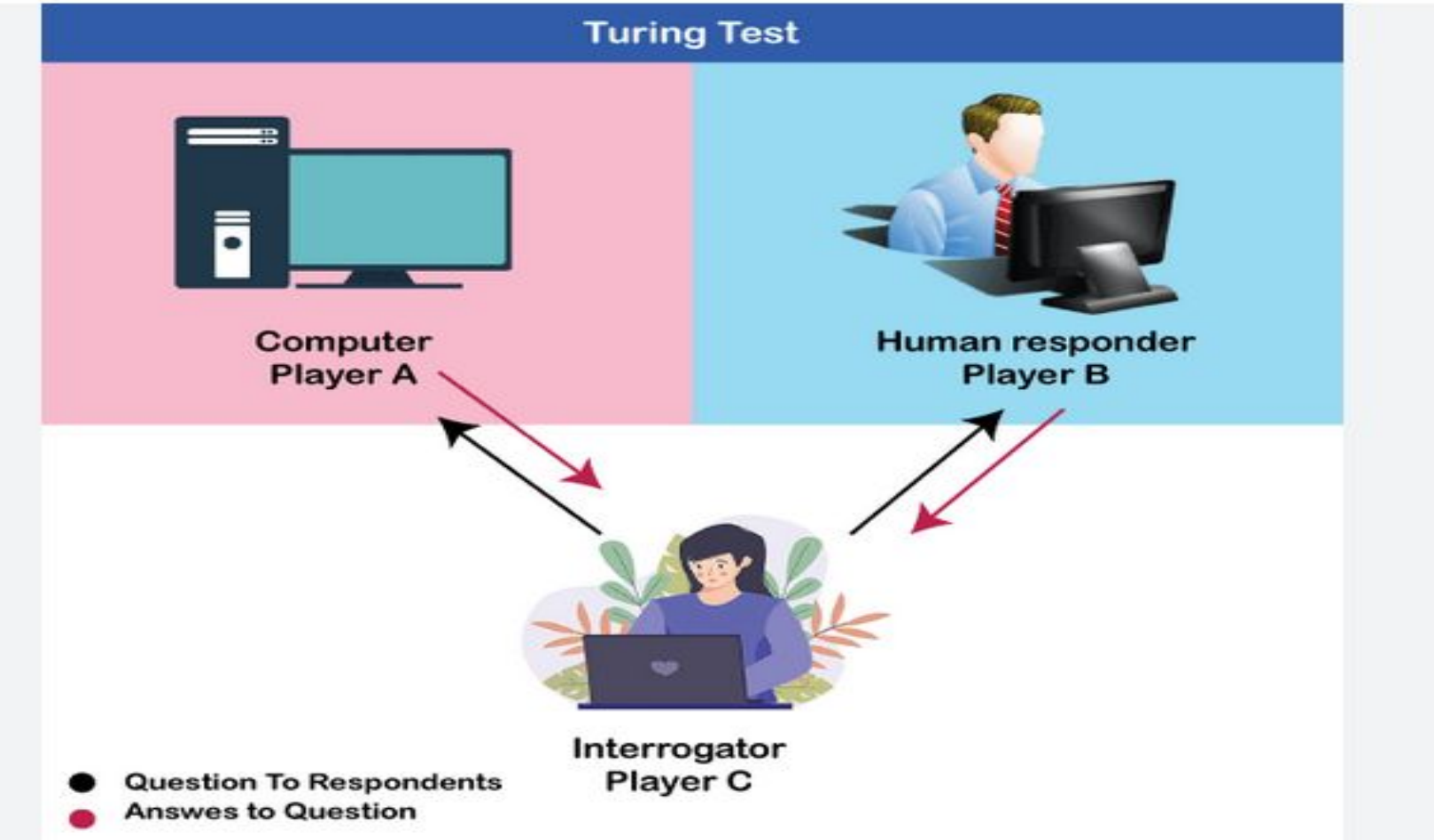
- **9. Conflict Resolution**
- Agents may compete for shared or limited resources. Mechanisms for negotiation, arbitration, or resource allocation become essential.

- **10. Ensuring Safety, Fairness, and Ethics**
- Agents must behave safely in shared environments. Challenges include:
- Avoiding harmful actions
- Maintaining fairness
- Handling adversarial agents

Turing Test

- The success of an intelligent behavior of a system can be measured with Turing Test.
- Two persons and a machine to be **evaluated participate** in the test. Out of the **two persons**, one plays the role of the **tester.**
- Each of them sits in **different rooms**. The **tester** is **unaware of who** is **machine** and **who is a human**.
- He interrogates the questions by typing and sending them to **both intelligences**, to which he receives typed responses.
- This test aims at fooling the tester. If the tester fails to determine **machine's response** from the **human response**, then the machine is said to be intelligent.

Turing Test


Turing Test
Computer
Player A
Human responder
Player B
Interrogator
Player C
Question To Respondents
Answes to Question

Properties of Environment

- **The environment has multifold properties –**
  - **Discrete / Continuous** – If there are a limited number of distinct, clearly defined, states of the environment, the environment is discrete (For example, chess); otherwise it is continuous (For example, driving).
  - **Observable / Partially Observable** – If it is possible to determine the complete state of the environment at each time point from the percepts it is observable; otherwise it is only partially observable.
  - **Static / Dynamic** – If the environment does not change while an agent is acting, then it is static; otherwise it is dynamic.

Properties of Environment

- **Single agent / Multiple agents** – The environment may contain other agents which may be of the same or different kind as that of the agent.
- **Accessible / Inaccessible** – If the agent's sensory apparatus can have access to the complete state of the environment, then the environment is accessible to that agent.
- **Deterministic / Non-deterministic** – If the next state of the environment is completely determined by the current state and the actions of the agent, then the environment is deterministic; otherwise it is non-deterministic.
- **Episodic / Non-episodic** – In an episodic environment, each episode consists of the agent perceiving and then acting. The quality of its action depends just on the episode itself. Subsequent episodes do not depend on the actions in the previous episodes. Episodic environments are much simpler because the agent does not need to think ahead.

END TODAY

Thanks

Deep Neural Network

Sangeeta Biswas, Ph.D.

Associate Professor, Dept. of Computer Science & Engineering, University of Rajshahi, Rajshahi, Bangladesh

Artificial Intelligence, Machine Learning, Deep Learning

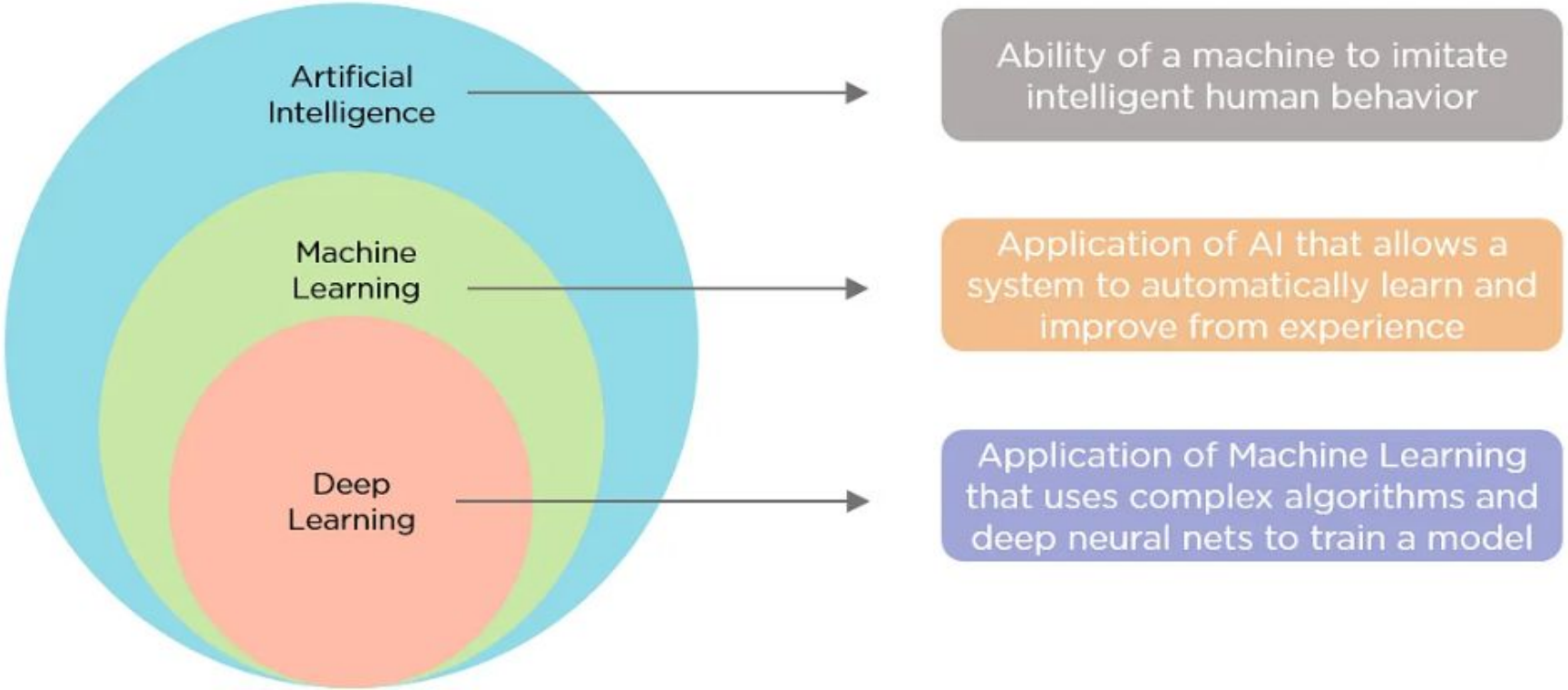
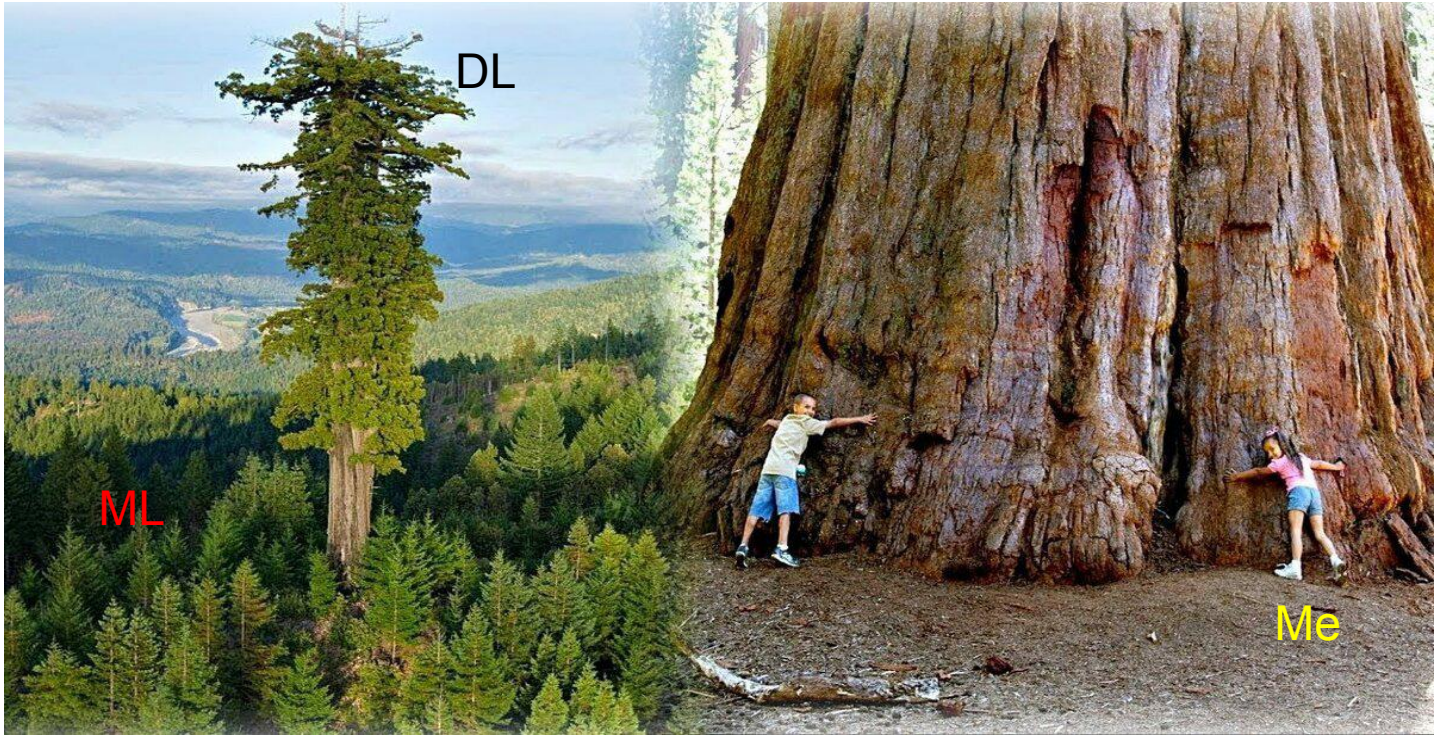


Image Source: Google Search Engine

Machine Learning Vs Deep Learning



Why Deep Learning is So Impressive!

Deep learning achieves:

- Discriminative power
- Predictive Power
- Generative power



Image Source: Google Search Engine

Universal Function Approximator

The main strong point of a deep neural network (DNN) is its universality power.

An DNN can approximate any function no matter how complicated the function is if we can ensure that the network has sufficient number of neurons.

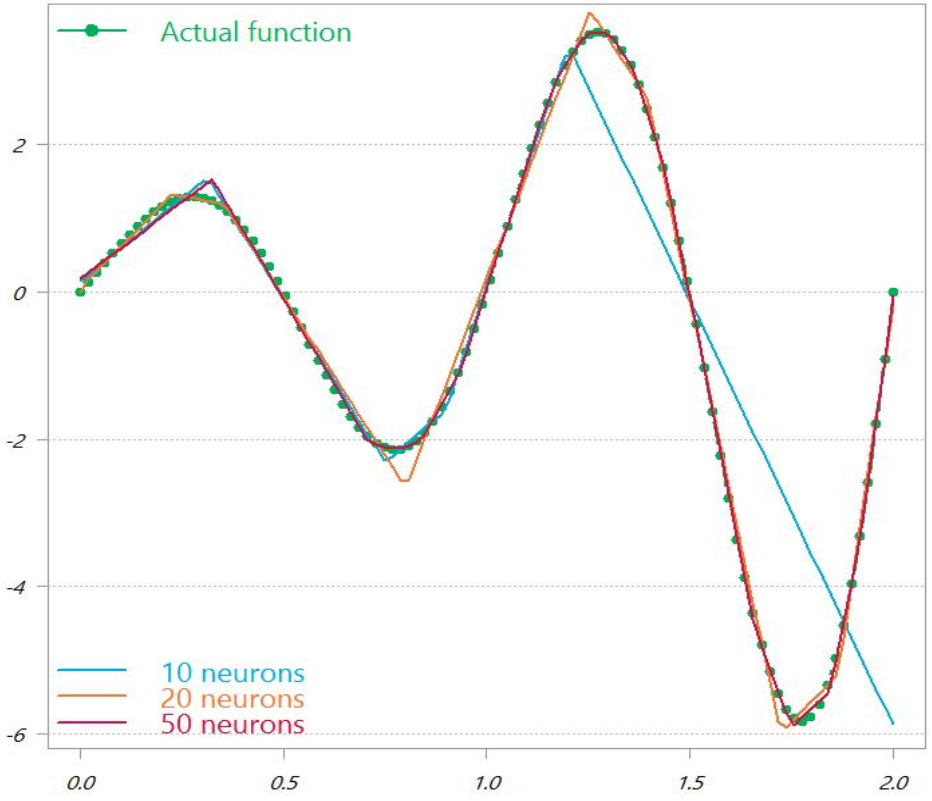


Image Source: Google Search Engine

Artificial Neuron

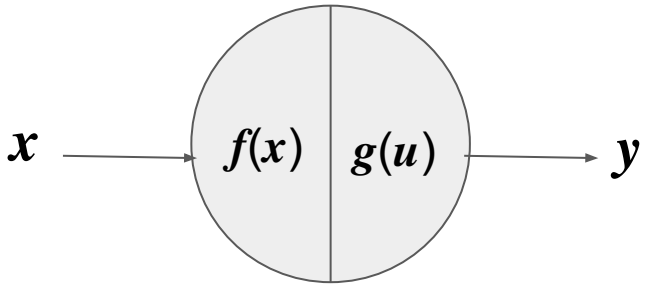
Artificial neuron is nothing but a composite function, $(g \circ f)(x)$

$$u = f(x)$$

$$y = g(u)$$

where

- x is input and y is output
- $f(x)$ is a linear function
- $g(u)$ is generally a nonlinear function



Artificial Neuron

Artificial Neuron

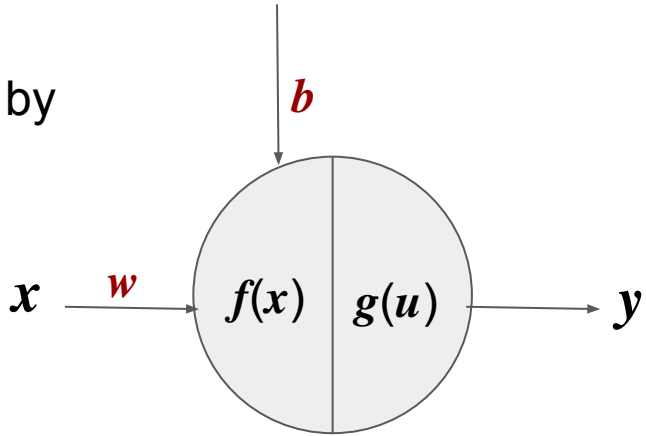
In an artificial neuron, at first input is multiplied by a weight and summed with bias.

$$y = g(f(x))$$

$$u = f(x) = wx + b$$

where

- w is weight which decides how much influence x will have on u.
- b is bias which ensures that u is not too big or too small on average



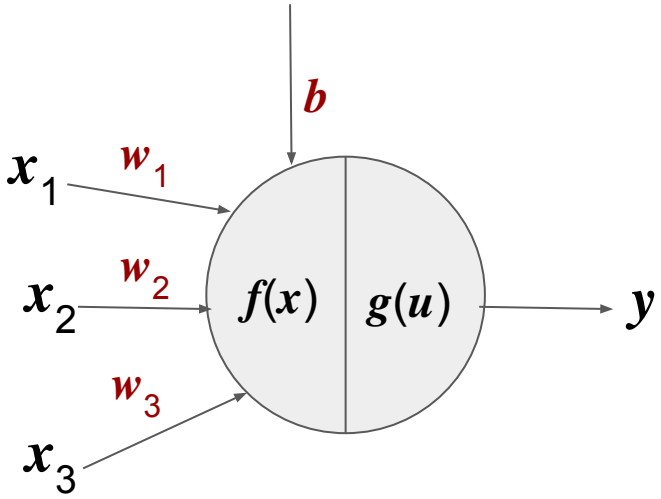
Artificial Neuron

Artificial Neuron

For a vector x, weight w is a vector and bias b is a scalar.

$$u = f(x)$$
$$= w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$
$$= w^T x + b$$

$g(u)$ is known as **activation function**. Generally nonlinear functions are used as activation functions.



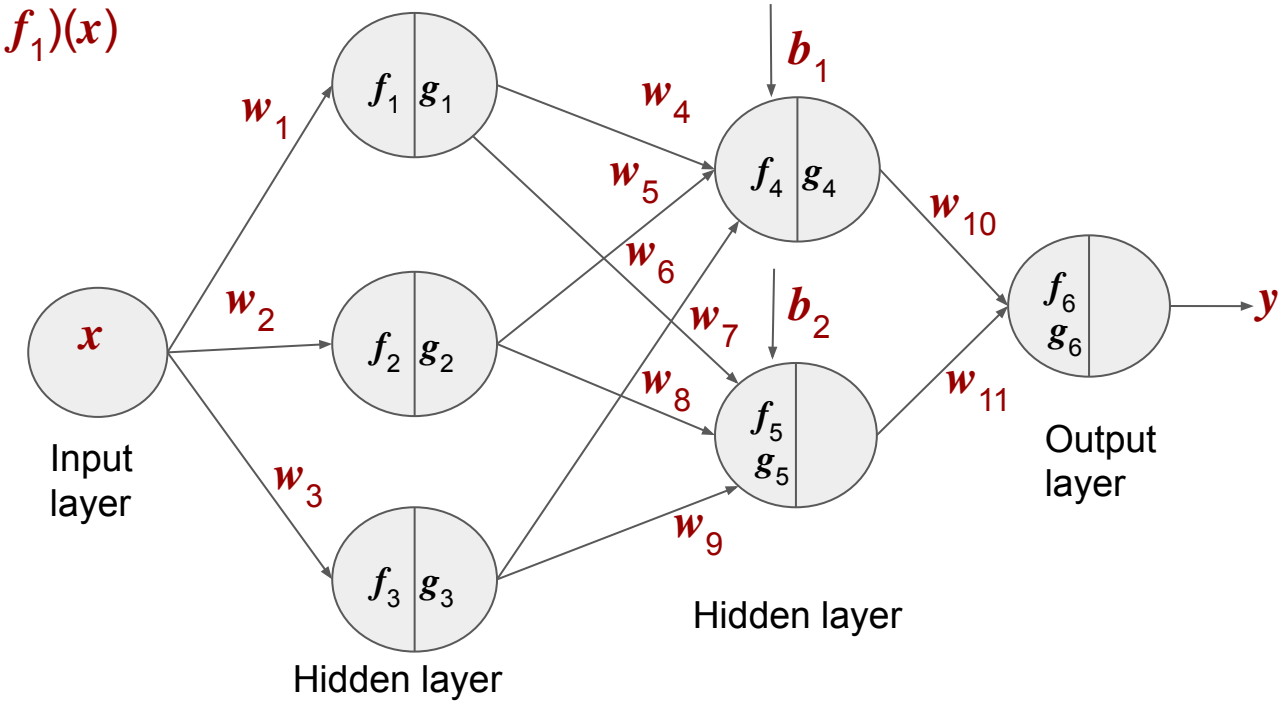
Artificial Neuron

Artificial Neural Network (ANN)

ANN is nothing but a composite of composite functions:

$(g_n \circ f_{n.....} g_2 \circ f_2 \circ g_1 \circ f_1)(x)$

Number of parameters: weights + bias (optional) + any variables in $g(.)$ (optional)



Parameter Vs Hyperparameter

Parameters: which NN will learn from data

- w, b, any variables in $g(.)$

Hyperparameters: which we need to decide based on our experience, intuition or error-trial such as

- number of layers
- number of neurons in each layer
- activation function
- epoch number
- learning rate

Performance Vs Parameters

In general, a model having more parameters performs better

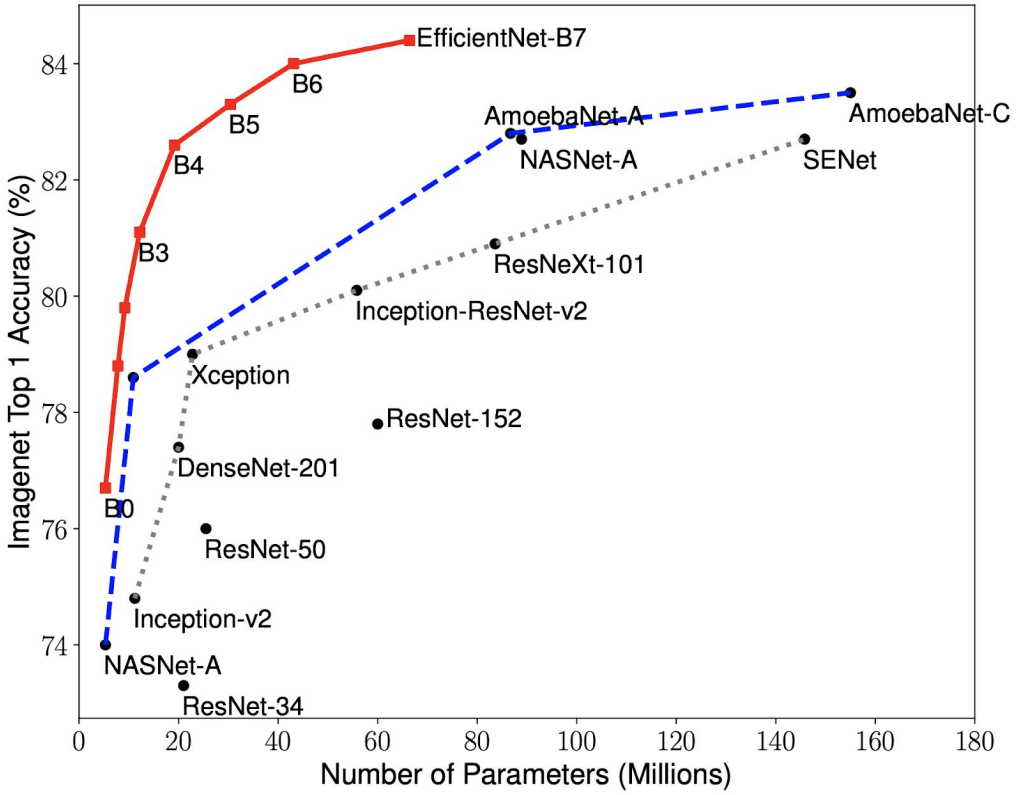


Image Source: Google Search Engine

Training set, Validation Set, Test Set

- **Training Set:** used for finding optimum parameters
- **Validation Set:** used for making decision i.e., finding optimum hyper-parameter
- **Test Set:** used for evaluating model performance

Be careful about:

- **Over-fitting:** neural network can be easily over-fitted to the training set, therefore, it cannot perform well for unseen data
- **Under-fitting:** if we cannot find appropriate hyper-parameter, neural network may not perform well even on the training set

Activation Functions

Depending on the range of output values we need to choose activation function.

LU (Linear Unit) family members such as ReLU, PReLU, ELU, etc., are quite popular as activation functions for the hidden layers.

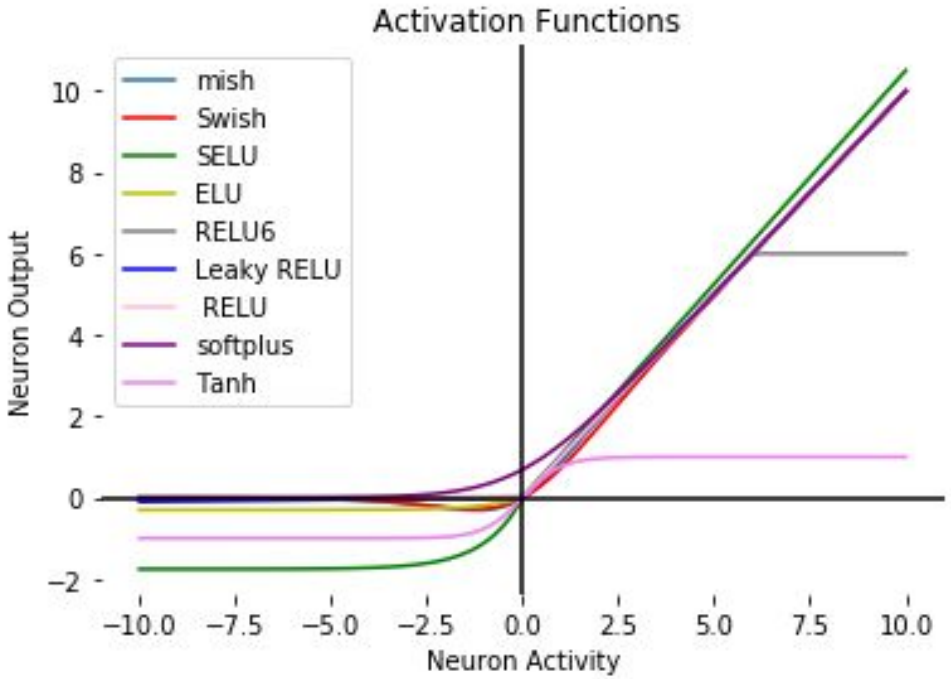


Image Source: Google Search Engine

Sigmoid Vs Softmax

In output layer, generally:

- Sigmoid is used for binary classification
- Softmax is used for multi-class classification

Both generate values in the range of 0-1.

Summation of softmax values is 1.

Summation of sigmoid values does not need to be 1.

$$sigmoid,\ y_i = \frac{e^{x_i}}{1 + e^{x_i}}$$

$$softmax,\ y_i = \frac{e^{x_i}}{\sum_{j=1}^{k} e^{x_j}}$$

Loss Functions

Different loss functions penalizes wrong predictions differently

Most popular losses:

- Regression: Mean-Squared Error (MSE)
- Classification: Log-loss or Cross-Entropy

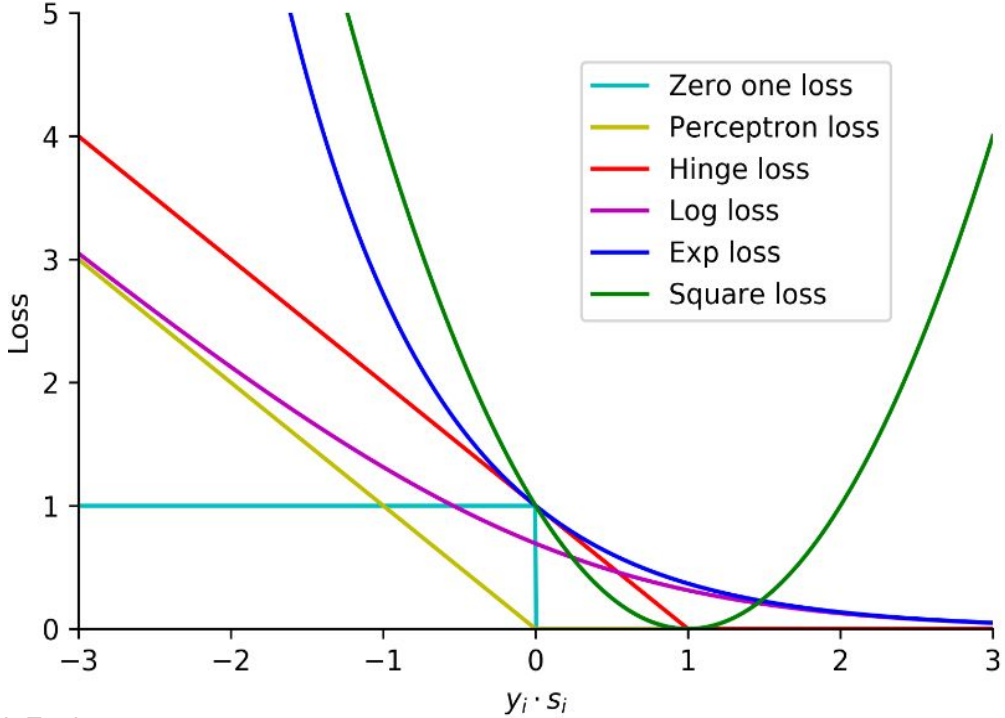


Image Source: Google Search Engine

Training Neural Network

During training we try to find the parameter set for which loss is minimum.
Starting at random values, we update parameters by gradient descent algorithm

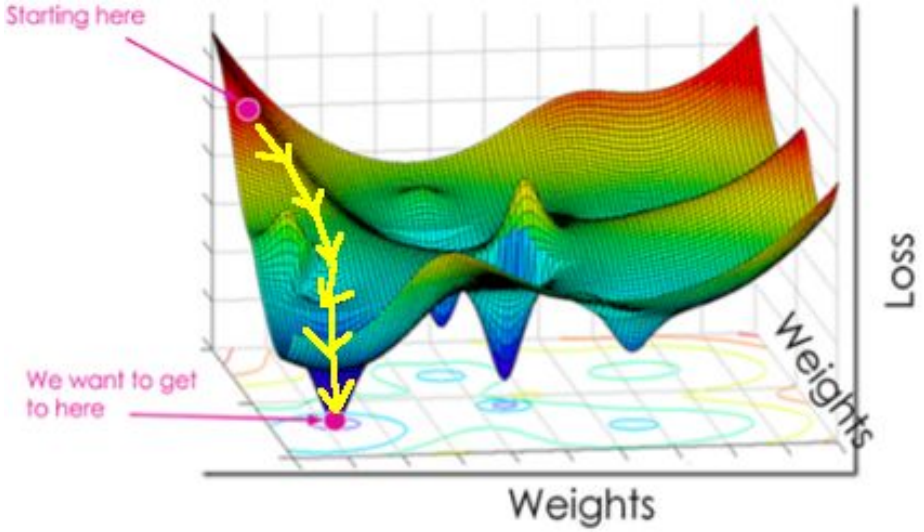


Image Source: Google Search Engine

Gradient Descent Algorithm

- Initialize parameters randomly
- Estimate outputs for inputs in a forward pass
- Estimate loss
- Estimate gradient of the loss function with respect to all parameters
- Move parameters in the opposite direction of gradient

$$w = w - \alpha \frac{d}{dw} J(w)$$

- Repeat these steps until loss is minimum

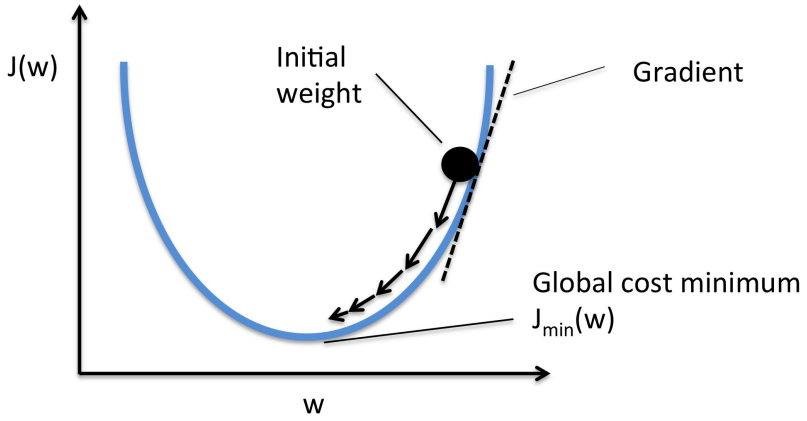
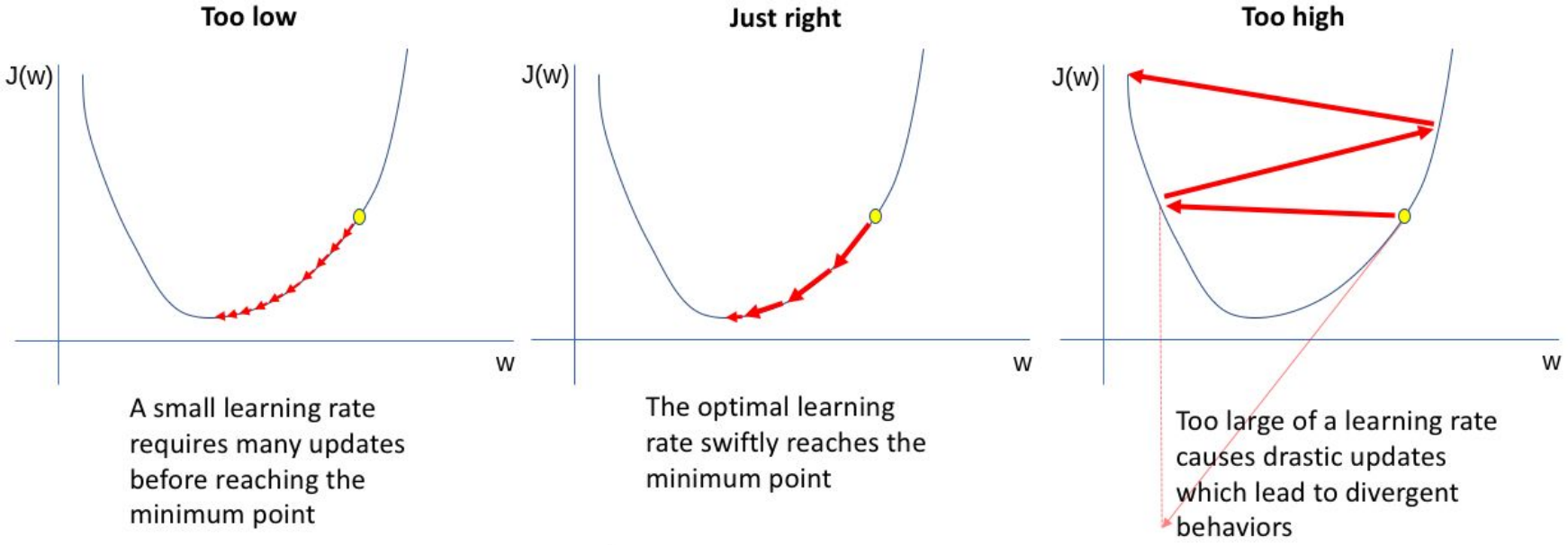


Image Source: Google Search Engine

Effect of Learning Rate



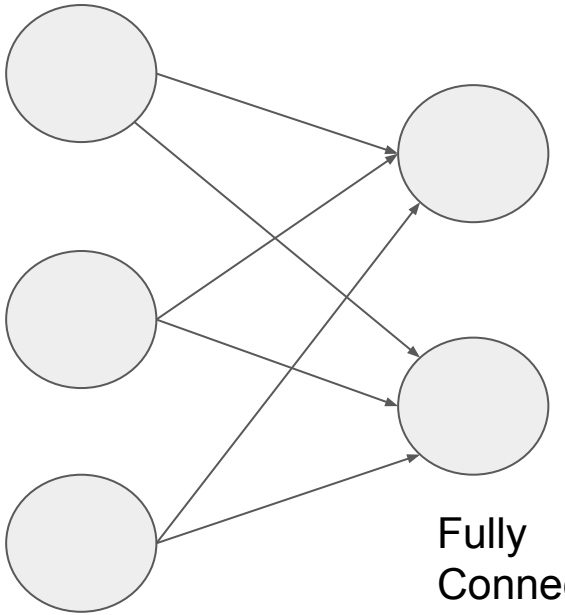
$$w = w - \alpha \frac{d}{dw} J(w)$$

Image Source: Google Search Engine

Fully Connected Layer

A fully connected layer connects every neuron in one layer to every neuron in its previous layer.

An image need to be turned into a vector before feeding into an fully connected layer.



Fully Connected Neural Network (FCNN)

FCNN consists of a series of fully connected layers

It is also known as multi-layer perceptron (MLP).

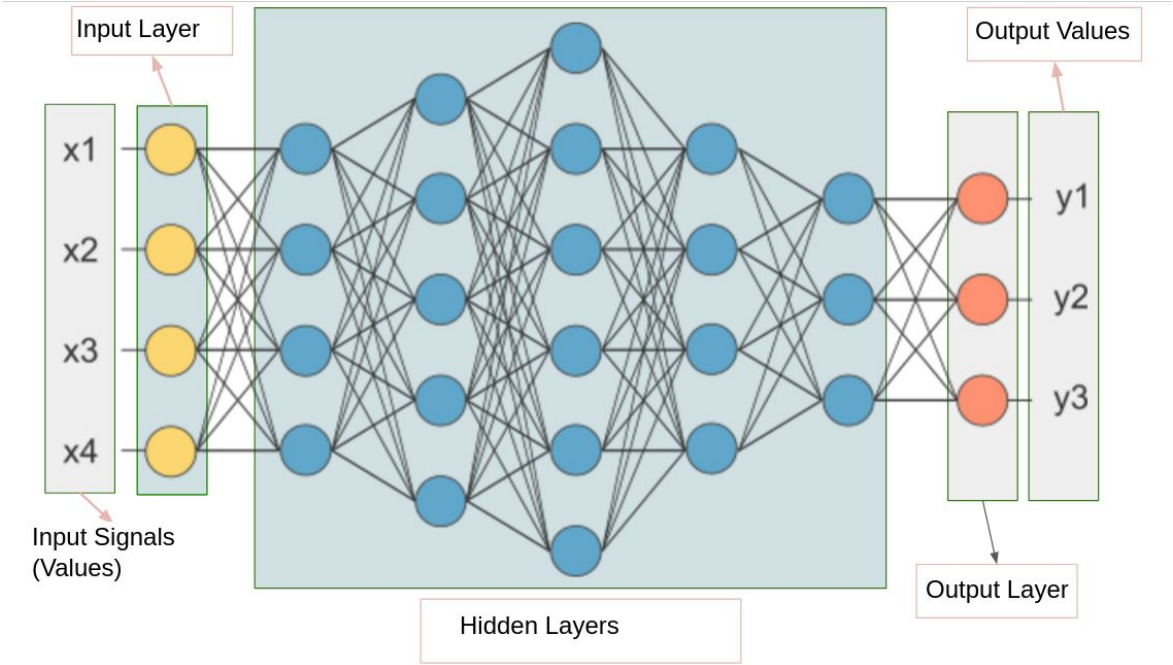
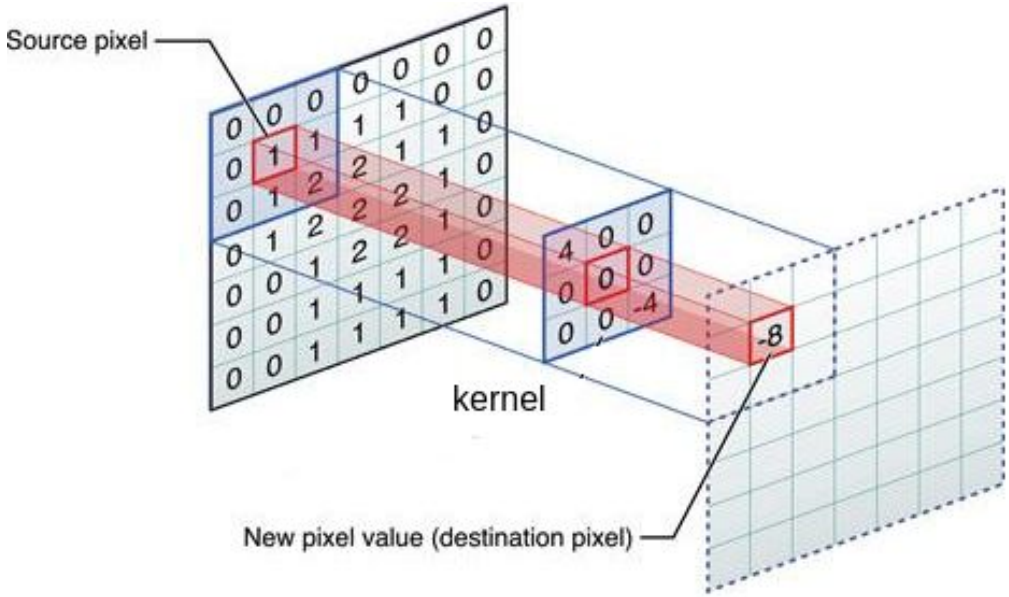


Image Source: Google Search Engine

Conventional Convolution

Convolution layer adds each element of a matrix to its local neighbors, weighted by the kernel.



Source pixel: 0

Destination pixel: 4x0 + 0x0 + 0x0 + 0x0 + 1x0 + 2x0 + 0x0 + 1x0 + (-4)x2 = -8

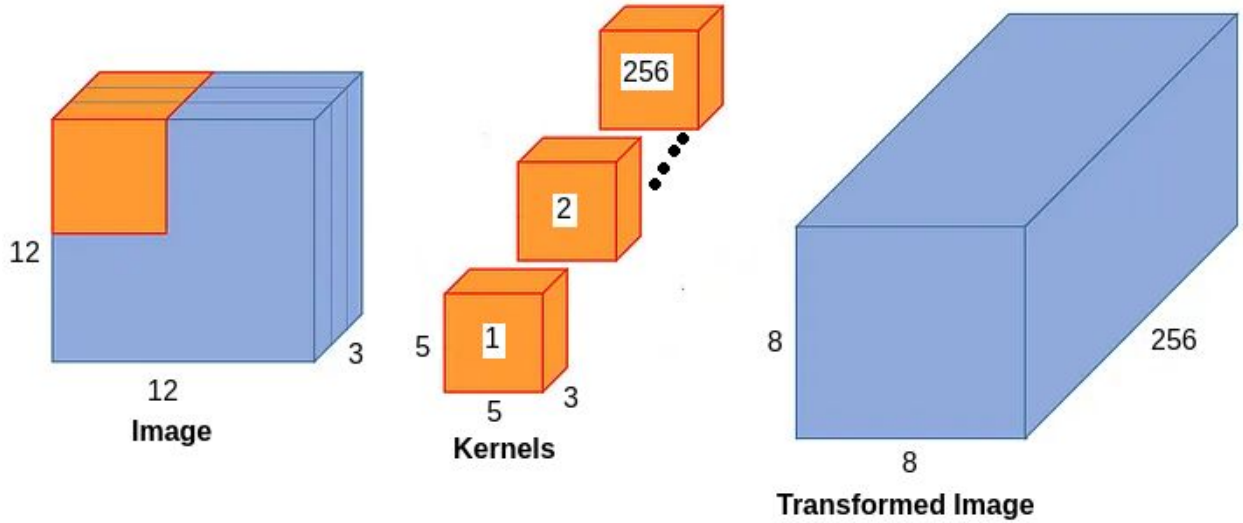
Total number of multiplication for one pixel = kernel (height x width x channel)

Image Source: Google Search Engine

Convolution Layer

Total number of multiplications will depend on:

- kernel size
- number of kernels
- stride
- whether zero padding is used or not.



For each pixel & 1 kernel: 5x5x3x1 = 75

For each pixel & 256 kernels: 5x5x3x256 = 19200

For 8x8 pixels & 256 kernels: 8x8x5x5x3x256 = 1228800

Image Source: Google Search Engine

Pooling Layer

A pooling layer divides the input feature map into a set of non-overlapping regions to:

- reduce the spatial dimensions (i.e., the width and height) of the feature maps
- preserve the depth (i.e., the number of channels)

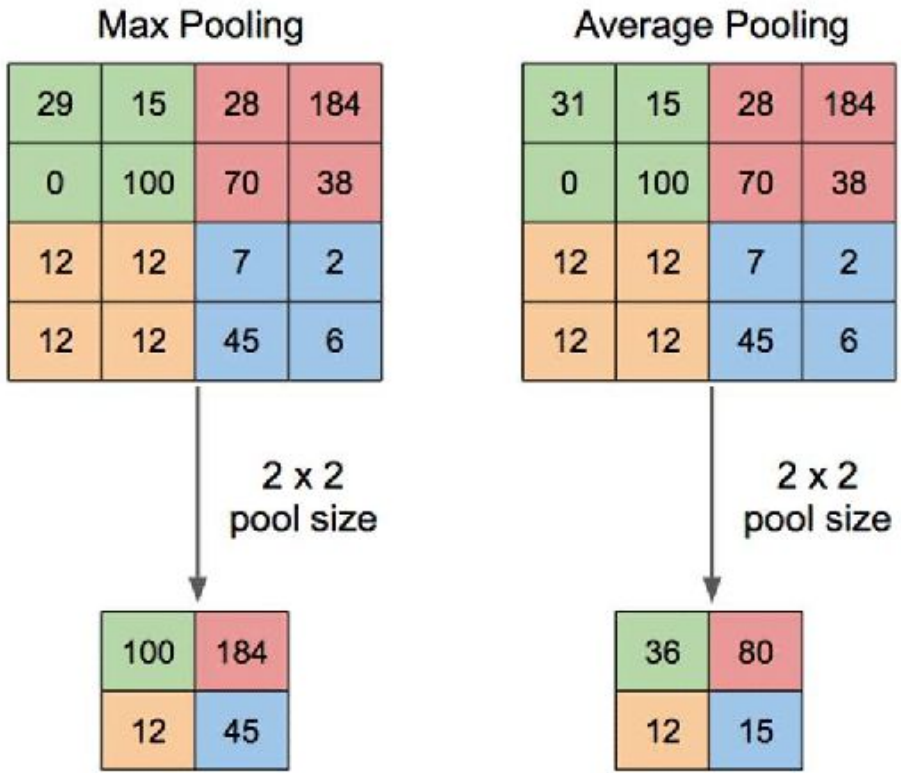


Image Source: Google Search Engine

Convolutional Neural Network (CNN)

CNN mainly consists of a series of convolutional layers. For classification, fully connected layers are added after convolutional layers.

Popular CNNs:

* VGG
* InceptionNet
* ResNet
* EfficientNet
* MobileNet
* XceptionNet
* DenseNet

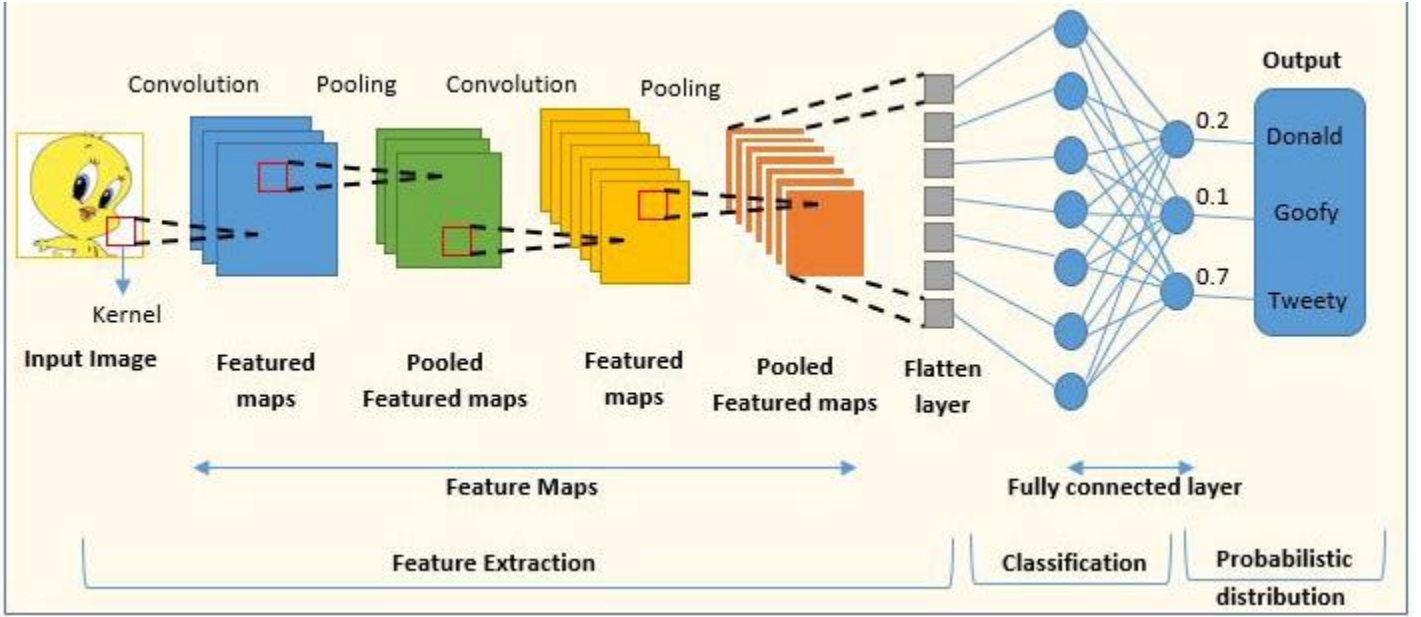


Image Source: Google Search

Model	Size (MB)	Top-1 Accuracy	Top-5 Accuracy	Parameters	Depth	Time (ms) per inference step (CPU)	Time (ms) per inference step (GPU)
Xception	88	79.0%	94.5%	22.9M	81	109.4	8.1
VGG16	528	71.3%	90.1%	138.4M	16	69.5	4.2
VGG19	549	71.3%	90.0%	143.7M	19	84.8	4.4
ResNet50	98	74.9%	92.1%	25.6M	107	58.2	4.6
ResNet50V2	98	76.0%	93.0%	25.6M	103	45.6	4.4
ResNet101	171	76.4%	92.8%	44.7M	209	89.6	5.2
ResNet101V2	171	77.2%	93.8%	44.7M	205	72.7	5.4
ResNet152	232	76.6%	93.1%	60.4M	311	127.4	6.5
ResNet152V2	232	78.0%	94.2%	60.4M	307	107.5	6.6
InceptionV3	92	77.9%	93.7%	23.9M	189	42.2	6.9
InceptionResNetV2	215	80.3%	95.3%	55.9M	449	130.2	10.0
MobileNet	16	70.4%	89.5%	4.3M	55	22.6	3.4
MobileNetV2	14	71.3%	90.1%	3.5M	105	25.9	3.8
DenseNet121	33	75.0%	92.3%	8.1M	242	77.1	5.4
DenseNet169	57	76.2%	93.2%	14.3M	338	96.4	6.3

VGG16

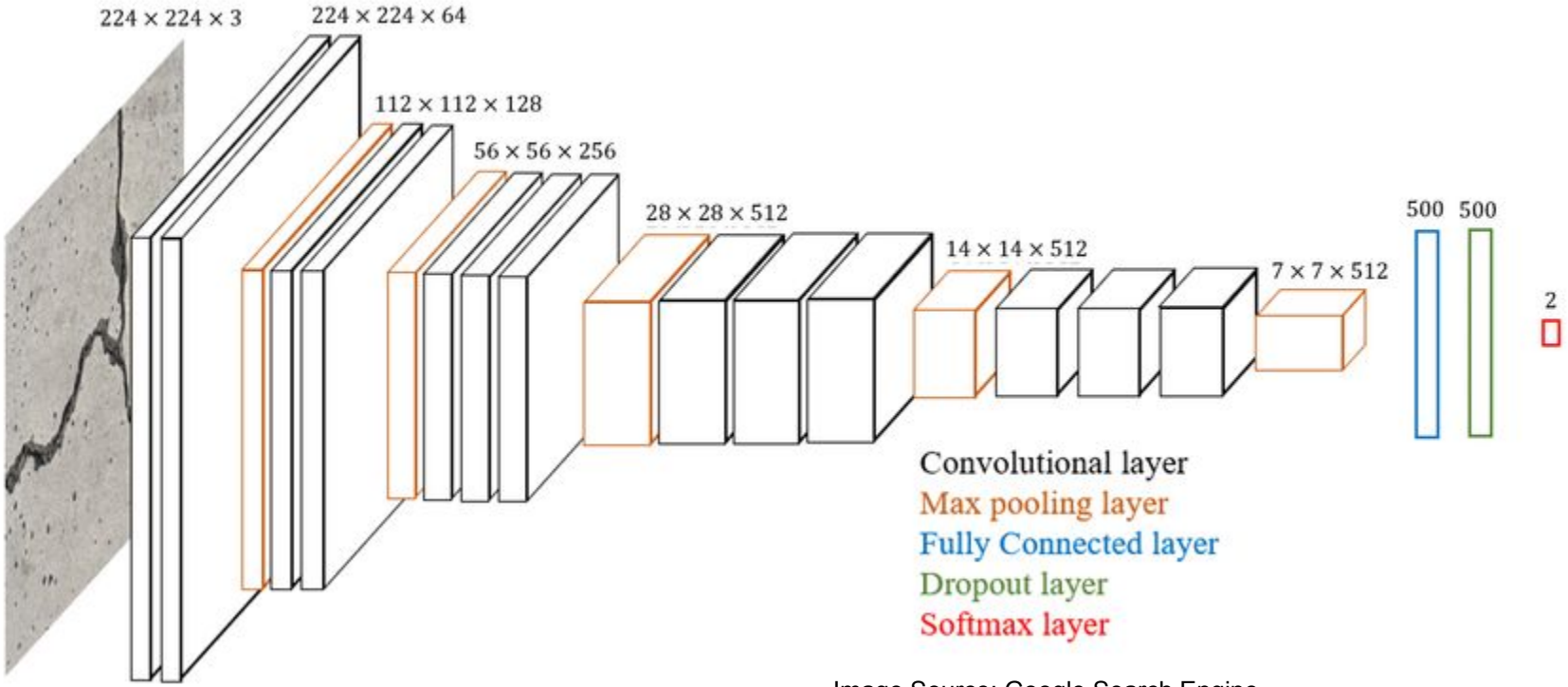
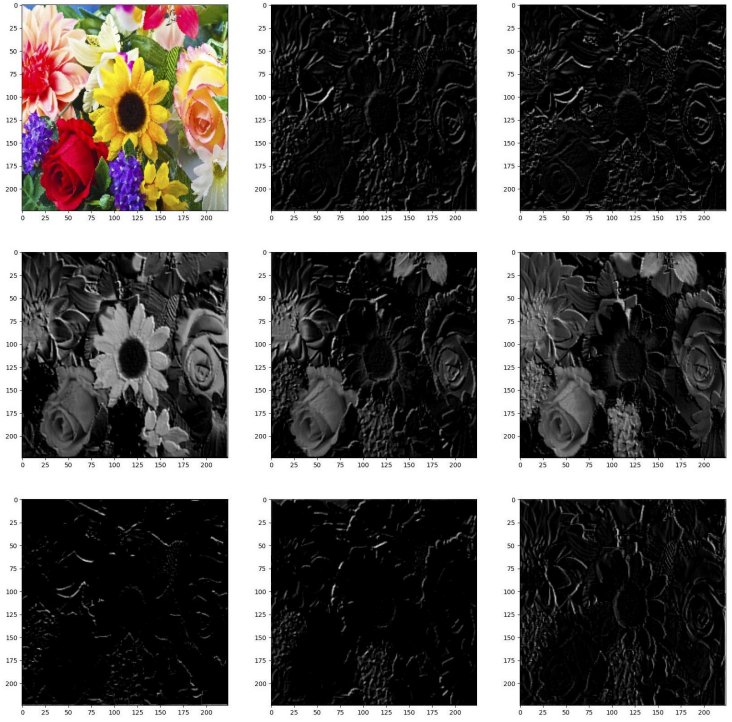
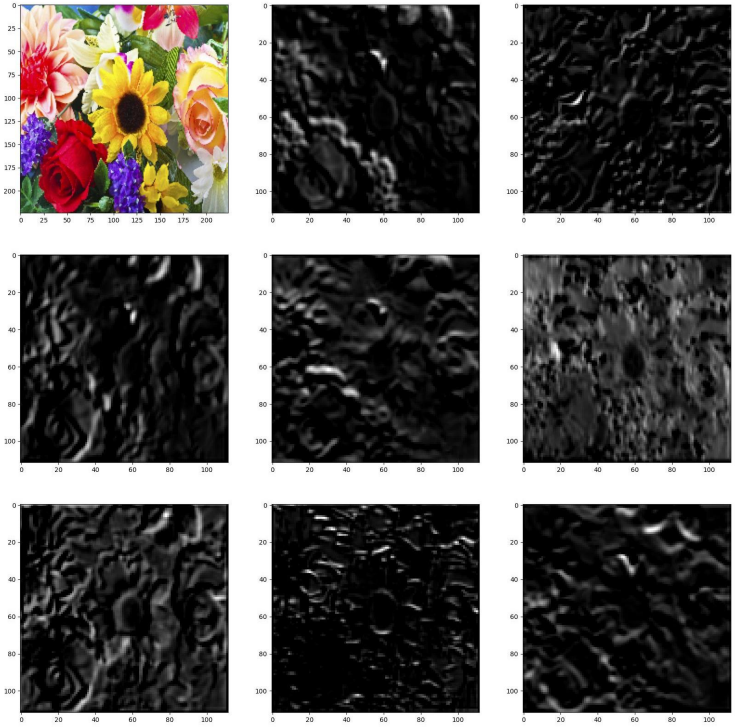


Image Source: Google Search Engine

What VGG16 Sees



Vgg16_Block1_Conv1



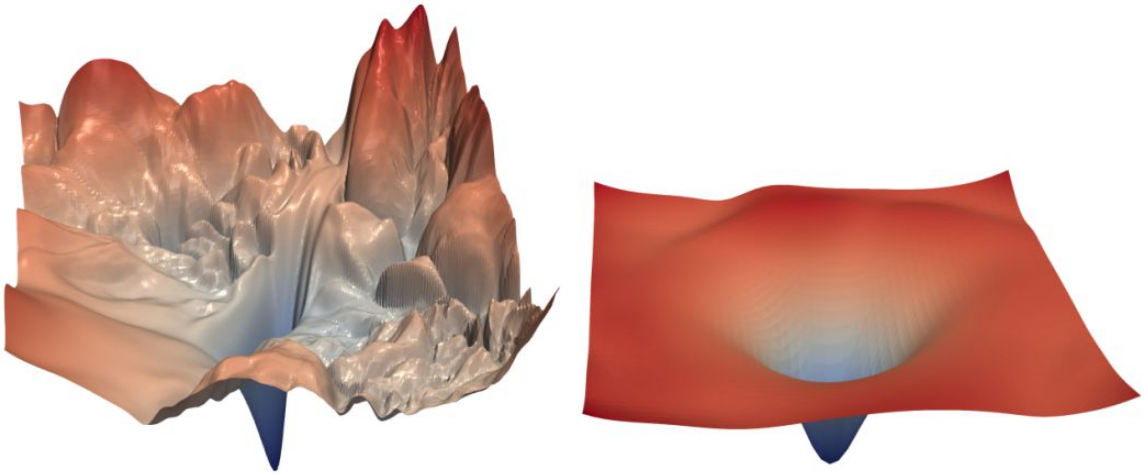
Vgg16_Block2_Conv2

Skip Connections

Skip connections allow CNNs to bypass some layers and connect directly to deeper or shallower ones.

Skip connections lead to:

- faster convergence
- vanishing gradient problem prevention
- loss of information prevention



(a) without skip connections (b) with skip connections

Image Source: Google Search Engine

ResNet

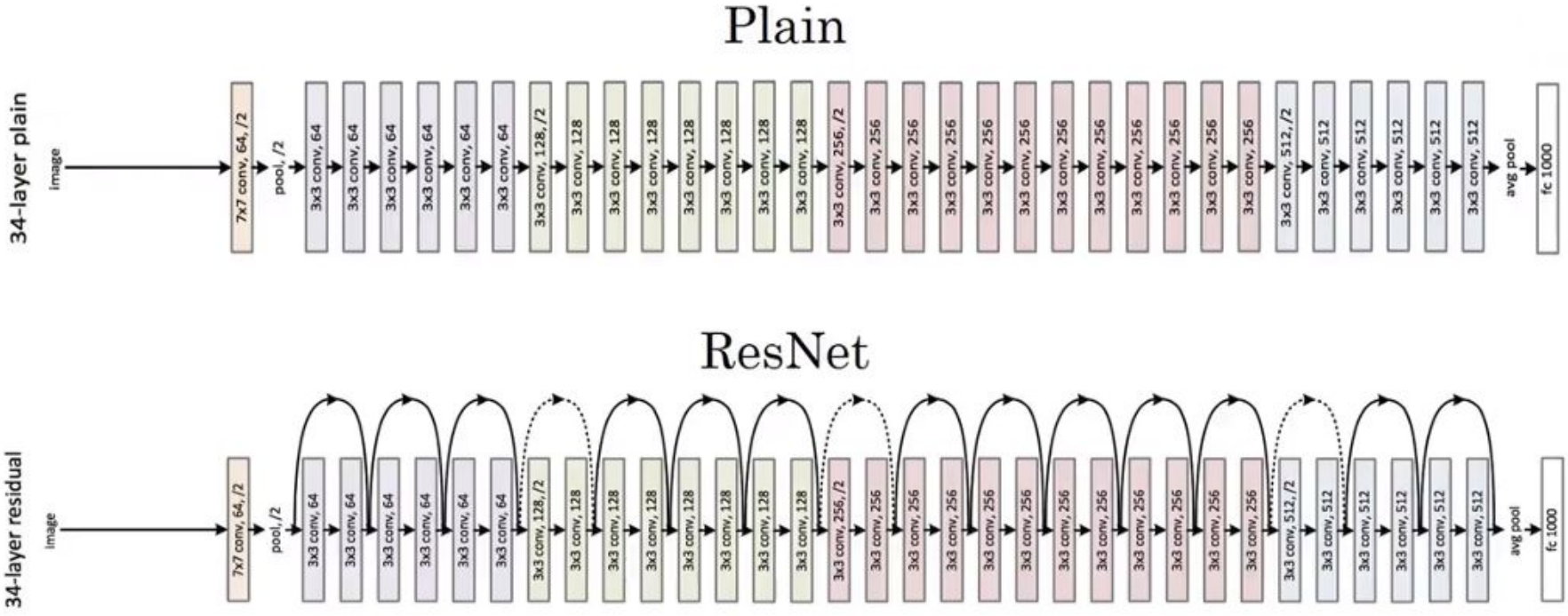


Image Source: Google Search Engine

ResNet Block

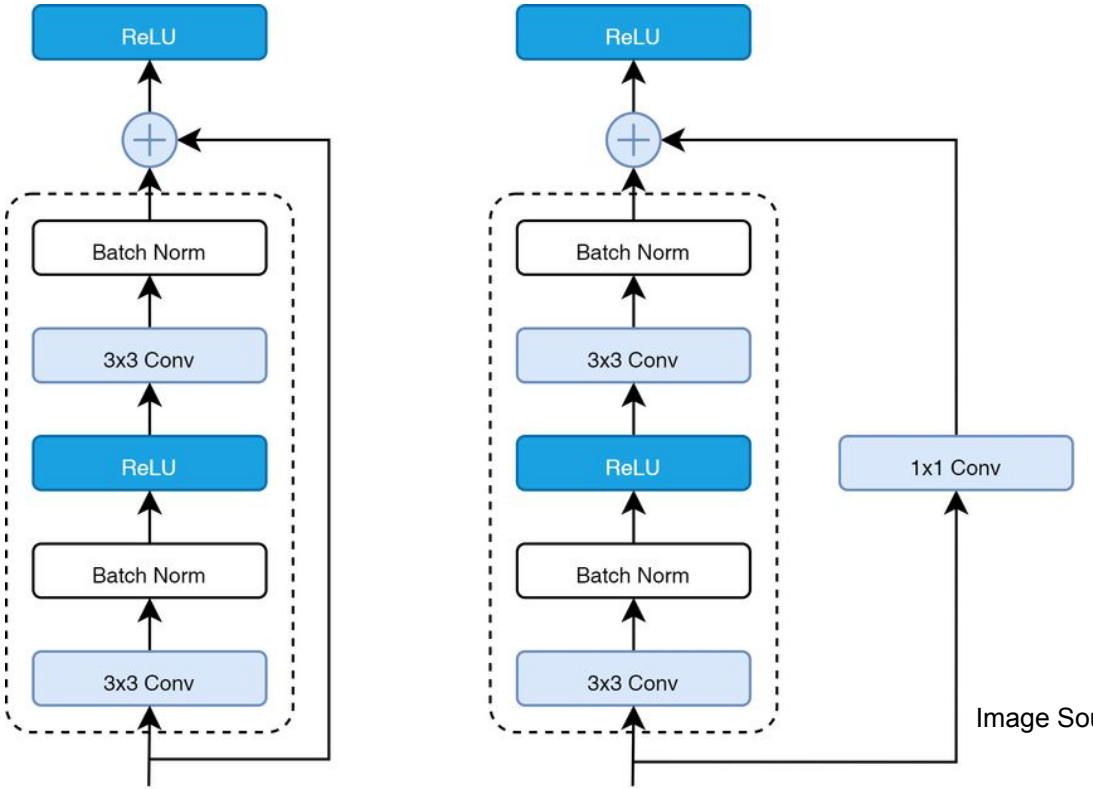
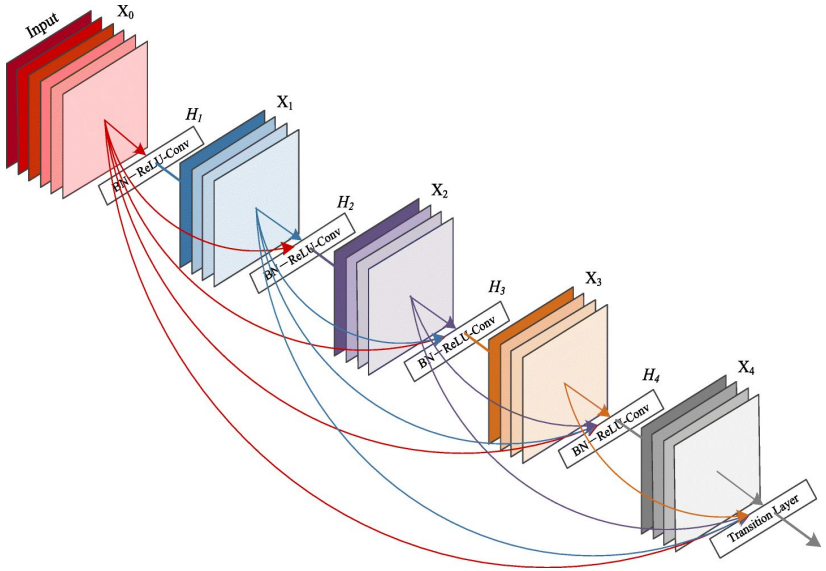


Image Source: Google Search Engine

DenseNet

Skip connections are used for having feature reusability.



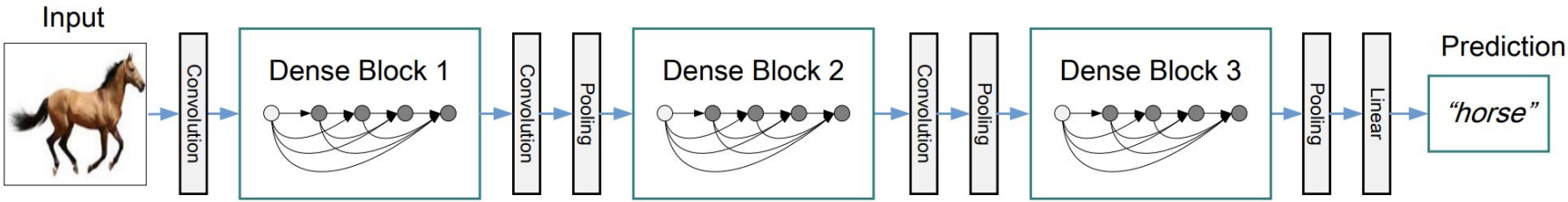


Image Source: Google Search Engine

U-Net

UNet is the most popular network of medical image segmentation.

Skip connections are used for use *fine-grained* details learned in the encoder part to construct an image in the decoder part.

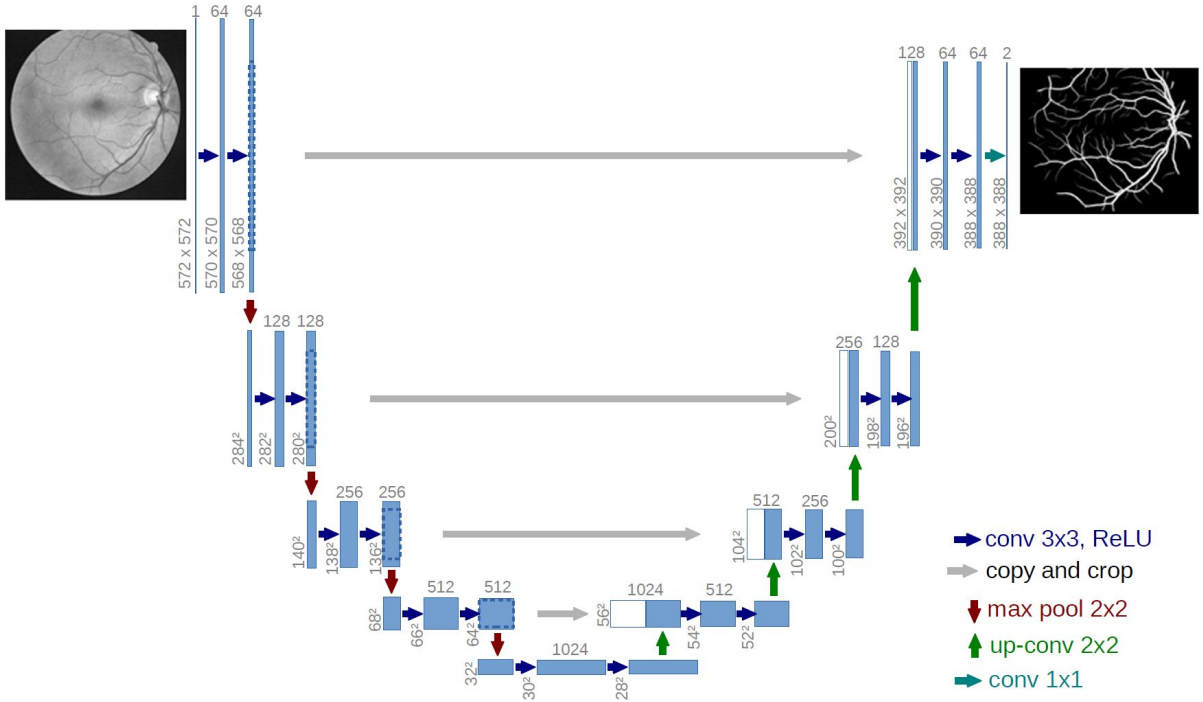


Image Source: Google Search Engine

Point-Wise convolution

1 x 1 kernel is used for point-wise convolution.

Advantage: Increase or reduce the depth (or number of channels) of an image.

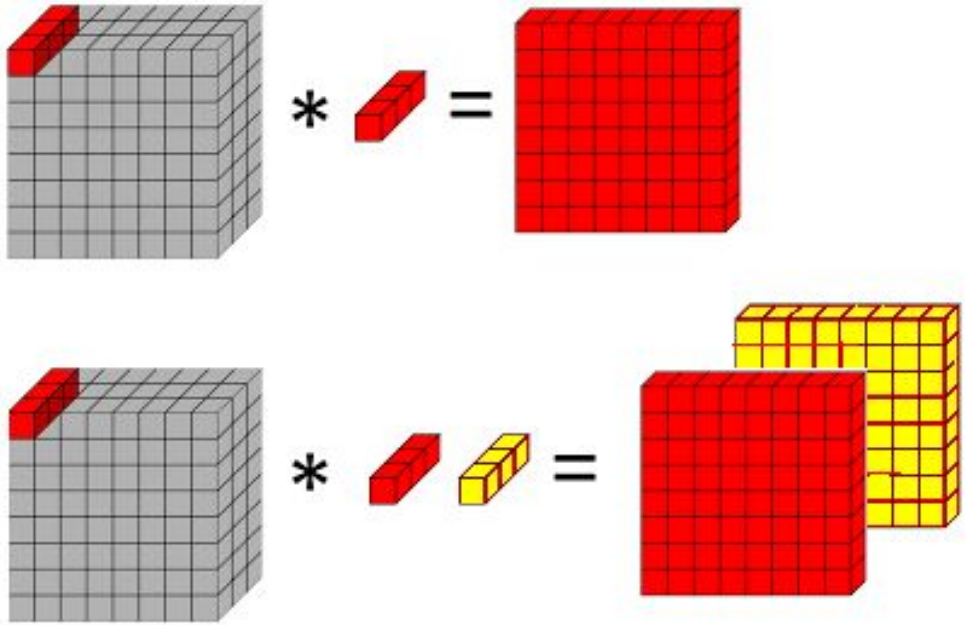


Image Source: Google Search Engine (Modified)

Spatial Separable Convolution

A spatial separable convolution simply divides a kernel into two, smaller kernels.

Advantage: Less parameters, less memory and less computations than regular convolution

Limitations: Not all kernels can be spatially “separated”

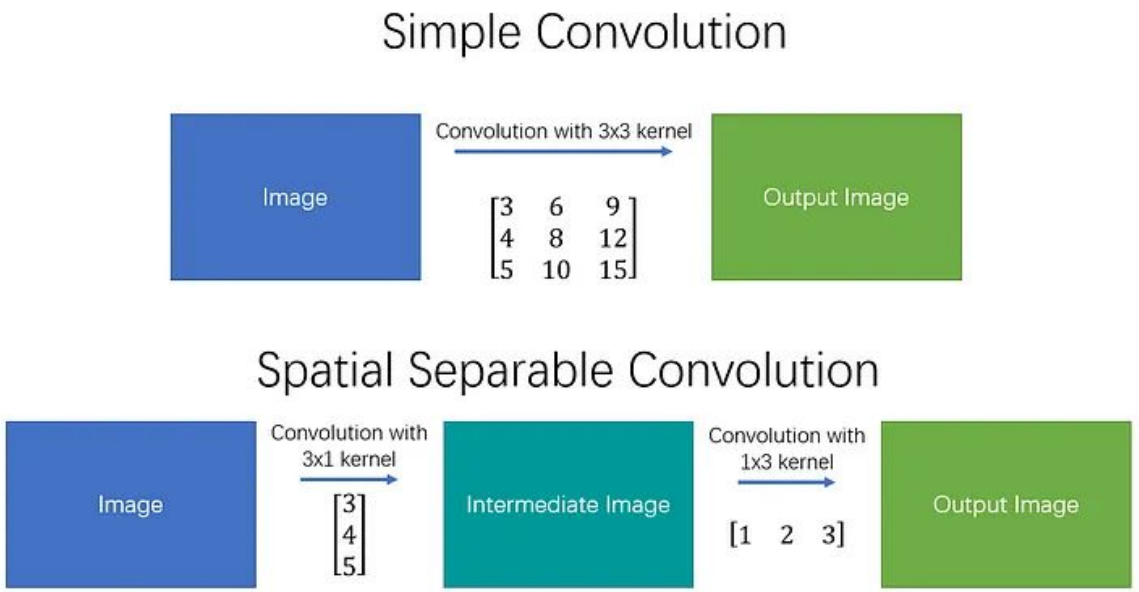


Image Source: Google Search Engine

InceptionNet

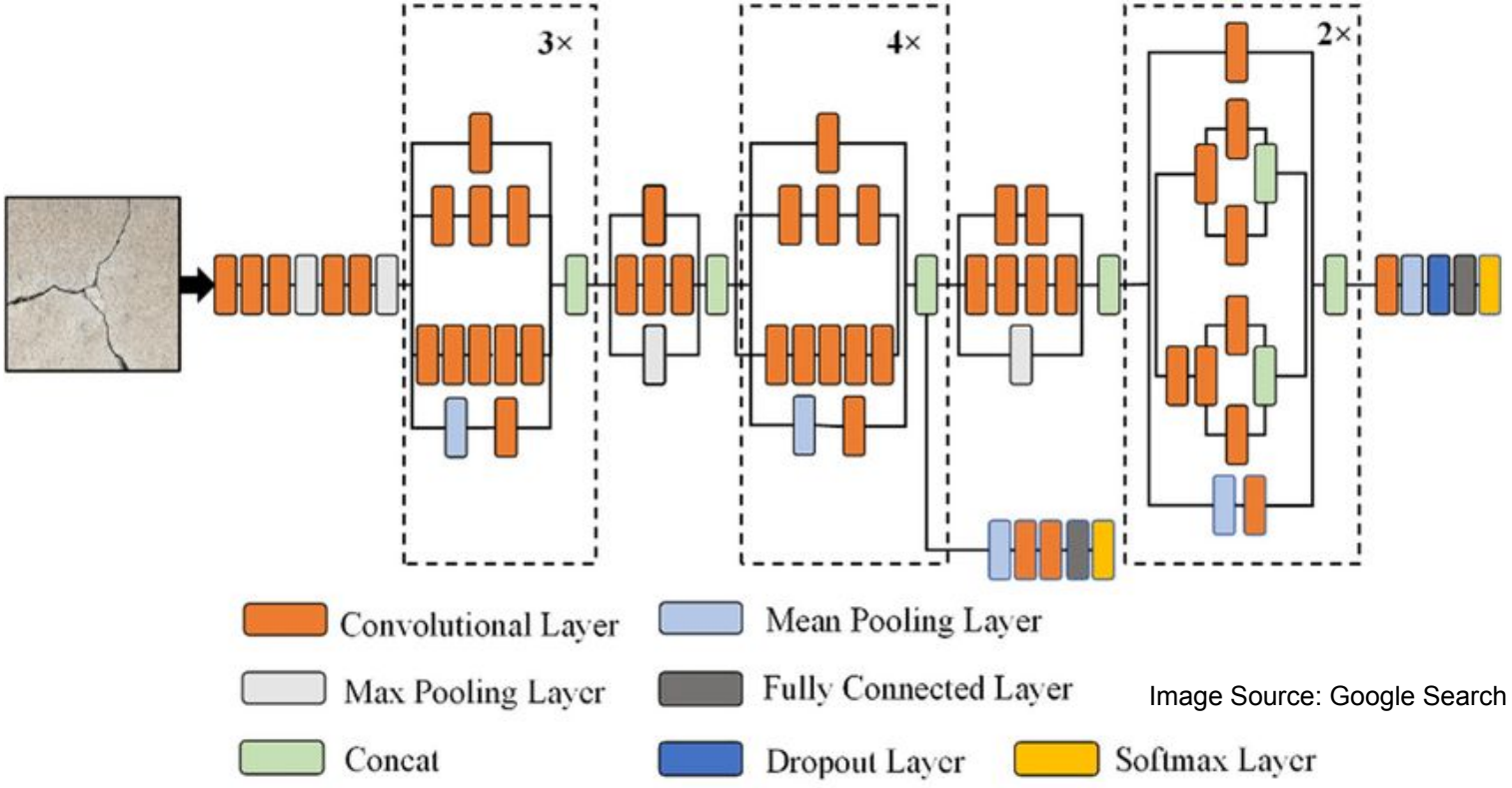


Image Source: Google Search Engine

Inception Modules

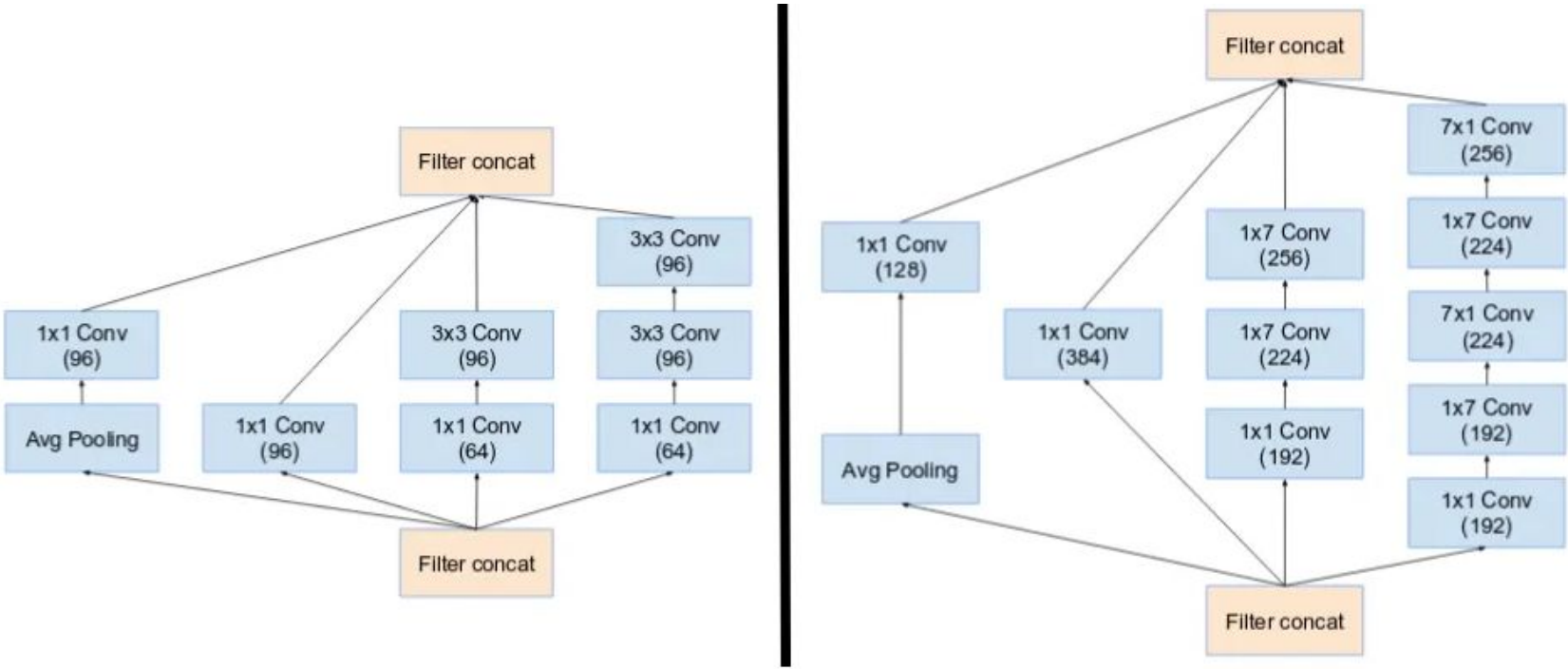


Image Source: Google Search Engine

Depthwise Separable Convolution

Each input channel is convolved with a different kernel (called a depthwise kernel).

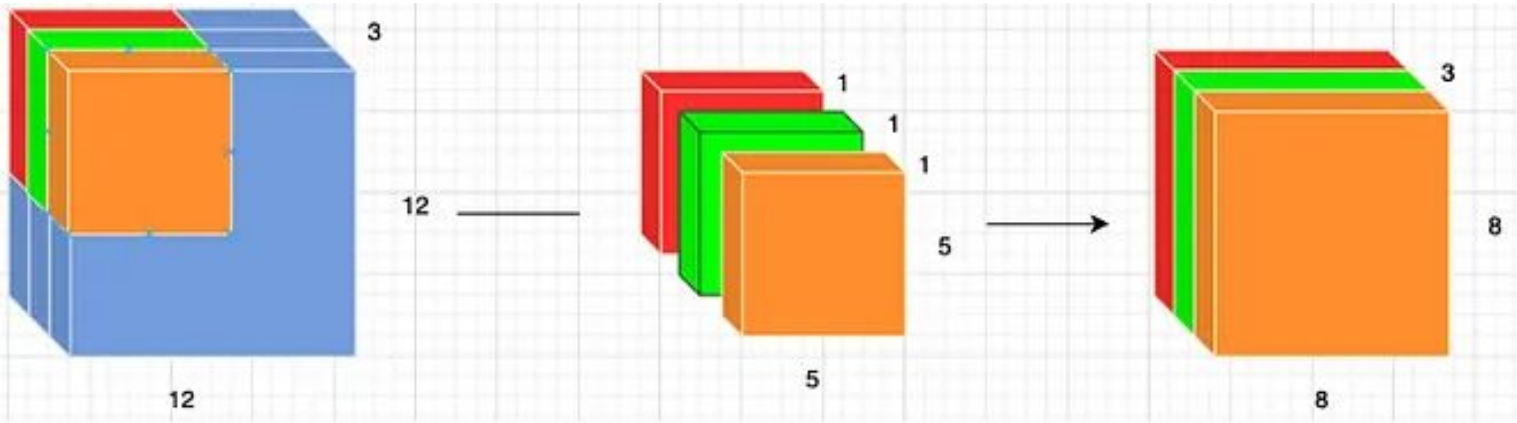


Image Source: Google Search Engine

Depthwise Separable Convolution

Generally, a Depthwise Convolution follows a Pointwise Convolution.

For standard Convolution, number of multiplications: 8x8x5x5x3x256 = 1228800

For Depthwise Separable Convolution, number of multiplications: 8x8x5x5x3 + 256x1x1x3x8x8 = 53952

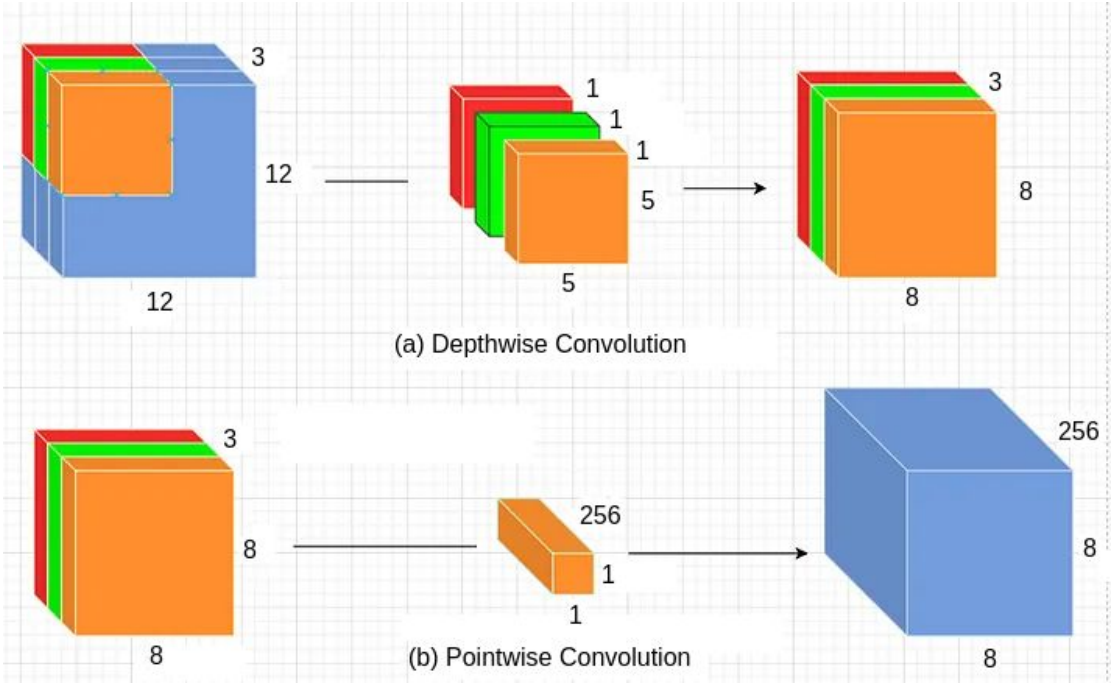


Image Source: Google Search Engine

MobileNet V2

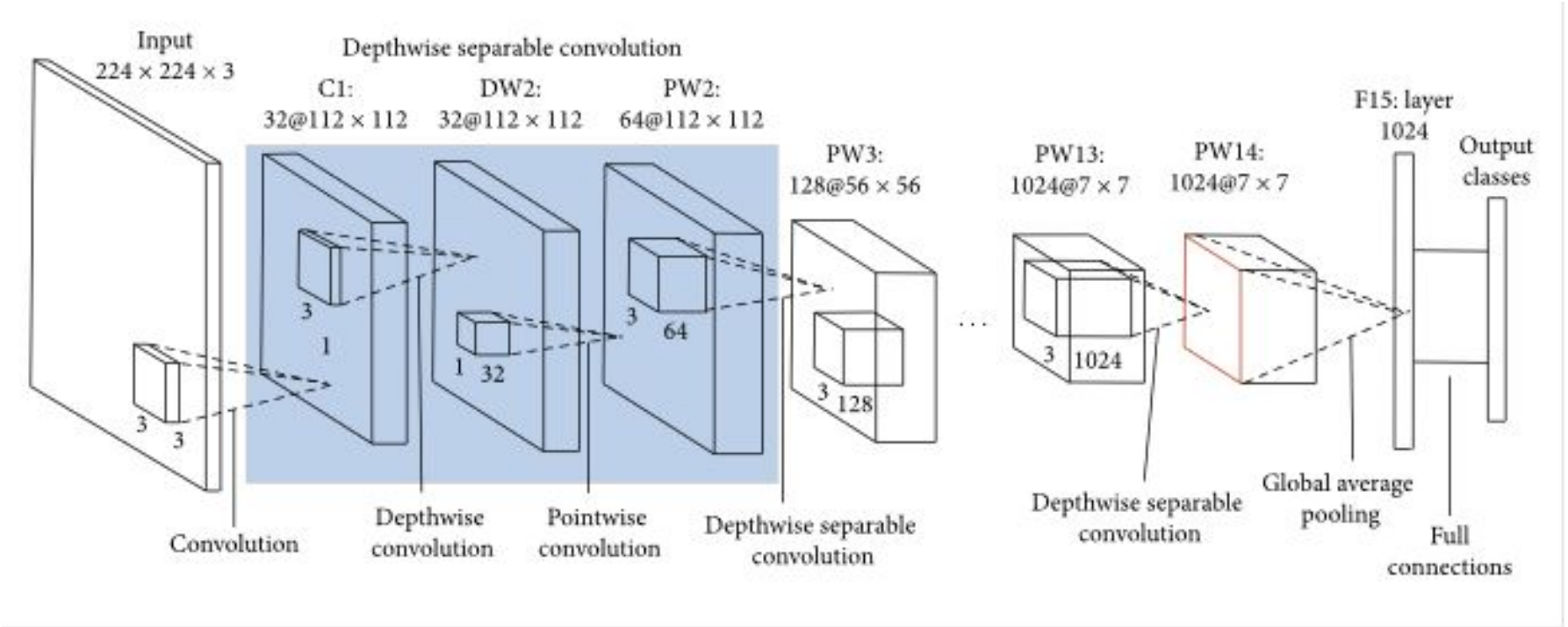


Image Source: Google Search Engine

Modified Depthwise Separable Convolution

In a Modified Depthwise Separable Convolution is the **pointwise convolution followed by a depthwise convolution**.

Xception uses Modified Depthwise Separable Convolution.

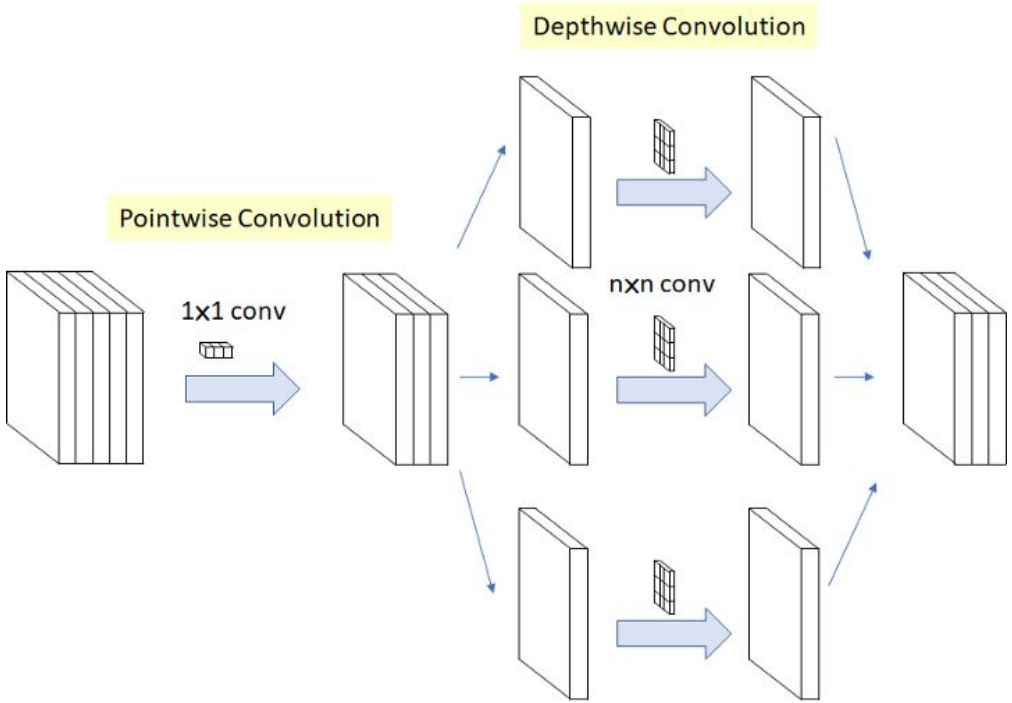


Image Source: Google Search Engine

Xception (Extreme version of Inception)

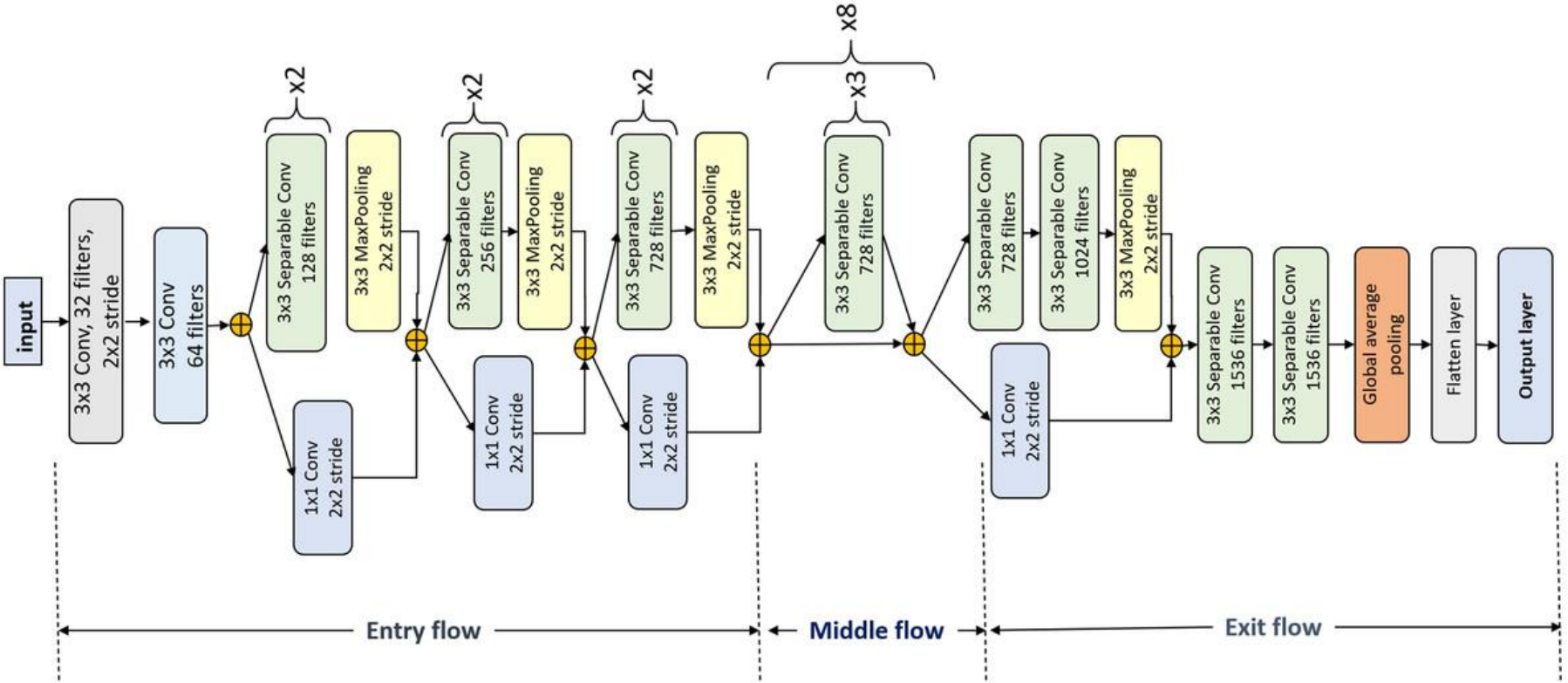


Image Source: Google Search Engine

Transfer Learning (TL)

By TL knowledge learned from a task is re-used in order to boost performance on a related task.

By fine-tuning weights of a pre-trained model are trained on new data.

Generally fine-tuning is started from the conv layers closer to the fully connected layers.

Depending on the amount of data layers need to be fine-tuned.

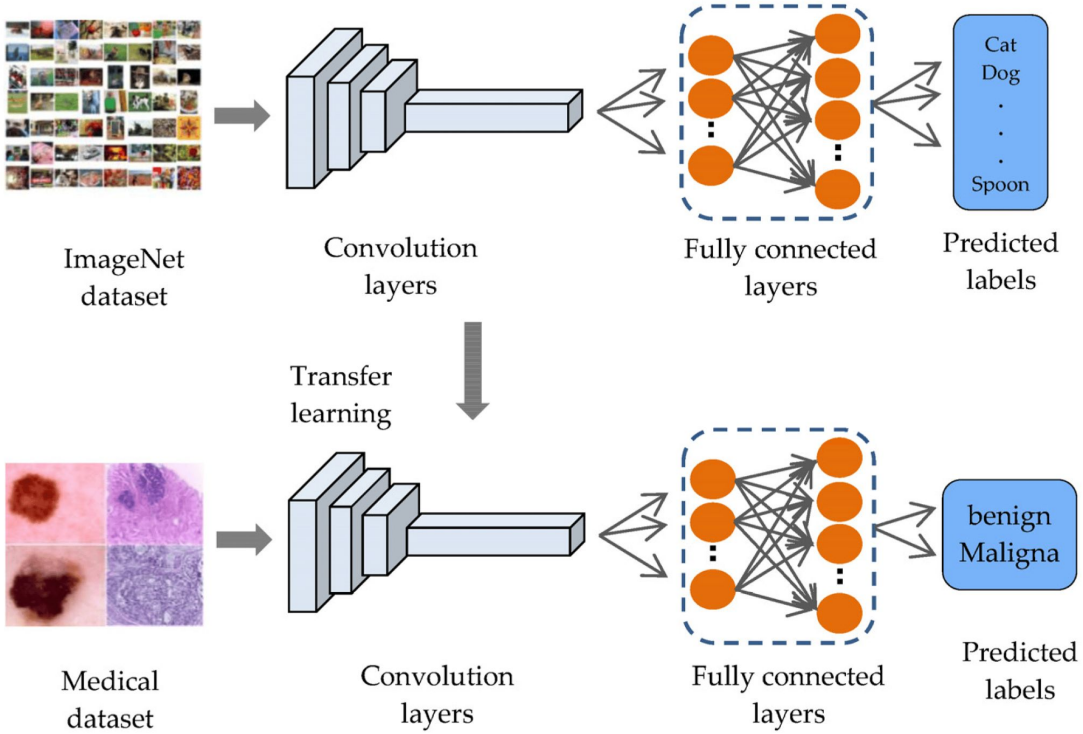
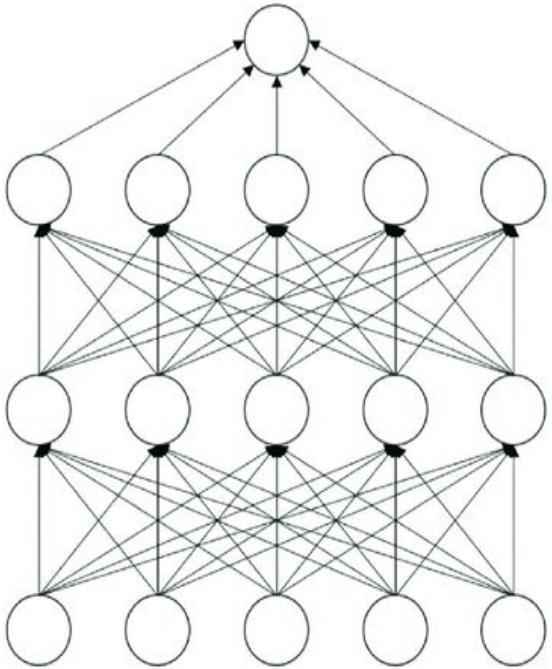


Image Source: Google Search

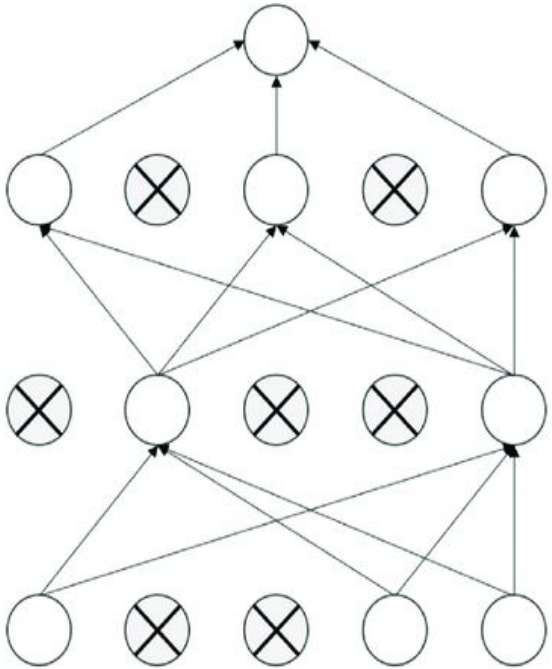
Dropout Layer

In dropout layer:

- individual nodes are excluded in various training runs
- prevents overfitting



Standard Neural Net



After applying dropout

Image Source: Google Search Engine

Vision Transformer (ViT)

- Transformer first proposed for natural language processing in 2017
- ChatGPT is transformer-based system
- In computer vision, it was successfully implemented in 2020.

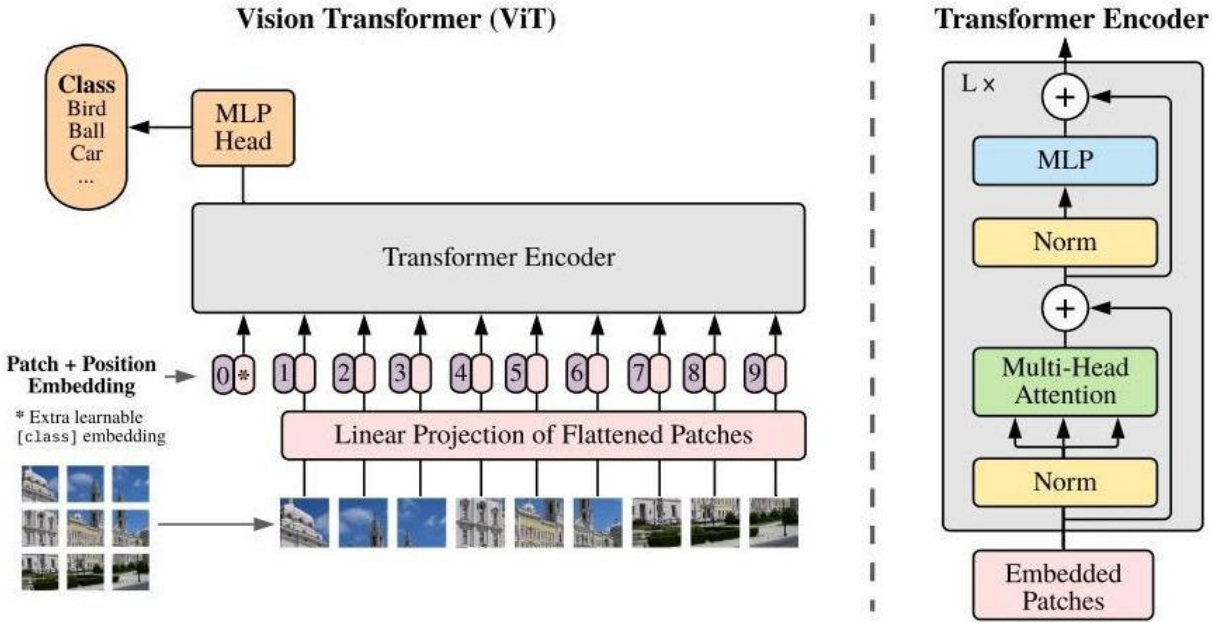


Image Source: Google Search Engine

Transformer is Very Data Hungry Model

- We need huge amount of data to get good performance from ViT
- In small amount of data, it performed worse than CNNs.

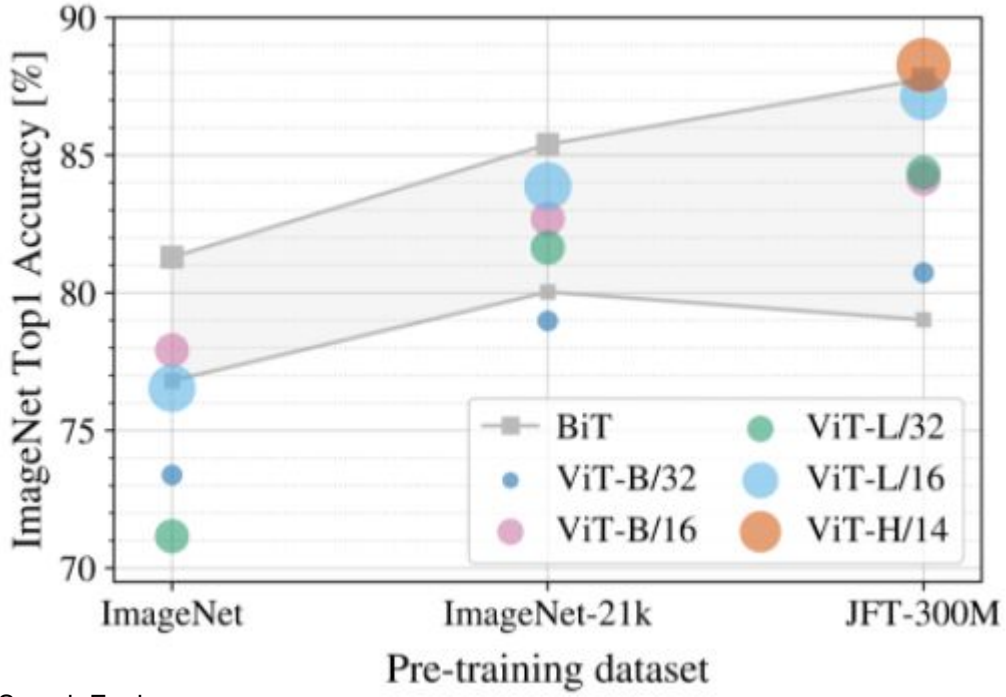


Image Source: Google Search Engine

Types of Training

1. Supervised Training
   - Labeled pairs, i.e., (x_1, y_1), (x_2, y_2), …, (x_n, y_n), are used for training
2. Semi-Supervised Training
   - Little amount of labeled pairs, i.e., (x_1, y_1), (x_2, y_2), …, (x_n, y_n), pairs and a lot of unlabeled data(x_{n+1}, x_{n+2}, x_{n+3},... x_{n+m}) are used for training
3. Self-Supervised Training
   - $((x_1, x_1), 1)$, $((x_1, x_2), 0)$, …, $((x_n, x_n), 1)$, $((x_n, x_1), 0)$, pairs are used for training
4. Unsupervised Training
   - (x_1, x_1), (x_2, x_2), …, (x_n, x_n), pairs can be used for training

Semi-Supervised Training

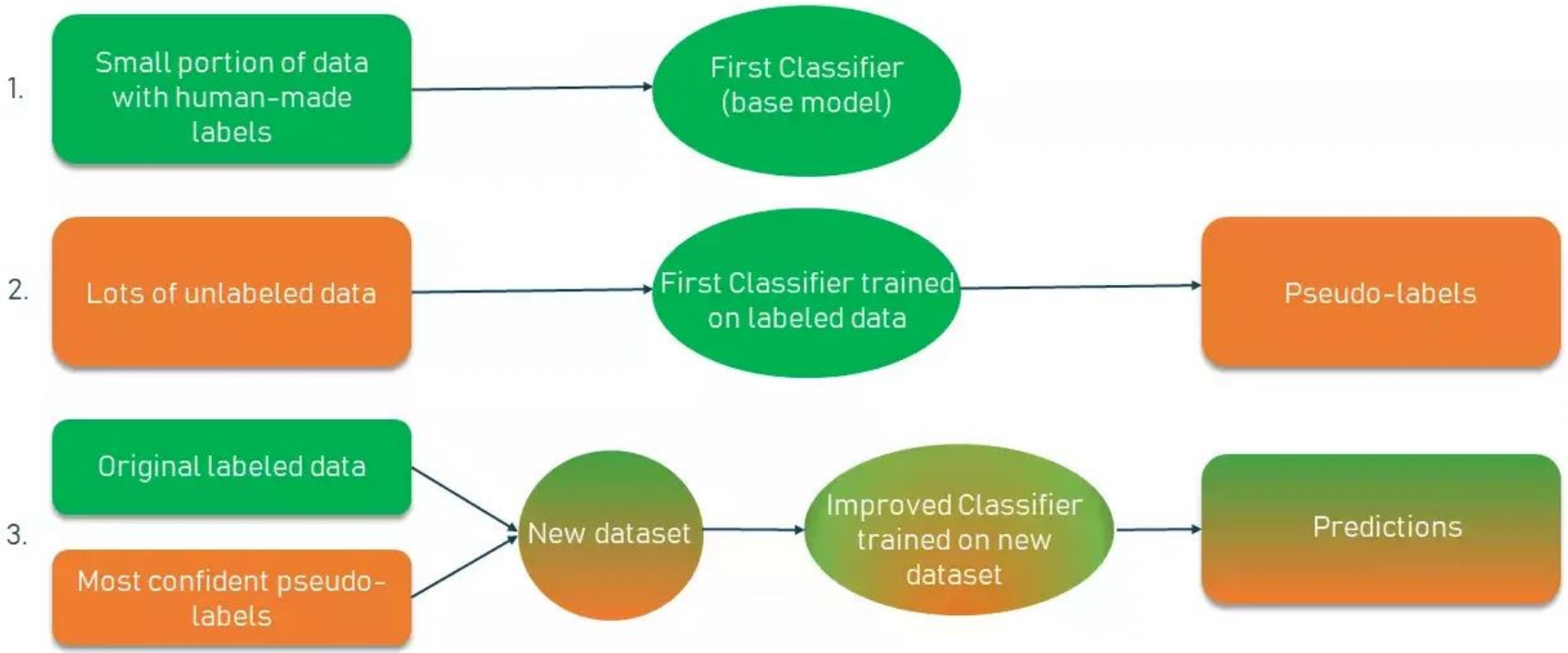


Image Source: Google Search Engine

Encoder-Decoder Architecture

This kind of architecture can be used for:

- Synthesizing images
- Denoising images
- Segmentation
- Unsupervised training

When input images and output images are the same, we call model autoencoder.

Training an autoencoder is an example of unsupervised training.

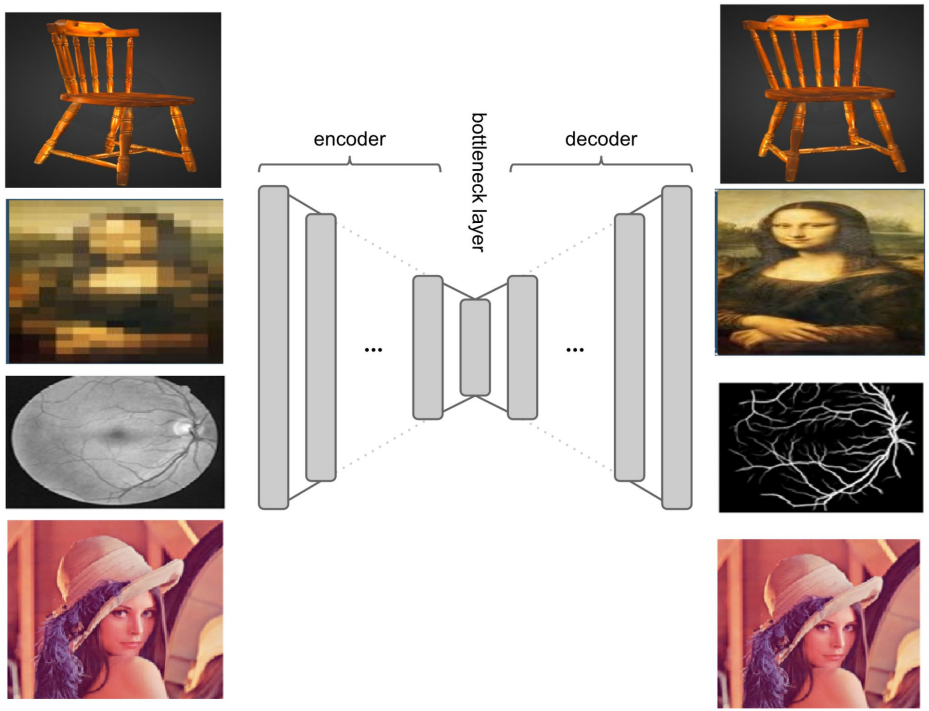
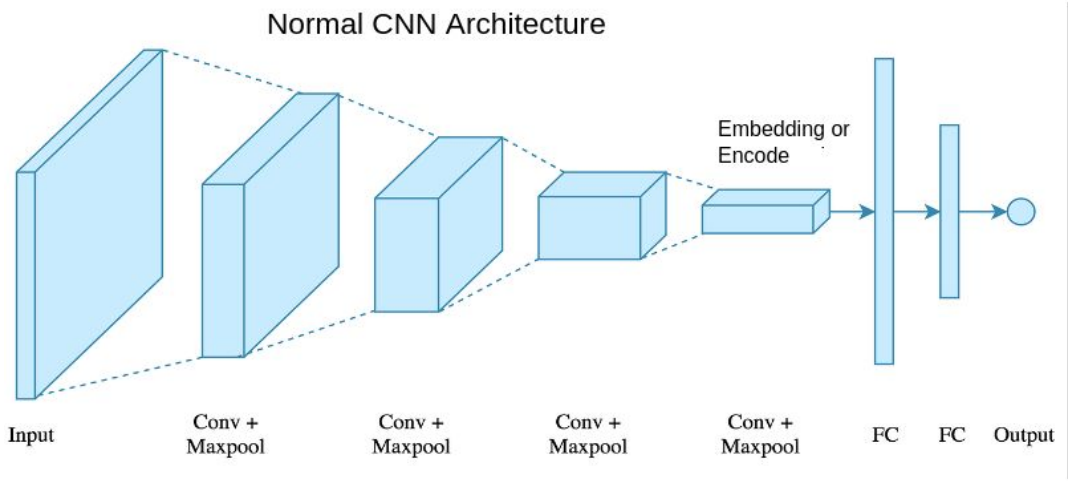


Image Source: Google Search Engine

Embedding / Encode

Encode or embedding or latent space representations can be used for training a smaller network when we have small amount of data or unsupervised training.



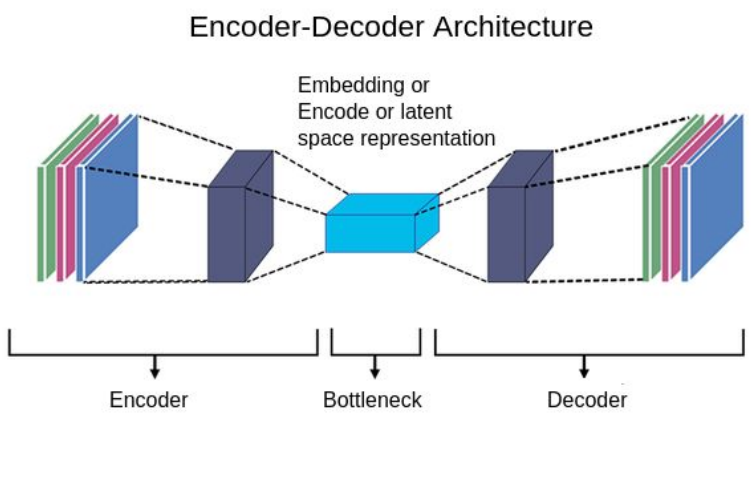
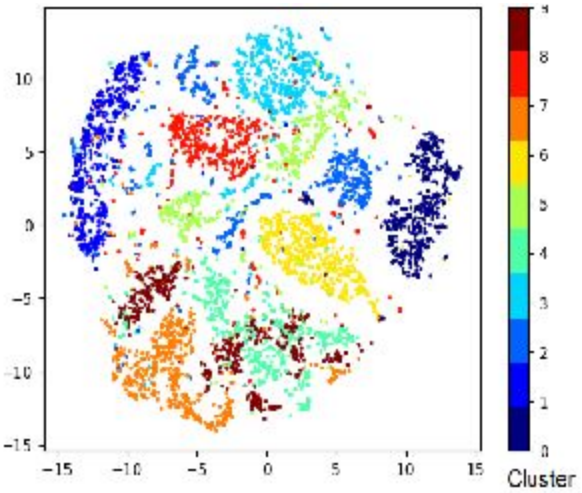


Image Source: Google Search Engine

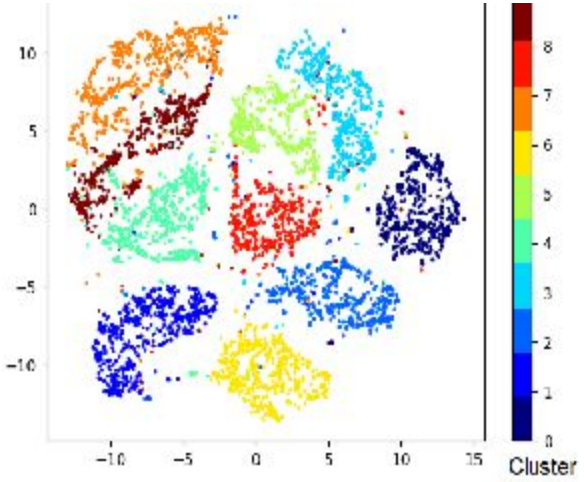
Embedding for Clustering



MNIST Digit Data



k-Means



Autoencoder + k-Means

Image Source: Google Search Engine

Siamese Network

Siamese network is a single network which accepts two or multiple inputs and compares.

It uses same weights for multiple input branches

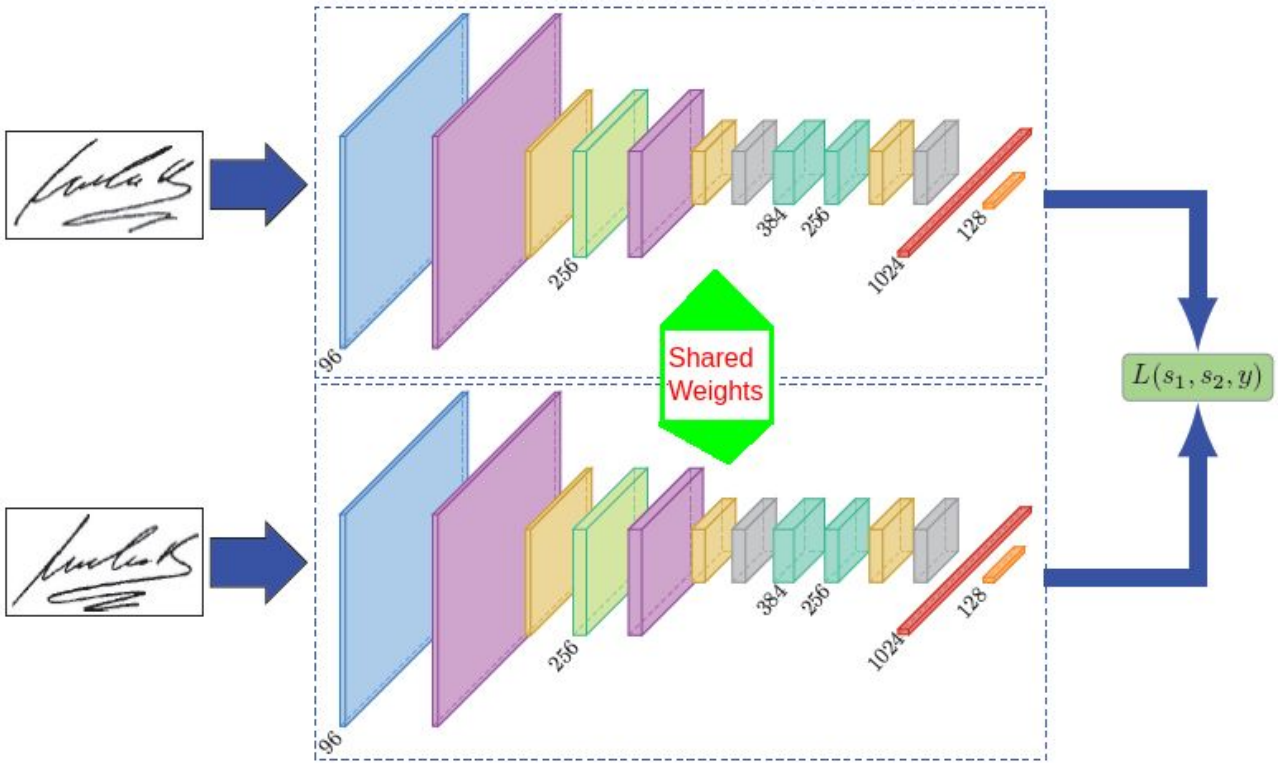


Image Source: Google Search Engine

Self-Supervised vs Supervised Contrastive Loss

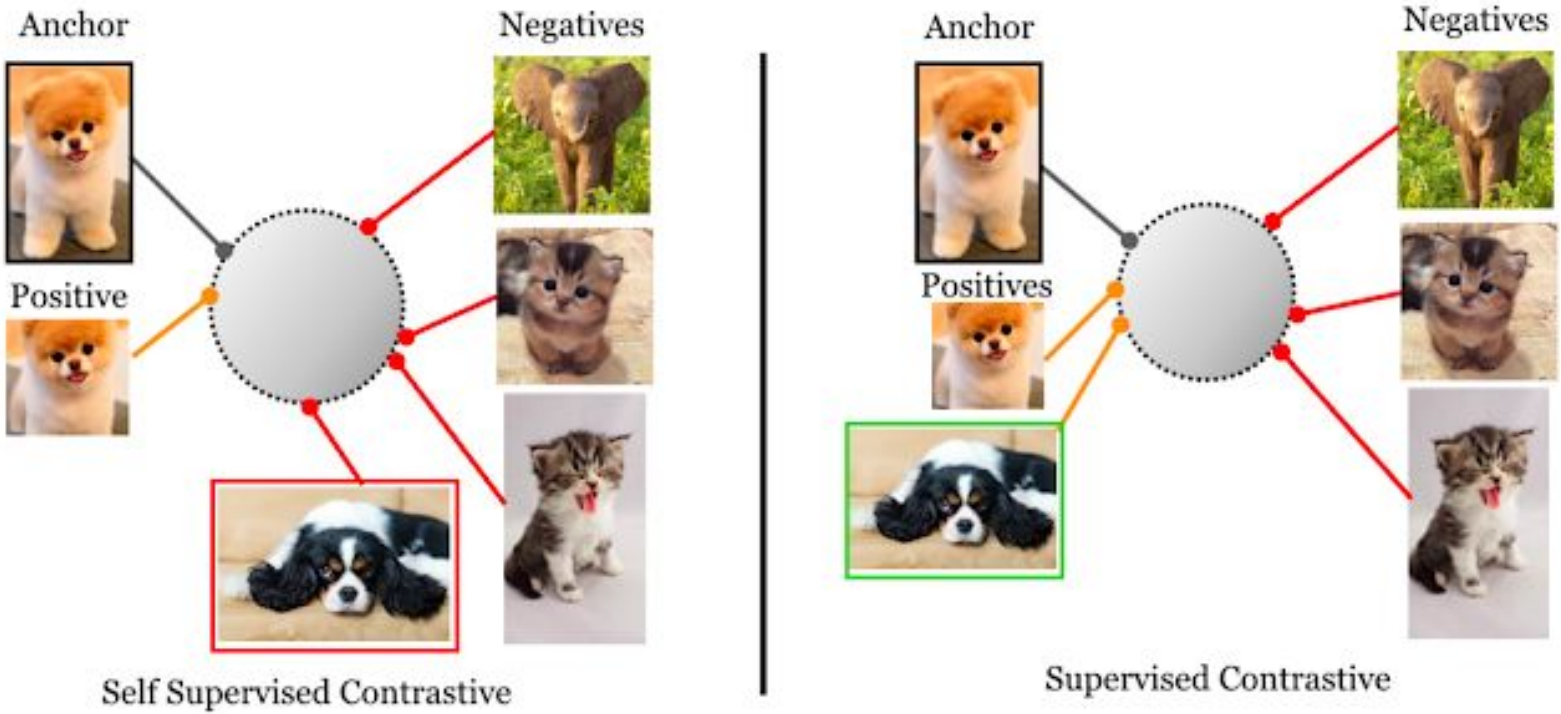


Image Source: Google Search Engine

Triplet Loss

Siamese network uses same weights for multiple input branches

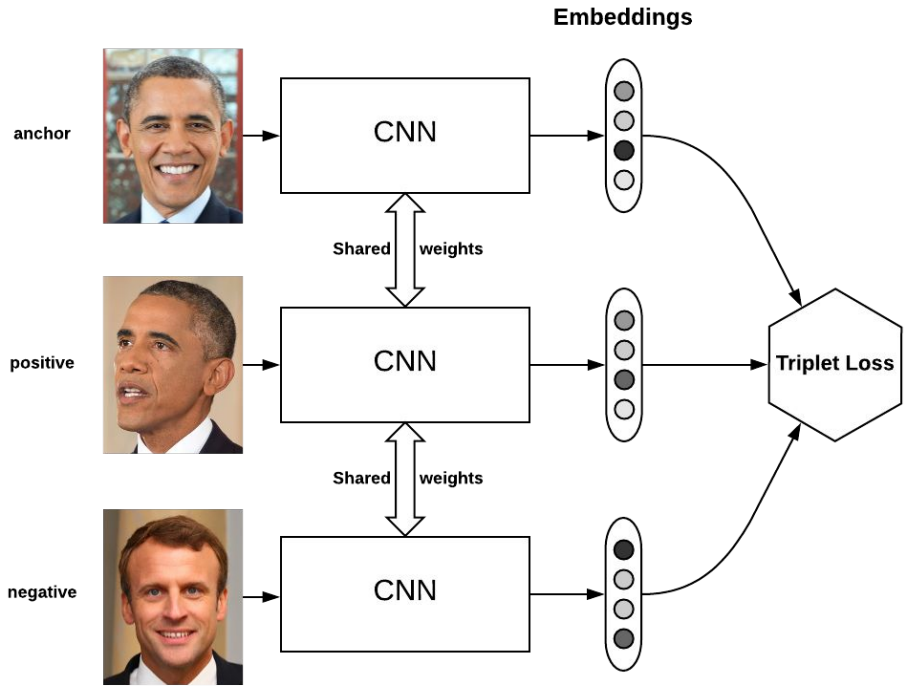
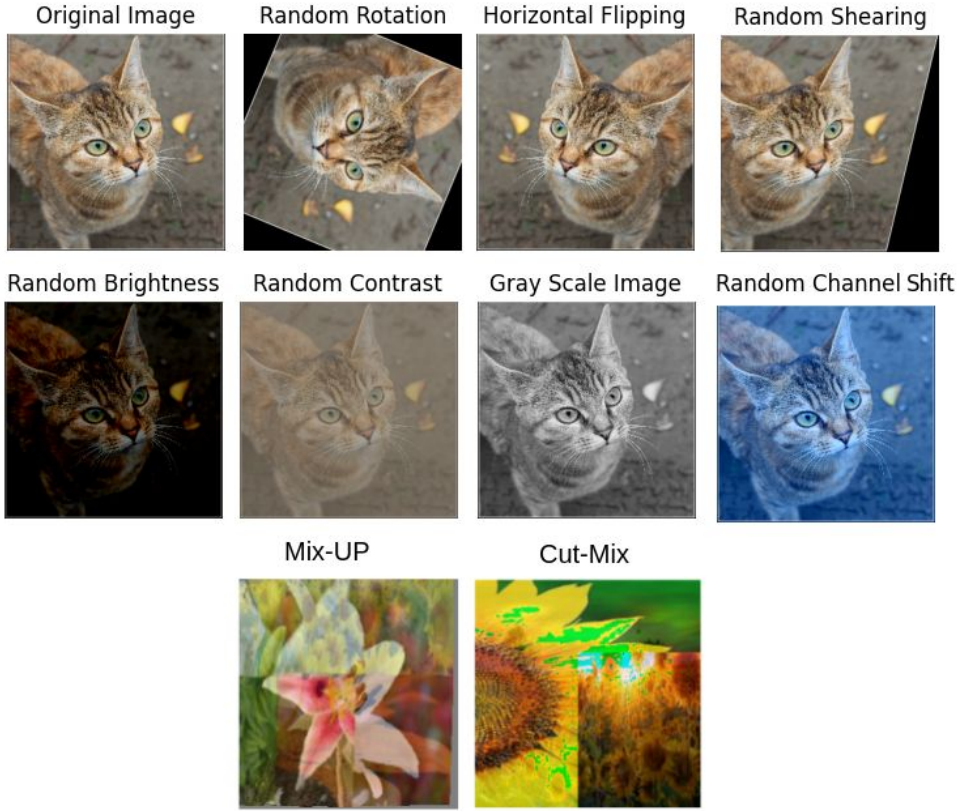


Image Source: Google Search Engine

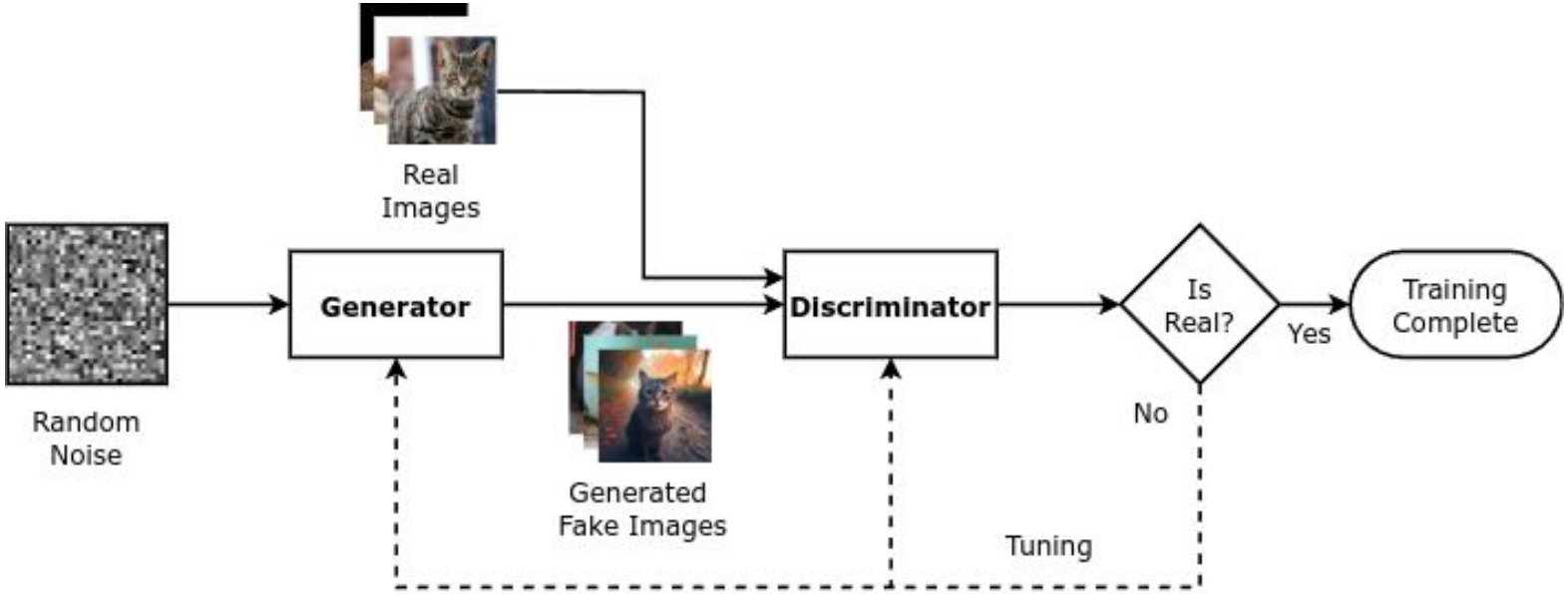
Data Augmentation

- DA is used to increase data
- Random DA should not be used for validation and test sets during validation and testing



Generative Adversarial Neural Network

- Two neural networks are trained one-by-one to beat each other



Explainable NN

Explainable NN help us to understand output of an NN.

Popular methods are:

- Grad-Cam
- Integrated Gradients



Thank You!

(Q&A)

Knowledge Representation
&
Predicate Logic

Professor Dr. A K M Akhtar Hossain
Dept. of CSE, RU

Knowledge Representation

- Knowledge representation (KR) is an important issue in both cognitive science and artificial intelligence.
  - ❑ In cognitive science, it is concerned with the way people store and process information.
  - ❑ In artificial intelligence (AI), main focus is to store knowledge so that programs can process it and achieve human intelligence.
- Knowledge representation in AI is not just about storing data in a database, it **allows a machine to learn from that knowledge and behave intelligently like a human being**.

Knowledge Representation

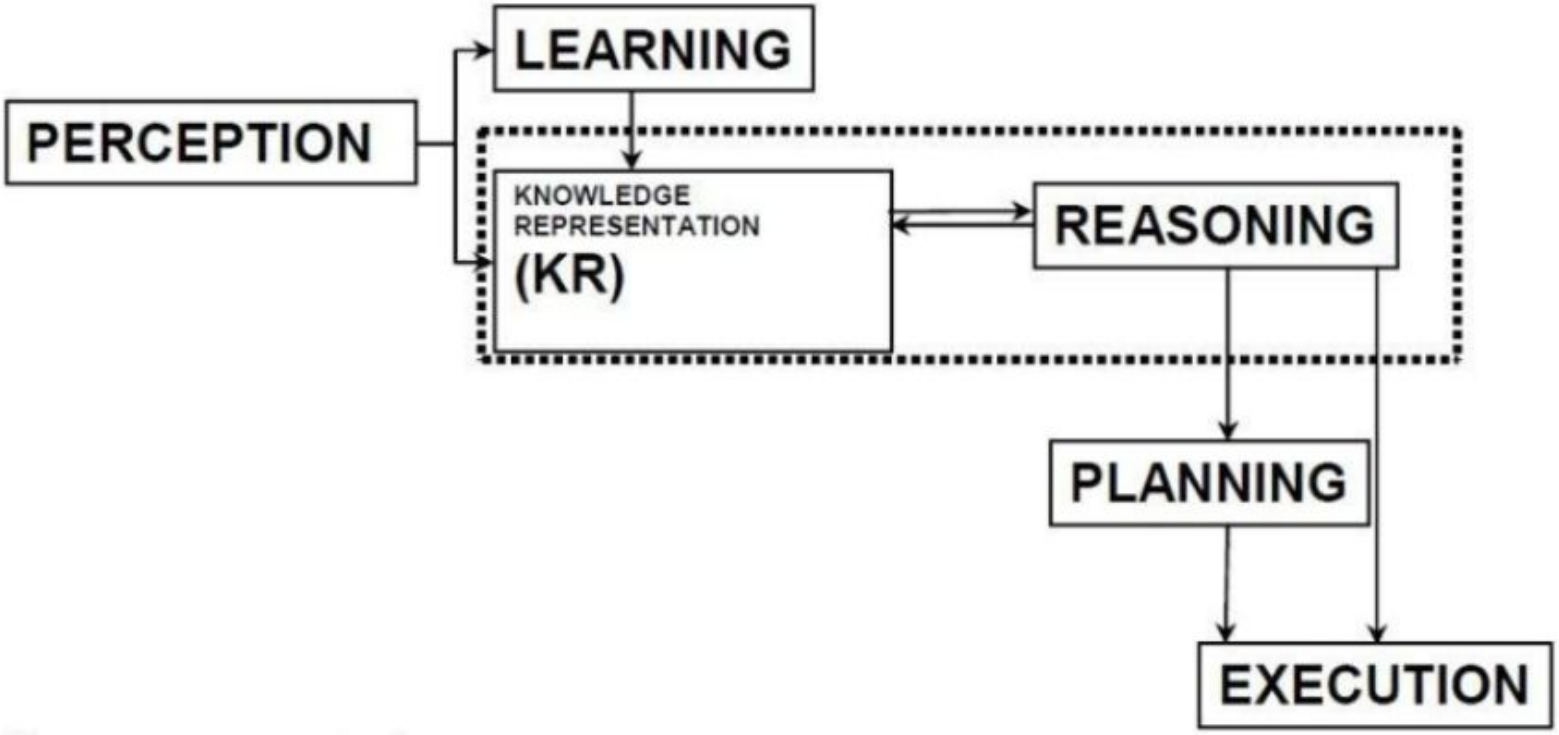


Figure: Knowledge Representation in AI

Components of knowledge Representation system

The knowledge representation function contains the following components.

- **Perception**
- **Learning**
- **KR and Reasoning**
- **Planning and Execution**

Components of knowledge Representation system

- **Perception helps in extracting the information and can be helpful in telling us the status of AI system.** It can detect any irregularity in the system and make us ready to decide whether an AI system has the potentiality of damage or not.
- **Learning component captures the data which are already sensed by the perception component.** Learning component tries to enable the computer to learn just like human instead of always programming it. This component solely tries to focus on how to self-improve the AI system.

Components of knowledge Representation system

- **KR and reasoning are used in AI to acquire knowledge in the smartest way. It focuses on the behavior of an AI agent and make sure that it more or less behaves like human.** It is used to formalize the knowledge in the knowledge base.

- **Planning and execution try to find the optimal solution of the current state and tries to understand the impact of the same.** Now it tries to seek out the solution that the final state holds and then it will try to terminate the entire process with a solution here itself.

Applications of Knowledge Representation

- Knowledge representation and reasoning are powerful tools that can be used **to build intelligent systems**.
- By representing knowledge in a computer-readable format, and by using reasoning to solve problems.
- AI applications can perform tasks that would be difficult or impossible for humans to do.

Applications of Knowledge Representation

- Knowledge representation **makes complex software easier to define and maintain than procedural code and can be used in expert system**
- **Reasoning** is the use of symbolic representations of some statements in order to derive new ones.

What is predicate logic (PL)?

- A **predicate logic** is an expression of one or more variables defined on some specific domain.
- A **predicate** with variables can be made a proposition by either assigning a value to the variable or by quantifying the variable.

Predicate logic (PL):

- A **predicate** is a statement that contains variables (**predicate** variables), and they may be true or false depending on the values of these variables.
- **Predicate logic** is the generic term for symbolic formal systems like first-order **logic**, second-order **logic**, many-sorted **logic**, or infinitary **logic**.

Predicate Logic (PL):

- The Predicate Logic has three **logical notions**.
  1) Term,
  2) Predicate
  3) Quantifier

(1). Term:

- a constant (single individual or concept i.e.,5, john etc.),
- a variable that stands for different individuals
- n-place function $\mathbf{f(t_1, \ldots, t_n)}$, where $t_1, \ldots, t_n$ are terms.
- A function is a mapping that maps **n terms** to a term.

Predicate Logic (PL):

- **Predicate:**
  - a relation that maps n terms to a truth value (T) or false value (F).

- **Quantifier:**
  - Universal ($\forall$) quantifier
  - Existential($\exists$) quantifier

  - **$\forall$ and $\exists$ are used for conjunction with variables.**

Examples

- *"x loves y"* is represented as **LOVE(x, y)** which maps it to true or false, when x and y get instantiated to actual values.

- *"john's father loves john"* is represented as **LOVE(*father*(john), john).**

  ❖ Here ***father*** is a **function** that maps *john* to his father.

Examples

- ***x is greater than y*** is represented in predicate calculus as **GT(x, y)**.
- It is defined as follows:

  GT(x, y) = T , if x > y

  = F , otherwise

- Symbols like **GT** and **LOVE** are called **predicates .**

  ❖ **Predicates two terms and map to T or F depending upon the values of their terms.**

Examples – Cont..

- Translate the sentence "***Every man is mortal***" into Predicate formula.
- Representation of statement in predicate form
  - ❖ "**x is a man**" and "**MAN(x)**,
  - ❖ **x is mortal**" by **MORTAL(x)**
- Every man is mortal :

  (∀ x) (MAN(x) → MORTAL(x))

Here, ∀ x is read as "for all x" and → is read as "implies".

Syntax and semantics for Propositional Logic

- **Valid statements or sentences in PL(Predicate Logic) are determined according to the rules of propositional syntax**.
- This syntax governs the combination of basic building blocks such as propositions and logical connectives.
- **Propositions are elementary atomic sentences.**

Cont……

- **Propositional Logic may be either true or false but may take on no other value.**
- **Examples** (**Simple propositions**):
  - ❖ It is raining.
  - ❖ It is a shining day.
  - ❖ Snow is white.
  - ❖ Snow is black.
  - ❖ People live on the Earth.
  - ❖ People live on the Moon.

Cont....

- **Examples (Compound propositions):**
  - ❖ It is raining and the wind is blowing.
  - ❖ If you study hard you will be rewarded.
  - ❖ The sum of 10 and 20 is not 40.
  - ❖ The sum of 20 and 10 is 40.
- T and F are special symbols having the values true and false.

Conti…

- **Logical Connectives:**

Symbol	Meaning
~	for not or **negation**
&	for **and** or **conjunction**
v	For **or** or **disjunction**
→	For **if … then** or **implication**
↔	For **if and only if** or **double implication**

Syntax

- The syntax of PL is defined recursively as follows:
- T and F are formulas.
- IF P and Q are formulas, the following are also formulas:
- (˜P)
- (P&Q)
- (PVQ)
- (P→Q)
- (P↔Q)

Examples

- 1. Marcus was a man.
- PL: Man(Marcus)
- 2. Marcus was a Pompeian.
- PL: Pompeian(Marcus)
- 3. All Pompeian's were Romans.
- PL: $\forall$ x :Pompeian(x) $\rightarrow$ Roman(x)
- 4. Caesar was a ruler.
- PL: Rular(Caesar)

Examples

- Represent the following facts in predicate logic:
- *(i). All employees earning Tk. 3,00,000/= or more per year have to pay taxes.*
- $\forall x ((E(x) \& GE(i(x), 300000)) \rightarrow T(x))$
- *(ii). People only try to assassinate rulers they are not loyal to.*
- $\exists y: \forall x : person(x) \wedge ruler(y) \wedge tryassassinate(x, y) \rightarrow \neg loyalto(x, y)$
- *(iii). John likes all kinds of food.*
- *Like (john, all-kinds-of-food)*

Semantics

- **The semantics or meaning of a sentence is just the value true or false: that is, it is as assignment of a truth value to the sentences.**
- An interpretation for a sentence or group of sentences in an assignment of the truth value to each propositional symbol.

Conti…

- **Example:** Consider the statement = (P & ~Q)
- Clearly, there are four distinct interpretations for this sentences.

Interpretation	P	Q
1	True	False
2	True	True
3	False	True
4	False	False

Semantic Rules for statements

Consider t and t' denotes true statements, f and f' denotes false statements, and a is any statement.

Rules Number	True Statements	False Statements
1.	T	F
2.	¬f	¬t
3.	t&t'	f&a
4.	tVa	a&f
5.	aVt	fVf'
6.	a→t	t→f
7.	f→a	t↔f
8.	t↔t'	f↔t
9.	f↔f'	

Example:

- Let ***I*** assign true to ***P*** , false to ***Q*** and false to ***R*** in statement = **((P & ¯Q) →R)VQ**.
- **What is the meaning of the statement?**

Answer:

- Rule 2 gives **¯Q** as **true.**
- Rule 3 gives **(P & ¯Q)** as **true**.
- Rule 6 gives **(P & ¯Q) →R** as **false**.
- **Rule 5** gives the statement **((P & ¯Q) →R)VQ** value as **false**.

Assignment-3

- Let ***I*** assign true to ***A*** , false to ***B***, false to ***Q and true to R*** in statement = **((A & ~B) →R)V (~Q V R)** .
- **What is the meaning of the statement?**

- THE END
- THANKS

Syntax and Semantics for FOPL

Prof. Dr. A K M Akhtar Hossain

Dept. of CSE , University of Rajshahi

FOPL (Fist Order Predicate Logic)

- In **first-order predicate logic (FOPL)**, a **predicate can** only refer to a single subject.
- **First-order logic** is also known as **first-order predicate calculus** or **first-order** functional **calculus**.
- A sentence in **first-order Predicate logic** is written in the form Px or P(x), where P is the **predicate** and x is the subject, represented as a variable.

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

FOPL (Fist Order Predicate Logic)

- The syntax for FOPL, like PL, is determined by the allowable symbols and rules of combination.
- **The semantics of FOPL are determined by interpretations assigned to predicates, rather than propositions.**

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Syntax of FOPL

- **The symbols and rules of combination permitted in FOPL are defined as follows:**
- **Connectives:** There are five connective symbols:
  - ~(not or negation)
  - & (and or conjunction)
  - V (**or** or inclusive disjunction)
  - → (implication)
  - ↔ (equivalence or if and only if).

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Syntax of FOPL

- Quantifiers: The two quantifier symbols are ∃ (existential quantification) and ∀ (universal quantification).
- Where (∃**x**) means for some **x** or there is an **x** .

  and (∀**x**) means for **all x**.
- When more than one variable is being quantified by the same quantifier, such as, (∀**x**) (∀**y**) (∀**z**), we abbreviate with a single quantifier and drop the parentheses to get ∀**xyz**.

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Syntax of FOPL

- **Constants:** Constants are **fixed-value** terms that belong to a given domain of discourse.
- They are denoted by **numbers**, **words**, and **small letters** near the beginning of the alphabet.
- **Examples:** a , b , c , 5.256, -67, -75.65 , flight-305, john, , Marina, etc.

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Syntax of FOPL

- **Variables:** Variables are terms that can assume **different values** over a given domain.
- They are denoted by **words** and **small letters** near the end of the alphabet.
- **Examples:** aircraft-type, individuals, x, y, and z.

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Syntax of FOPL

- **Functions**: Function symbols denote **relations** defined on a domain **D**. They map ***n*** elements (***n≥0***) to a single element of the domain.
- Symbols *f* , *g*, & *h*, and words such as **father-of** , or **age-of**, represent functions.
- An ***n*** place (n-ary) function is written as $f(t_1, t_2, t_3, .. \quad .. .. t_n)$ where the t_i are terms (constants, variables, or functions) defined over some domain. **A 0-ary function is a constant.**

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Syntax of FOPL

- **Predicates:** Predicate symbols denote relations or functional mappings from the elements of a domain **D** to the values **true** or **false**.
- Capital letters and capitalized words such as ***P***, ***Q***, ***R***, ***EQUAL***, and ***MARRIED*** are used to represent **predicates**.

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Syntax of FOPL

- Like functions, predicates may have ***n*** (***n ≥ 0***) terms for arguments written as $P(t_1, t_2, t_3, \ldots \ldots, t_n)$
- Where the terms t_i , i =1 , 2, 3, ... , n are defined over some domain.
- **A *0-ary* predicate is a proposition**, that is, a **constant predicate**.

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Syntax of FOPL

- **Constants**, **variables**, and **functions** are referred to as **terms**, and predicates are referred to as **atomic formulas** or **atoms** for short.

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Syntax of FOPL

- **Examples:**
- E1: All employees earning Tk. 300000 or more per year pay taxes.
- E2: Some employees are sick today.
- E3: No employee earns more than the president.

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Syntax of FOPL

- To represent such **expressions FOPL**, we must define abbreviations for the predicates and functions.
- E(*x*) for *x* is an employee.
- P(*y*) for *y* is president.
- i(*x*) for the income of *x* (lower case denotes a function).
- GE(*x*, *y*) for *x* is greater than or equal to *y.*
- S(*x*) for *x* is sick today.
- T(*x*) for *x* pays taxes.

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Syntax of FOPL

- Using the above abbreviations, we represent E1, E2, and E3 as:
- **E1**: ∀x ((E(x) & GE(i(x),300000)) →T(x))
- **E2**: ∃y (E(y) →S(y))
- **E3**: ∀xy ((E(x) & P(y)) → ~GE(i(x),i(y)))

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Syntax of FOPL

- An atomic formula is a **wffs** (**well-formed formulas**) .
- If P and Q are wffs, then ~P, P & Q, P V Q, P $\rightarrow$Q, P$\leftrightarrow$Q, $\forall$x P(x), and $\exists$x P(x) are wffs.
- wffs are formed only by applying the above rules a finite number of times.
- **The above rules state that all wffs are formed from atomic formulas and the proper application of quantifiers and logical connections.**

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Syntax of FOPL

- **Some examples of valid wffs are**
- MAN(john)
- PILOT(father-of(bill))
- ∃ xyz((FATHER(x,y)&FATHER(y,z)) →GRANDFATHER(x,z))
- ∀x NUMBER(x) →(∃y GREATER-THAN(y,x))

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Syntax of FOPL

- **Some examples of statements that are not wffs are:**
- **∀P P(x)→Q(x)**
- /* Universal quantification is applied to the predicate P(x). This is **invalid** in FOPL. */
- **MAN(~john)**
- /*The expression is **invalid** since the term John, a constant, is negated. */
- **father-of(Q(x))**
- /* The expression is **invalid** due to it is function with a predicate argument. */
- **MARRIED(MAN,WOMAN)**
- /* The expression **fails** since it is predicate with two predicate arguments.*/

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Semantics for FOPL

- When considering specific wffs, we always have in mind some **domain D**. If not stated explicitly, D will be understood from the context.
- The arguments predicates must be terms (**constant, variables or functions**). Therefore, the domain of each n-place predicate is also defined over D.

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Semantics for FOPL

- **For Example**, our domain might be **all entities** that make up the Computer Science & Engineering Department at the University of Rajshahi.
- In this case, **constants** would be Professors (Dell, Cooke, Gelfond, and so on), Staff (Martha, Pat, Linda and so on), books, labs, offices and so forth.

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Semantics for FOPL

- The **functions** we may choose might be advisor-of(x), lab-capacity(y), dept-grade-average(z), and the **predicates** MARRIED(x), TENURED(y), COLLABORATE(x,y) to name a few.
- **When an assignment of values is given to each term and to each predicate symbol in a wffs, we say an interpretation is given to the wffs.**

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Semantics for FOPL

- If the truth values for **two different wffs** are the **same under every interpretation**, they are said to be **equivalent**.
- A predicate (or wffs) that has **no variables** is called a **ground atom**.

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Semantics for FOPL

- **For example**, the predicate P(x) in ∀x P(x), is true only if **it is true for every value** of x in the domain D.
- Likewise, the P(x) in ∃x P(x) is **true** only if it is true for **at least one value** of x in the domain.
- If the **above conditions are not satisfied**, the predicate evaluates to **false**.

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Semantics for FOPL

- **For example**, we want to evaluate the truth value of **the expression E**, where
- **E:** ∀x **((A**(a,x) ∨ **B(**f(x)**))** & **C**(x)**)**→**D**(x)
- In this expression, there are **four predicates**: **A, B, C,** and **D**.
- The predicate **A** is a **two-place predicate**, the first argument being the **constant a**, and the second argument, a **variable x**.

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Semantics for FOPL

- The predicates **B**, **C** and **D** are all **unary predicates** where the argument of **B is a function f(x),** and the argument of **C and D is the variable x**.
- Since the whole expression E is quantified with the universal quantifier ∀x, it will evaluate to **true** only if it **evaluates are to be true** for all x in the domain D.

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Semantics for FOPL

- Example: **E:** ∀x **((A**(a,x) V **B(**f(x)**))** & **C**(x)**)**→**D**(x)
- Suppose E is interpreted as follows: Define the domain **D = {1, 2}** and from **D** let the interpretation ***I*** assign the following values:
- a = 2
- X =1 x =2
- f(1) = 2, f(2) = 1
- A(2,1) = true, A(2,2) = false
- B(1) = true, B(2) = false
- C(1) = true, C(2) = false
- D(1) = false, D(2) = true

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Semantics for FOPL

- Using a table such as **Table 4.3** we can evaluates E as follows: **E:** ∀x **((A**(a,x) V **B(**f(x)**))** & **C**(x)**)**→**D**(x)

a) . If **x = 1, A(2,1)** evaluates to **true**,

B(1) evaluates to True , and

(A(2,1) V B(1)) evaluates to true.

C(1) evaluates to true.

Therefore, the expression in the outer parentheses **evaluates to true**.

Hence, since **D(1) evaluates to false**,

Then, the expression E evaluates to false.

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Semantics for FOPL

(b) In a similar way, **if x =2**, the expression can be shown to evaluate to **true**.

Consequently, since E is not true for all x, the expression evaluates to false.

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Properties of WFFS (well-formed formulas)

- As in the case of PL, **the evaluation of complex formulas in FOPL can often be facilitated** through the **substitution of equivalent formulas**.
- **Table 4.2 lists a number of equivalent expressions.**
- **Table 4.4 and Table 4.2 are similar**, but there are some notable differences, particularly in the wffs containing quantifiers.

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Table 4.2 lists some of the important laws of PL (Some Equivalence Laws)

Name of Laws	Statements
Idempotency	P V P = P P & P = P
Associativity	(PVQ)VR = P V (Q V R) (P & Q) & R = P & (Q & R)
Commutativity	P V Q = Q V P P & Q = Q & P P ↔Q = Q↔P
Distributivity	P & (Q V R) = (P & Q)V (P &R) P V (Q & R) = (P V Q) & (P V R)
De Morgan's Laws	~(P V Q) = ~P & ~Q ~(P & Q) = ~P V ~Q
Conditional Elimination	P → Q = ~P V Q
Bi-conditional Elimination	P ↔Q = (p → Q) & (Q →P)

© Professor Dr. A. K. M. Akhtar Hossain,
Dept. of CSE, University of Rajshahi

Table 4.4 Equivalent Logical Expressions

Name of the Rules	Expressions
Double negation	~(~F) = F
Commutativity	F & G = G & F, F V G = G V F
Associativity	(F & G) & H = F & (G & H) (F V G) V H = F V (G V H)
Distributivity	F V (G & H) = (F V G) & (F V H) F & (G V H) = (F & G) V (F & H)
De Morgan's Laws	~(F & G) = ~F V ~G ~(F V G) = ~F & ~G
Conditional Elimination	F → G = ~F V G

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Table 4.4 Equivalent Logical Expressions Cont..

Bi-conditional Elimination	$F \leftrightarrow G = (\sim F \vee G) \& (\sim G \vee F)$
Quantifiers	$\forall x\, F[x] \vee G = \forall x\, (F[x] \vee G)$ $\exists x\, F[x] \vee G = \exists x\, (F[x] \vee G)$ $\forall x\, F[x] \& G = \forall x\, (F[x] \& G)$ $\exists x\, F[x] \& G = \exists x\, (F[x] \& G)$ $\sim(\forall x)\, F[x] = \exists x\, (\sim F[x])$ $\sim(\exists x)\, F[x] = \forall x\, (\sim F[x])$ $\forall x\, F[x] \& \forall x\, G[x] = \forall x\, (F[x] \& G[x])$ $\exists x\, F[x] \vee \exists x\, G[x] = \exists x\, (F[x] \vee G[x])$

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Properties of WFFS

- For example, In Table 4.4 attention is called to the last four expressions which govern substitutions involving negated quantifiers and the movement of quantifiers across conjunctive and disjunctive connections.
- **A wff is said to be valid if it is true under every interpretation**.
- **A wff that is false under every interpretation is said to be inconsistent (or unsatisfiable)**.

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Properties of WFFS

- **If a wff is not valid, then it is called invalid**.
- Likewise, **a wff that is not inconsistent is satisfiable.**
- Again, this means that a valid wff is satisfiable and an inconsistent wff is invalid, but the respective converse statements do not hold.

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Properties of WFFS

- **Finally,** we say that a wff Q is a logical consequence of the wffs $P_1, P_2, P_3, ... , P_n$ if and only if whenever P_1 & P_2 & P_3 & .. .. .. & P_n is **true** under an interpretation.
- Then, **Q is also true**.

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Properties of WFFS

- **Example :**

(a). P &~P is inconsistent and P V ~P is valid since the first is false under every interpretation and the second is true under every interpretation.

(b). From the two wffs

CLEVER(bill) and

$\forall x$ CLEVER(x) $\rightarrow$ SUCCEED(x)

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Properties of WFFS

- We can show that SUCCEED(bill) is a logical consequence. Thus, assume that both

  CLEVER(bill) and

  $\forall x$ CLEVER(x) $\rightarrow$ SUCCEED(x)

  are **true under an interpretation**.

- Then

  CLEVER(bill) $\rightarrow$ SUCCEED(bill)

Is certainly true since the wff was assumed to be true for all x, including x= bill.

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Properties of WFFS

- But,

CLEVER(bill) → SUCCEED(bill) = ~ CLEVER(bill) V SUCCEED(bill)

[Conditional elimination P → Q = ~P V Q]

are equivalent and, since CLEVER(bill) **is true**,

~CLEVER(bill) is false and, therefore,

SUCCEED(bill) must be true.

Thus, we conclude SUCCEED(bill) is a logical consequence of

CLEVER(bill) and ∀x CLEVER(x) → SUCCEED(x)

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

CONVERSION TO CLAUSAL FORM

- **One method we shall examine is called resolution.**
- It requires that all statements be converted into a normalized clausal form.
- **We define a clause as the disjunction of a number of literals.**
- **A ground clause is one in which no variables occur in the expression.**
- **A Horn clause is a clause with at most one positive literal.**

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

CONVERSION TO CLAUSAL FORM

- **Clausal Conversion Procedure:**
- **Step 1.:** Eliminate all implication and equivalency connectives (Use ~P V Q in place of P → Q and (~P V Q) & (~Q V P) in place of P↔Q).
- **Step 2:** Move all negations in to immediately precede an atom (use P in place of ~(~P), and DeMorgran's laws, ∃x ~F[x] in place of ~(∀x) F[x] and ∀x ~F[x] in place of ~(∃x) F[x]).

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

CONVERSION TO CLAUSAL FORM

- **Step 3:** Rename variables, if necessary, so that all quantifiers have variable assignments; that is, rename variables so that variables bound by a different quantifier.
- **For example**, in the expression
- $\forall x\ (P(x) \rightarrow (\exists x\ Q(x)))$

rename the second "dummy" variable **x** which is bound by the existential quantifier to be a different variable, say **y**, to give

$$\forall x\ (P(x) \rightarrow (\exists y\ Q(y))).$$

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

CONVERSION TO CLAUSAL FORM

- **Step 4:** Skolemize by replacing all existentially quantified variables with Skolem functions as described above, and deleting the corresponding existential quantifiers.
- **Step 5:** Move all universal quantifiers to left of the expression and put the expression on the right into CNF (Conjunctive Normal Form).
- **[Skolemization: The process of removing all the existential quantifiers from formula.]**

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

CONVERSION TO CLAUSAL FORM

- **Step 6:** Eliminate all universal quantifiers and conjunctions since they are retained implicitly. **The resulting expressions are clauses and the set of such expressions is said to be in clausal form.**

- **Example: Page No. 64, Patterson**

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

•THANKS

© Prof. Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Inference Rules & Resolution

Prof. Dr. A. K. M. Akhtar Hossain

Dept. of CSE, University of Rajshahi

Inference Rules

- Like PL, a key inference rule in FOPL is **modus ponens**.
- From the assertion "**Leo is a lion**" and the implication "**all lions are ferocious**" .
- We can conclude that **Leo is ferocious**.
- Written in symbolic form we have :
  - **Assertion: LION (leo)**
  - **Implication : $\forall x$ LION (x) $\rightarrow$ FEROCIOUS (x)**
  - **Conclusion : FEROCIOUS (leo)**

© Prof. Dr. A K M Akhtar Hossain, Dept. of CSE, RU.

Resolution

- Its an inference rule used in both propositional as well as First-Order Predicate Logic (FOPL/FOL) in different ways.
- It is also called **proof by Refutation**.
- It is basically used for proving the satisfiability of a sentence.
- Proof by call refutation technique is used to prove the given statement. It's a **iterative process**:
- **Step 1**: Select two clauses that contain conflicting terms.
- **Step 2**: Combine those two clauses and cancel out the conflicting terms, resulting a new clause that has been inferred from them.

© Prof. Dr. A K M Akhtar Hossain, Dept. of CSE, RU.

Resolution

- The key idea is to use the knowledge base and **negated goal** to obtain null clause (which indicates contradiction).
- Since the knowledge base itself is consistent, **the contradiction must be introduced by a negated goal.**
- As a result we have to conclude that the original goal is to true.

© Prof. Dr. A K M Akhtar Hossain, Dept. of CSE, RU.

Steps in Resolution (Algorithm)

- 1. Convert facts into First Order Predicate Logic (FOPL/FOL).
- 2. Convert into FOPL to CNF (Conjunctive Normal Form).
- 3. Negate the statement to be proved and add the result to the knowledge base.
- 4. Draw Resolution graph.
- 5. If empty clause (Nil) is produced, stop and report that original theorem is true.

© Prof. Dr. A K M Akhtar Hossain, Dept. of CSE, RU.

Table: Lists some of the important laws of PL (Some Equivalence Laws)

Name of Laws	Statements
Idempotency	P V P = P P & P = P
Associativity	(PVQ)VR = P V (Q V R) (P & Q) & R = P & (Q & R)
Commutativity	P V Q = Q V P P & Q = Q & P P ↔Q = Q↔P
Distributivity	P & (Q V R) = (P & Q)V (P &R) P V (Q & R) = (P V Q) & (P V R)
De Morgan's Laws	~(P V Q) = ~P & ~Q ~(P & Q) = ~P V ~Q
Conditional Elimination	P → Q = ~P V Q
Bi-conditional Elimination	P ↔Q = (p → Q) & (Q →P)

© Professor Dr. A. K. M. Akhtar Hossain, Dept. of CSE, University of Rajshahi

Table: Equivalent Logical Expressions

Name of the Rules	Expressions
Double negation	~(~F) = F
Commutativity	F & G = G & F, F V G = G V F
Associativity	(F & G) & H = F & (G & H) (F V G) V H = F V (G V H)
Distributivity	F V (G & H) = (F V G) & (F V H) F & (G V H) = (F & G) V (F & H)
De Morgan's Laws	~(F & G) = ~F V ~G ~(F V G) = ~F & ~G
Conditional Elimination	F → G = ~F V G

Table: Equivalent Logical Expressions Cont..

Bi-conditional Elimination	$F \leftrightarrow G = (\sim F \vee G) \& (\sim G \vee F)$
Quantifiers	$\forall x\, F[x] \vee G = \forall x\, (F[x] \vee G)$ $\exists x\, F[x] \vee G = \exists x\, (F[x] \vee G)$ $\forall x\, F[x] \& G = \forall x\, (F[x] \& G)$ $\exists x\, F[x] \& G = \exists x\, (F[x] \& G)$ $\sim(\forall x)\, F[x] = \exists x\, (\sim F[x])$ $\sim(\exists x)\, F[x] = \forall x\, (\sim F[x])$ $\forall x\, F[x] \& \forall x\, G[x] = \forall x\, (F[x] \& G[x])$ $\exists x\, F[x] \vee \exists x\, G[x] = \exists x\, (F[x] \vee G[x])$

Example #1

- Consider the following Knowledge Base:

1. The humidity is high or the sky is cloudy.
2. If the sky is cloudy, then it will rain.
3. If the humidity is high, then it is hot.
4. It is not hot.

Goal: It will rain.

© Prof. Dr. A K M Akhtar Hossain, Dept. of CSE, RU.

Step1: Converted into FOPL/FOL

1. **The humidity is high or the sky is cloudy.**
2. If the sky is cloudy, then it will rain.
3. If the humidity is high, then it is hot.
4. It is not hot.

- **Let P : The humidity is high**
- **Let Q: sky is cloudy**

The humidity is high or the sky is cloudy

P V Q

© Prof. Dr. A K M Akhtar Hossain, Dept. of CSE, RU.

Step1: Converted into FOPL/FOL

1. **The humidity is high or the sky is cloudy.**
2. **If the sky is cloudy, then it will rain.**
3. If the humidity is high, then it is hot.
4. It is not hot.

Goal: It will rain.

- **Let Q: sky is cloudy**
- **Let R: it will rain**

If the sky is cloudy, then it will rain

$Q \rightarrow R$

© Prof. Dr. A K M Akhtar Hossain, Dept. of CSE, RU.

Step1: Converted into FOPL/FOL

1. The humidity is high or the sky is cloudy.
2. If the sky is cloudy, then it will rain.
3. If the humidity is high, then it is hot.
4. It is not hot.

- **Goal:** It will rain.

- Let P : The humidity is high
- Let S: it is hot

If the humidity is high, then it is hot

$P \rightarrow S$

© Prof. Dr. A K M Akhtar Hossain, Dept. of CSE, RU.

Step1: Converted into FOPL/FOL

1. The humidity is high or the sky is cloudy.
2. If the sky is cloudy, then it will rain.
3. If the humidity is high, then it is hot.
4. It is not hot.

- **Goal:** It will rain.

- Let S: it is hot

It is not hot

¬S

© Prof. Dr. A K M Akhtar Hossain, Dept. of CSE, RU.

Step1: Converted into FOPL/FOL

1. The humidity is high or the sky is cloudy.
2. If the sky is cloudy, then it will rain.
3. If the humidity is high, then it is hot.
4. It is not hot.

- P V Q
- Q→R
- P→S
- ¬S

- **Goal:** It will rain. (R)

© Prof. Dr. A K M Akhtar Hossain, Dept. of CSE, RU.

Step 2: Convert into FOPL to CNF &

- P V Q
- Q→R
- P→S
- ¬S

- P V Q
- ¬Q V R
- ¬P V S
- ¬S

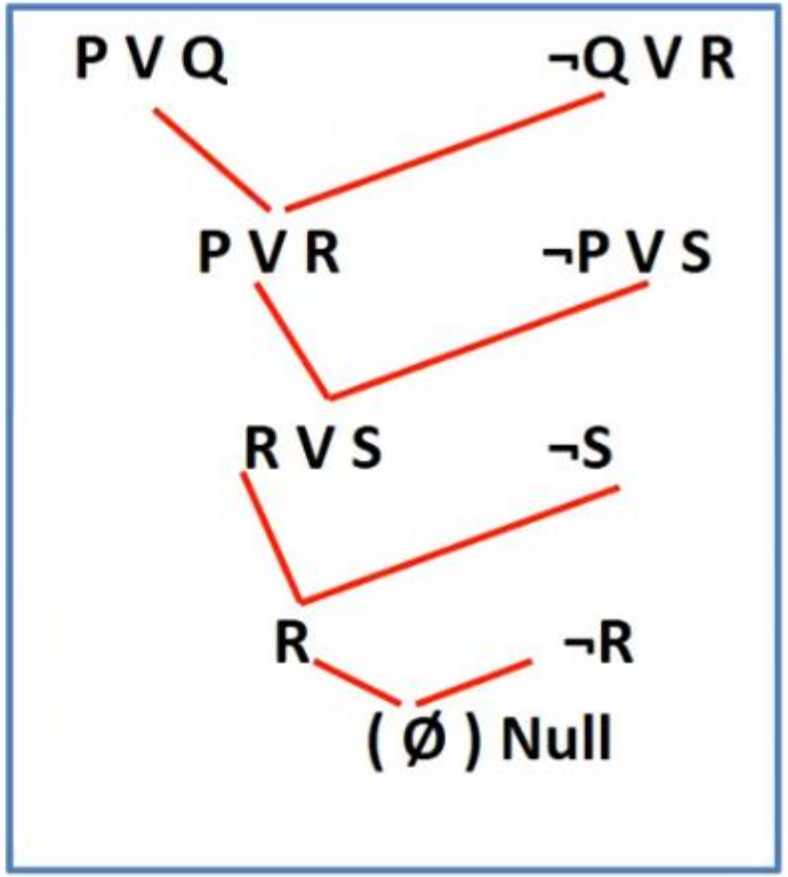
Step 3: Negation the Goal

- GOAL: It will rain . (R)

Negation of Goal (¬R): It will not rain.

Step 4: Resolution of Graph

- P V Q
- ¬Q V R
- ¬P V S
- ¬S
- ¬R (Goal)



© Prof. Dr. A K M Akhtar Hossain, Dept. of CSE, RU.

Assignment #4

- **Consider following the knowledge based:**
- 1. Marcus was a man.
- 2. Marcus was a Pompeian.
- 3. All Pompeian's were Romans.
- 4. Caesar was a ruler.
- 5. All Romans were either loyal to Caesar or hated him.
- 6. Everyone is loyal to someone.
- 7. People only try to assassinate rulers they are not loyal to.
- 8. Marcus tried to assassinate Caesar.
- ***GOAL:***

  Is Marcus hate Caesar?
- Or, **hate(Marcus, Caesar)**

Step 1: Convert into FOPL/FOL

- 1. Marcus was a man.
- PL: man(Marcus)
- 2. Marcus was a Pompeian.
- PL: Pompeian(Marcus)
- 3. All Pompeian's were Romans.
- PL: $\forall x$:Pompeian(x) $\rightarrow$ Roman(x)
- 4. Caesar was a ruler.
- PL: rular(Caesar)
- 5. All Romans were either loyal to Caesar or hated him.
- PL: $\forall x$: Roman(x) $\rightarrow$ loyalto(x, Ceaser) V hate (x, Caesar)

© Prof. Dr. A K M Akhtar Hossain, Dept. of CSE, RU.

Step 1: Convert into FOL

- 6. Everyone is loyal to someone.
- PL: $\forall x :\rightarrow y$: loyalto (x, y)
- PL: $\exists y$: $\forall x$: loyalto (x, y)
- 7. People only try to assassinate rulers they are not loyal to.
- PL: $\exists y$: $\forall x$: person(x) $\wedge$ ruler(y) $\wedge$ tryassassinate(x, y) $\rightarrow$ $\neg$loyalto (x, y)
- 8. Marcus tried to assassinate Caesar.
- PL: tryassassinate (Marcus, Caesar)

© Prof. Dr. A K M Akhtar Hossain, Dept. of CSE, RU.

Step2: Convert FOL into CNF

- 1. man(Marcus)
- 2. Pompeian(Marcus)
- 3. ¬Pompeian(x_1) V Roman(x_1)
- 4. rular(Caesar)
- 5. ¬Roman(x_2) V loyalto(x_2,Caesar) V hate(x_2, Caesar)
- 6. loyalto(x_3, fl(x_3))
- 7. ¬man(x_4) V ¬ruler(y_1) V ¬tryassassinate(x_4,y_1) V loyalto(x_4, y_1)
- 8. tryassassinate(Marcus, Caesar)

GOAL

- You have to write the remaining steps in order to prove the goal.

© Prof. Dr. A K M Akhtar Hossain, Dept. of CSE, RU.

Assignment #4

- 1. If it is sunny and warm day you will enjoy.
- 2. If it is raining then you will get wet.
- 3. It is warm day.
- 4. It is raining.
- 5. It is sunny.

- Goal: You will enjoy.
- Prove: enjoy.

© Prof. Dr. A K M Akhtar Hossain, Dept. of CSE, RU.

- END TODAY
- THANKS

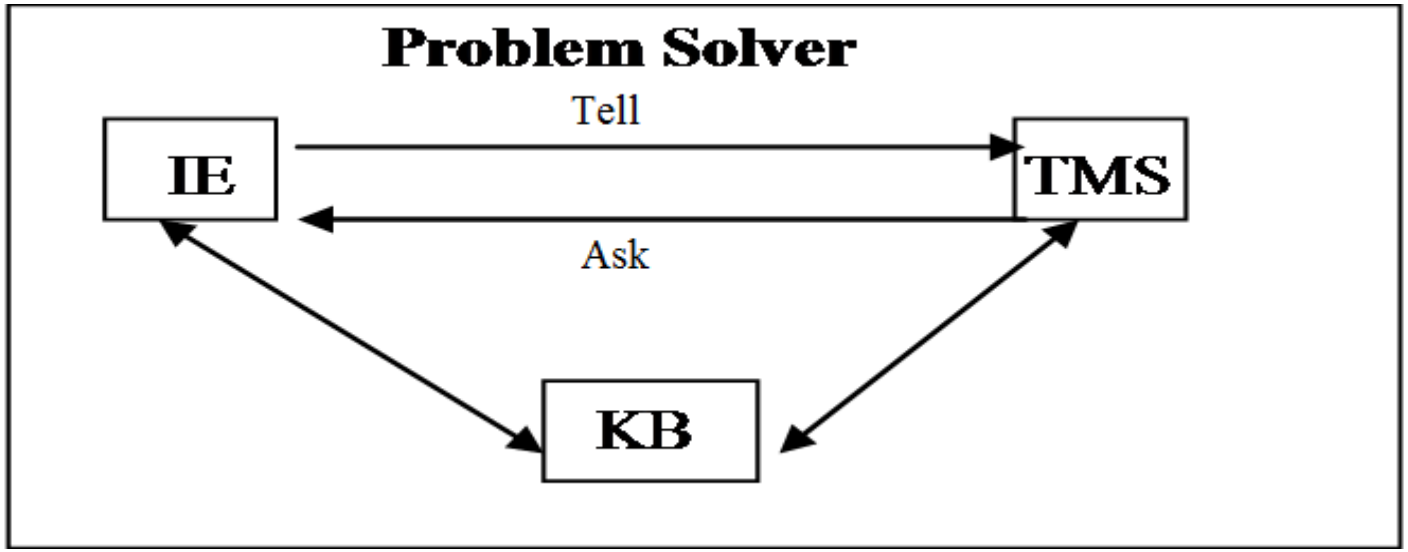
© Prof. Dr. A K M Akhtar Hossain, Dept. of CSE, RU.

Truth Maintenance System (TMS)

Prof. Dr. A K M Akhtar Hossain
Dept. of CSE, RU

Truth Maintenance System (TMS)

- **Definition of TMS:**
- **Truth maintenance system (TMS) works with inference engines for solving problems within large search spaces.**
- **The TMS and inference engine both put together can solve problems where algorithmic solutions do not exist.**
- TMS maintains the beliefs for general problem solving systems.
- IE – Inference Engine
- KB – Knowledge Base



TMS – Cont …

- **TMS can be used to implement monotonic or non-monotonic systems.**
- **In monotonic system**, once a fact or piece of knowledge is stored in KB, it can **not change**.
  - In monotonic reasoning, the world of axioms continually increases in size and keeps on expending.
  - Predicate logic is an example of monotonic form of reasoning. It is a deductive reasoning system where new facts are derived from the known facts.
- **Non-monotonic system** allows **retraction** of truths that are present in the system whenever contradictions arise.
  - So number of axioms can both increase and decrease and depending upon the changes in KB, it can be updated.

Monotonic TMS Methods

- Using Predicate logic – Resolution.
- Natural deduction.
- Logic Programming.
- Forward and Backward reasoning.
- Matching

Monotonic TMS

Monotonic Reasoning

- Once the conclusion is taken, then it will remain same even if we add some other information to existing information in our knowledge base.
- Decision are not affected by new facts, not suitable for real time system.
- Example:

1) Earth revolves around sun
2) Earth is not Round---new information

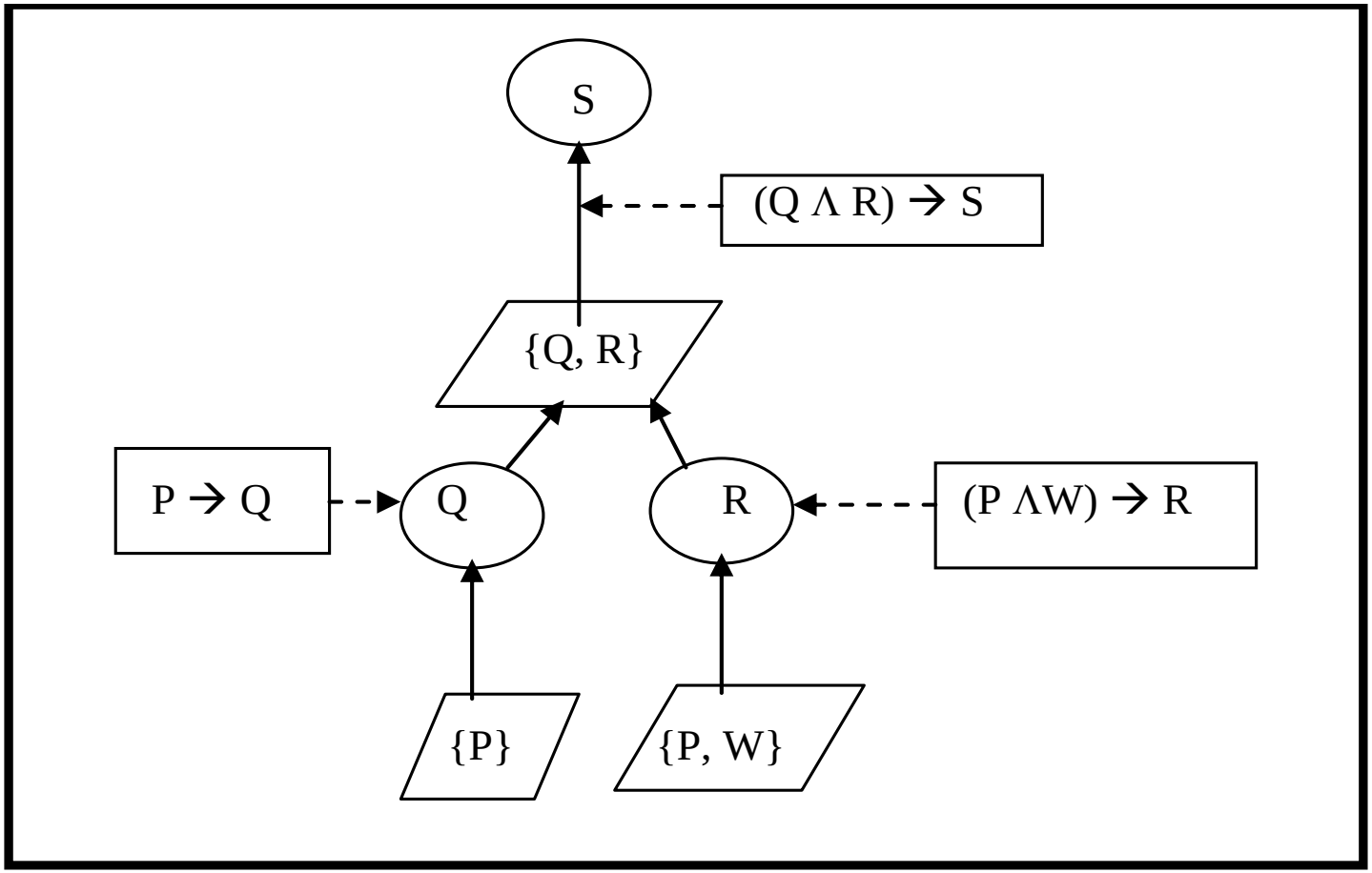
Example – Monotonic TMS

- Suppose we are given the premise set $\sum$ = {P, W} and the internal constraint set

  {P → Q, (P Λ W) → R, (Q Λ R) → S}.

- TMS are able to derive S from these constraints and the premise set $\sum$.
- TMS should provide the justifications of deriving S from constraints and premises.
- Therefore, for any given set of internal constraints and premise set $\sum$, if a formula S can be derived from these, then justification functions generate a justification tree for S.

Example # Justification Tree



Non Monotonic TMS

- **In a non-monotonic reasoning system new information can be added which will cause the deletion or alteration of existing knowledge.**
- **A logic is non-monotonic if some conclusions can be invalidated by adding more knowledge.**
- The logic of definite clauses with negation as failure is non-monotonic.
- Non-monotonic reasoning is useful for representing defaults.
- A default is a rule that can be used unless it overridden by an exception.

Non Monotonic TMS # Example

Non Monotonic Reasoning

- Conclusion may be invalidated if we add some more information to our knowledge base.
- Helpful in real world scenario.

Example:

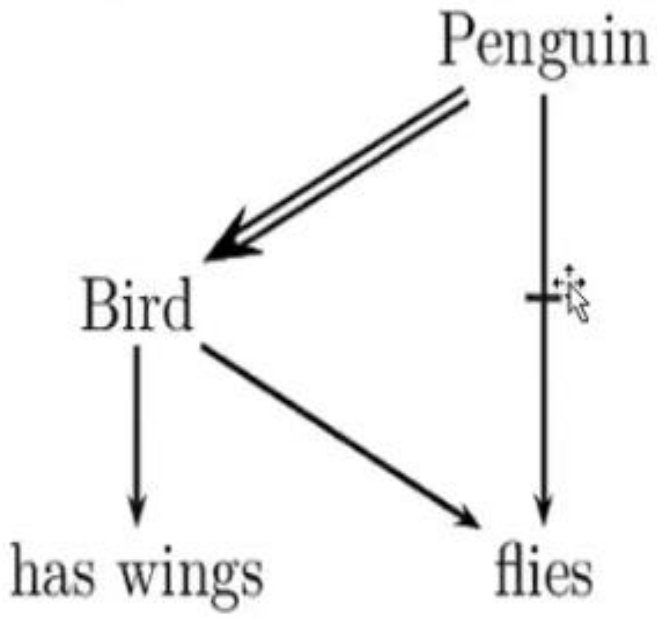
- Facts:
- Birds can fly.
- Penguins can't fly.
- Alex is bird
- Conclusion: Alex can Fly

Non Monotonic TMS # Example

Non Monotonic Reasoning

Example:

- Penguin cannot fly
- Birds has wings
- Birds can fly
- Penguin is a bird
- Conclusion: Penguin can fly



- **THE END**

Searching Techniques & Algorithms in AI

Prof. Dr. A K M Akhtar Hossain
Dept. of CSE, RU

Searching Techniques

- Almost every AI program depends on a search procedure to perform its prescribed functions.
- Problems are typically defined in terms of states, and solutions correspond to goal states.
- Solving a problem then amounts to searching through the different states until one or more of the goal states are found.

Searching Techniques Conti..

- Basically two types:

1. Blind or Uninformed Search
   I. Breadth-First Search
   II. Depth-First search
   III. Bi-directional Search
2. Informed Or Directed search
   I. Heuristics search
   II. Hill Climbing Search
   III. Best-First Search
   IV. A^* search

Blind or Uninformed Search

- In blind or uninformed search, no preference is given to the order of successor node generation and selection.
- The path selected is blindly nor mechanically followed.
- No information is used to determine the preference of one child over another.

Informed Or Directed search

- In informed or directed search, some information about the problem space is used to compute a preference among the children for exploration and expansion.
- Before proceeding with a comparison of strategies, we consider next some typical search problems.

Blind or Uninformed Search

- Breadth-First Search (BFS):
- Breadth-First searches are performed by exploring all nodes at a given depth before proceeding to the next level.
- This means that all immediate children of the nodes are explored before any of the children's children are considered.
- Following Figure shows the Breadth-First Search.

Breadth-First Search(BFS) Graphical Diagram

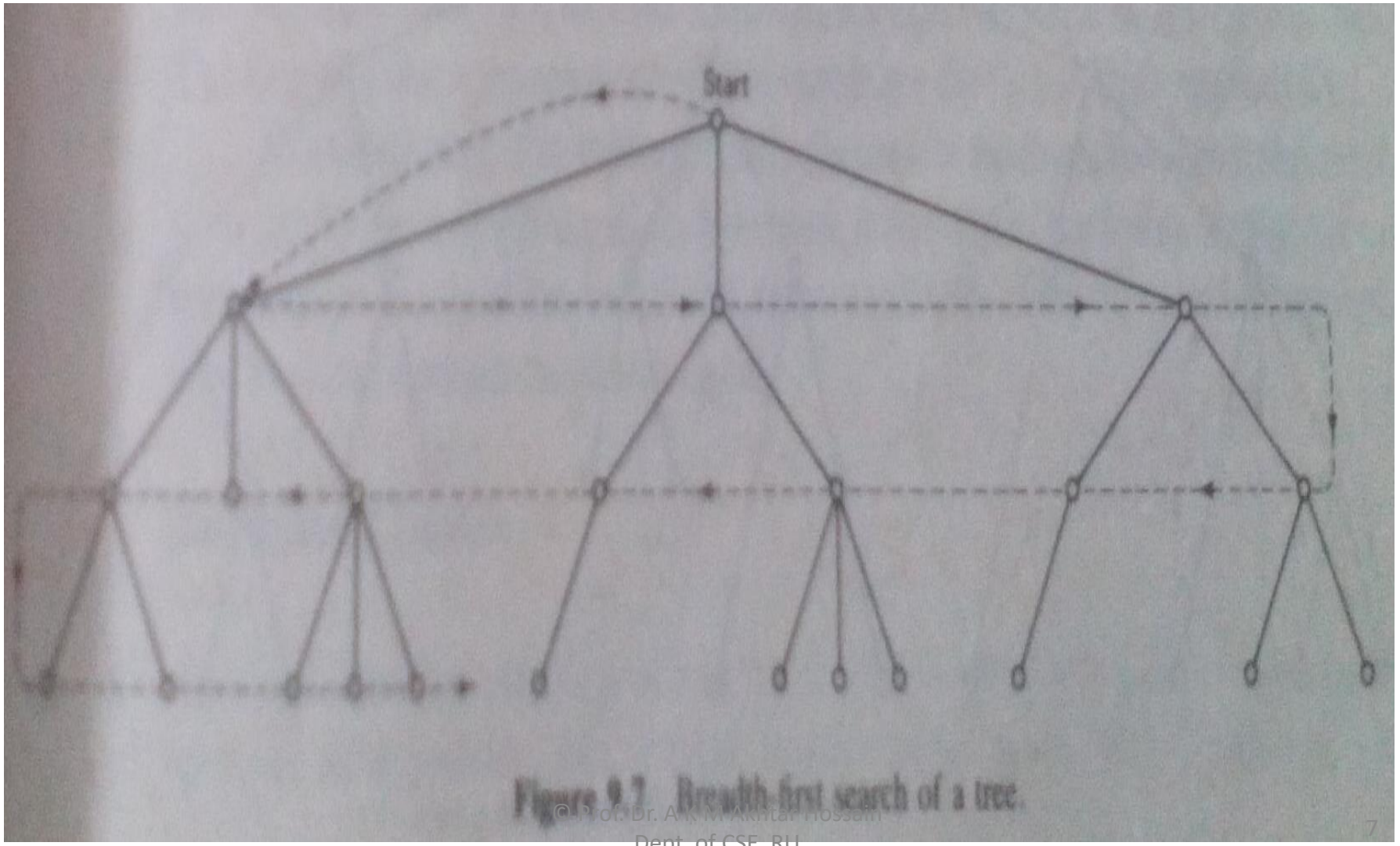


Figure 9.7 Breadth first search of a tree.

Breadth-First Search (BFS)

- **Breadth-first search** (**BFS**) is an algorithm for traversing or searching tree or graph data structures.
- It starts at the tree root (or some arbitrary node of a graph, sometimes referred to as a **search key**) and explores the neighbor nodes first, before moving to the next level neighbors.

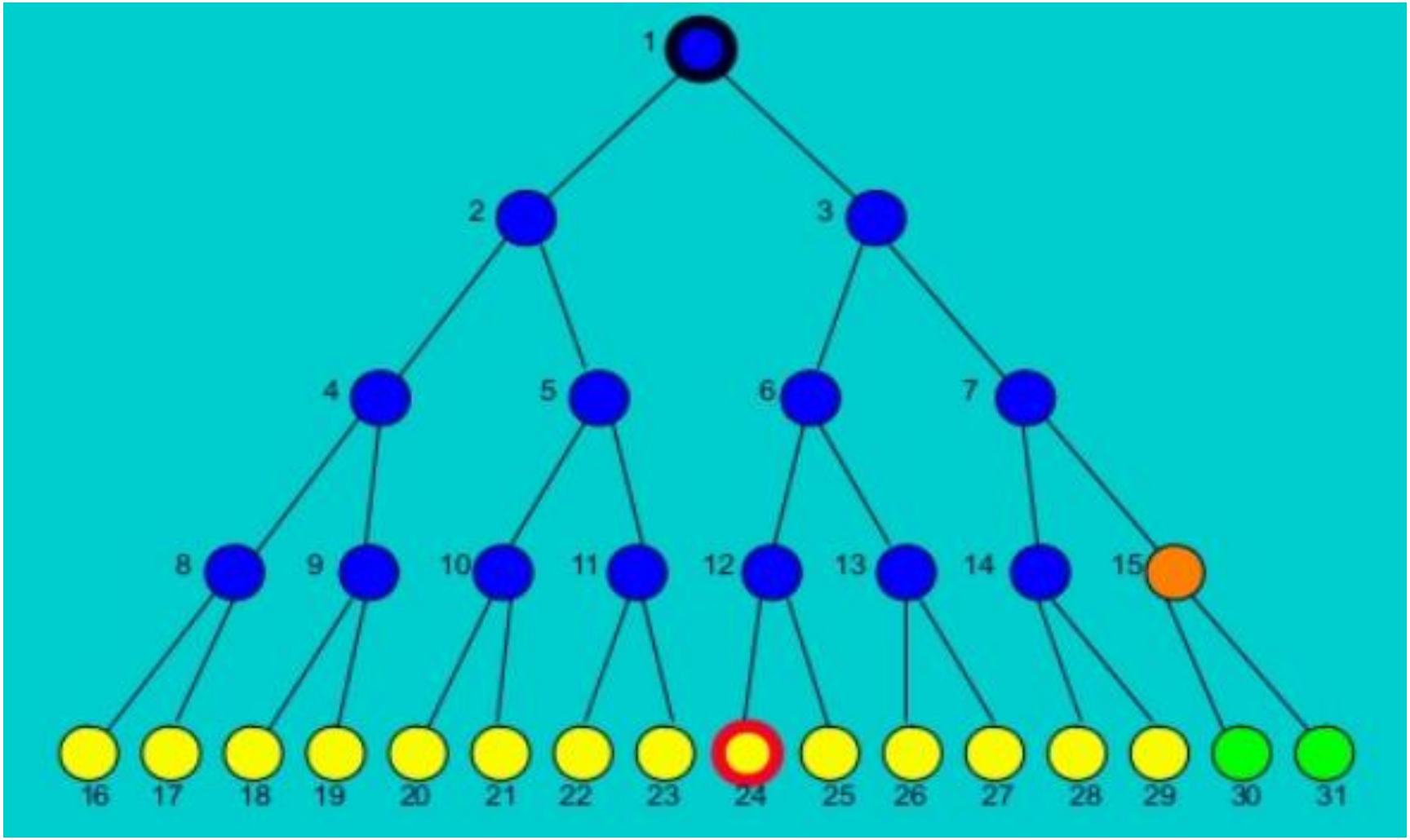
Breadth-First Search

- **Algorithm:**
- **Step 1:** Place the starting node **S** on the queue.
- **Step 2:** If the queue is empty, return failure and stop.
- **Step 3:** If the first element on the queue is a goal node, **g**, return success and stop.
- otherwise,

Breadth-First Search Conti…

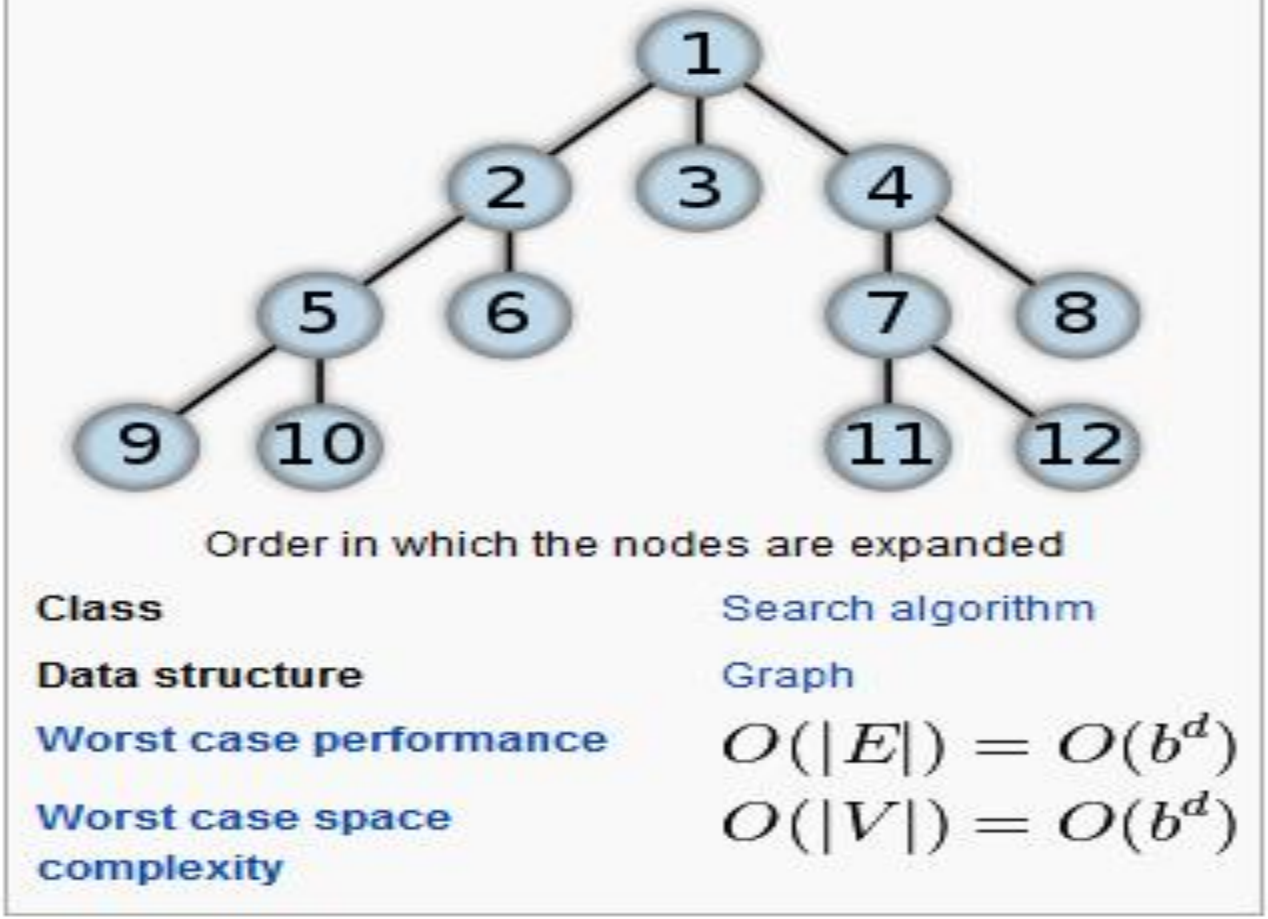
- **Step 4:** Remove and expand the first element from the queue and place all the children at the end of the queue in any order.
- **Step 5:** Return to **Step 2**.

Breadth-First Search(BFS) Graphical Diagram



Breadth-First Search(BFS)

Breadth-first search



Order in which the nodes are expanded

Class	Search algorithm
Data structure	Graph
Worst case performance	$O(\|E\|) = O(b^d)$
Worst case space complexity	$O(\|V\|) = O(b^d)$

Performance of BFS

- Time complexity
  - In worst case BFS must generate all nodes up to depth d

    $1 + b + b^2 + b^3 + \ldots + b^d = O(b^d)$

    - Note on average, half of the nodes at depth d must be examined.
  - So average case time complexity is $O(b^d)$
- Space complexity
  - Space required for storing nodes at **depth d** is $O(b^d)$

Depth-First Search (DFS)

- Depth-first searches are performed by diving downward into a tree as quickly as possible.
- It does this by always generating a child node from the most recently expanded node, then generating that child's children, and so on until a goal is found or some cutoff depth point d is reached.

Depth-First Search Conti….

- If a goal is not found when a leaf node is reached or at the cutoff point, the program backtracks to the most recently expanded node and generates another of its children.
- This process continues until a goal is found or failure occurs.

Depth-first search (DFS)

- **Depth-first search** (**DFS**) is an algorithm for traversing or searching tree or graph data structures.
- One starts at the root (selecting some arbitrary node as the root in the case of a graph) and explores as far as possible along each branch before backtracking.

Depth-first search (DFS) Graphical Diagram

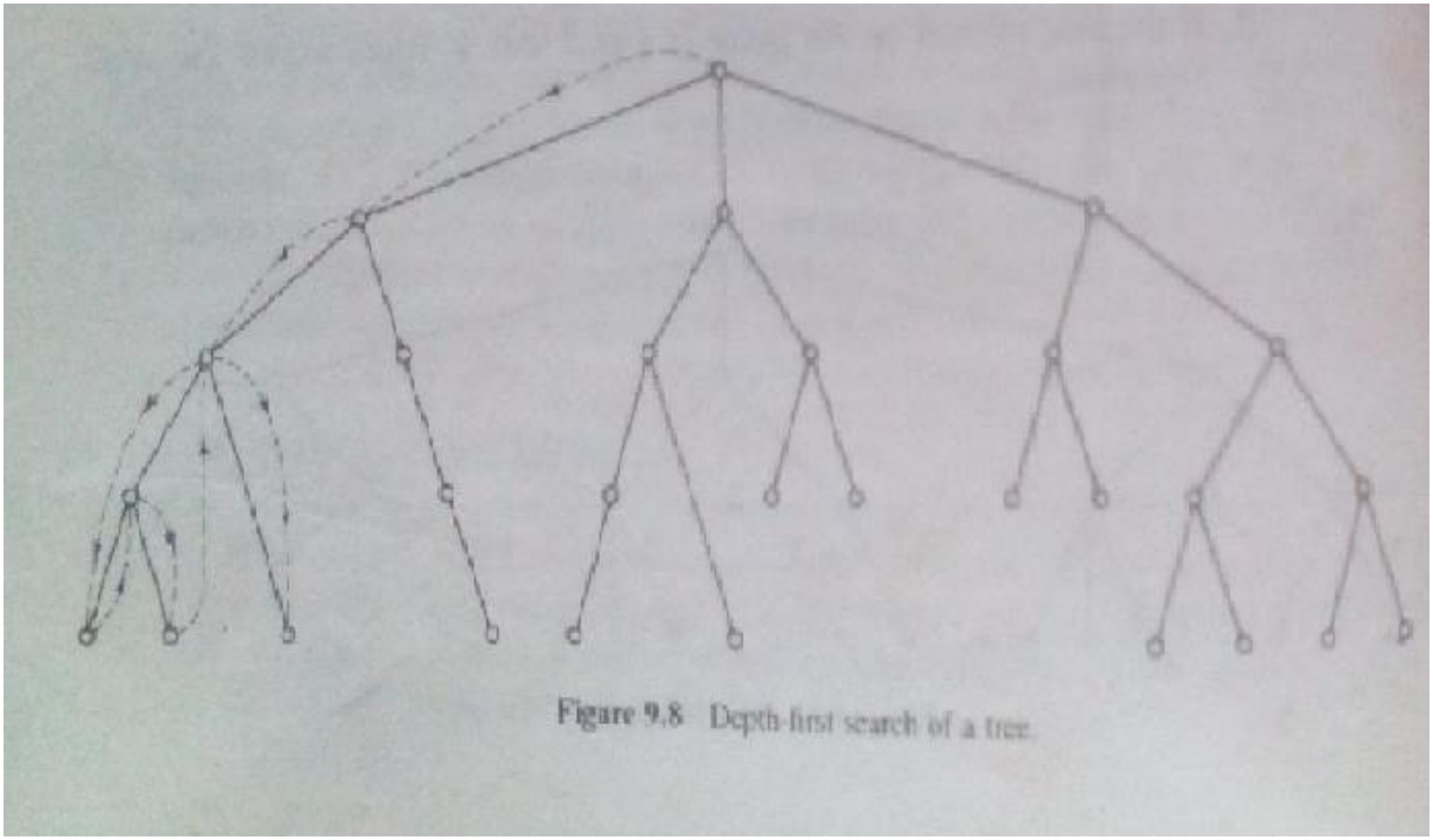


Figure 9.8 Depth-first search of a tree.

Depth-First Search Conti…

- At the start of the algorithm all vertex will be in initial state.
- **ALGORITHM:**
- **Step: 0**
- **Initially Stack is Empty.**
- **Step:1**
- **PUSH starting vertex into the stack.**
- **Step:2**
- **POP a vertex from the stack.**

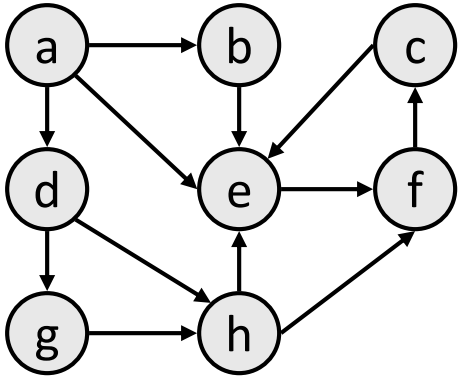
Depth-First Search Conti…

- **Step:3**
- **If popped vertex is in initial state, visit it and change the state from initial to visited state, Push all unvisited vertices adjacent to popped vertex.**
- **Step:4**
- **Repeat Step 2 and 3 until stack is empty.**

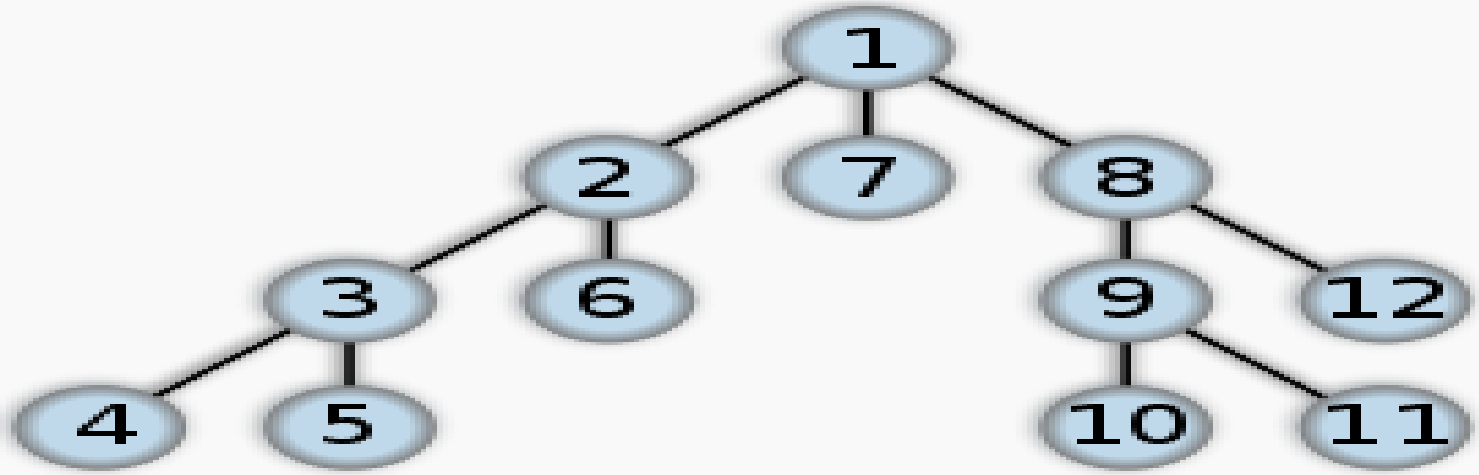
Example: DFS

- **Depth-First Search** (DFS): Finds a path between two vertices by exploring each possible path as far as possible before backtracking.
  - ❑ Often implemented recursively.
  - ❑ Many graph algorithms involve *visiting* or *marking* vertices.

- **Depth-first paths from *a* to all vertices:**
  - to b: {a, b}
  - to c: {a, b, e, f, c}
  - to d: {a, d}
  - to e: {a, b, e}
  - to f: {a, b, e, f}
  - to g: {a, d, g}
  - to h: {a, d, g, h}



Depth-first search



Order in which the nodes are visited

Class	Search algorithm
Data structure	Graph
Worst case performance	$O(\|E\|)$ for explicit graphs traversed without repetition, $O(b^d)$ for implicit graphs with branching factor b searched to depth d
Worst case space complexity	$O(\|V\|)$ if entire graph is traversed without repetition, O(longest path length searched) for implicit graphs without elimination of duplicate nodes

Performance of DFS

- Time complexity
  - In worst case time complexity is $O(b^d)$
- Space complexity
  - If the depth cut off is d the space requirement is O(d)
- DFS requires an arbitrary cut off depth.
  - If branches are not cut off and duplicates are not checked for, the algorithm may not terminate.

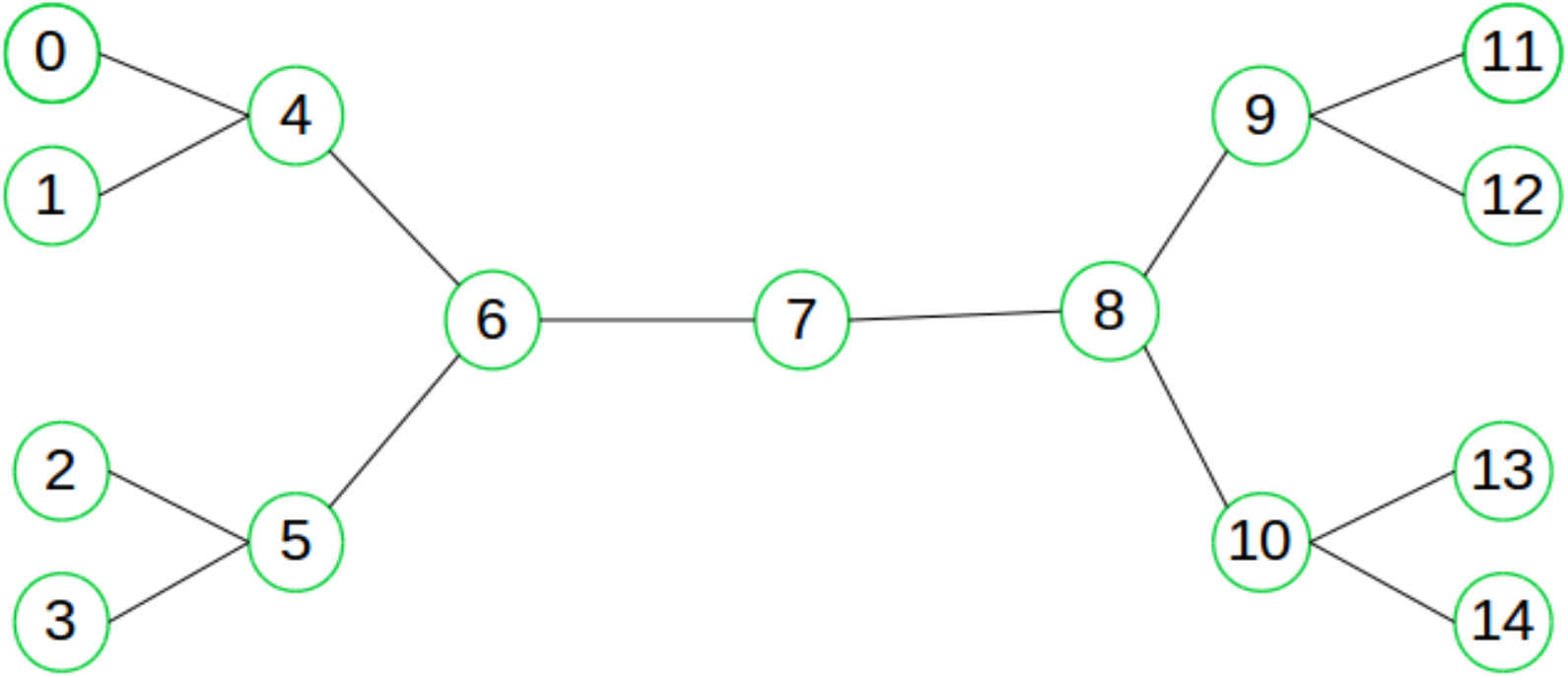
Bi-directional Search

- Bidirectional search is a graph search algorithm which find smallest path form source to goal vertex/node.
- It runs two simultaneous search:
    - Forward search form source/initial vertex toward goal vertex
    - Backward search form goal/target vertex toward source vertex

Bi-directional Search

- Bidirectional search replaces single search graph with two smaller sub graphs, one starting from initial vertex and other starting from goal vertex.
- **The search terminates when two graphs intersect.**
- Bidirectional search can be guided by a heuristic estimate of remaining distance from source to goal and vice versa for finding shortest path possible.

Bi-directional Search Graphical Diagram



Bi-directional Search

- **Consider above simple example:**
- Suppose we want to find if there exists a path from vertex 0 to vertex 14.
- Here we can execute two searches, one from vertex 0 and other from vertex 14.
- When both forward and backward search meet at **vertex 7**, we know that we have found a path from node 0 to 14 and search can be terminated now.

Performance measures

- Suppose if branching factor of tree is **b** and distance of goal vertex from source is **d**, then the normal BFS/DFS searching complexity would be $O(b^d)$.
- On the other hand, if we execute two search operation then the complexity would be $O(b^{d/2})$ for each search and total complexity would be $O(b^{d/2} + b^{d/2})$ which is far less than $O(b^d)$.

© Prof. Dr. A K M Akhtar Hossain
Dept. of CSE, RU

When to use bidirectional approach?

- We can consider bidirectional approach when,
  - Both initial and goal states are unique and completely defined.
  - The branching factor is exactly the same in both directions.

Informed Or Directed search

Informed Or Directed search:

I. Heuristics search
II. Hill Climbing Search
III. Best-First Search
IV. A^* search

Heuristics search

- **Heuristic search** refers to a **search** strategy that attempts to optimize a problem by iteratively improving the solution based on a given **heuristic function** or a cost measure.
- Heuristic search is a “rule of thumb” which is used to help guiding the search.
- It is something learned experientially and recalled when needed.
- **Heuristic Function:**
- It is a function which is applied to a state in a search space to indicate a likelihood of success if that state is selected.

Heuristics search

- **Heuristic Search** –
  - ❖given a **search space**, a **current state** and a **goal state**.
  - ❖generate all successor states and evaluate each with our heuristic function.
  - ❖select the move that finds the **best heuristic value**.
- Here in the associated notes, we examine various heuristic search algorithms.
  - ❖heuristic functions can be generated for a number of problems like games.

Heuristic Search Technique

- In order to find out the heuristic values, we have to consider the following equation (**Heuristic Function**) to select the best value:
- $f(n) = g(n) + h(n)$
- Where,
  - f(n)= cost of selecting state n
  - g(n) = cost of reaching state n from the start state
  - h(n) = heuristic value for state n

Example for Heuristic Search
8-puzzle Problem:

We Know,
f(n) = g(n) + h(n)
f(n) = 0 + 4
So, f(n) = 4

Where, g(n) = 0
h(n) = 4

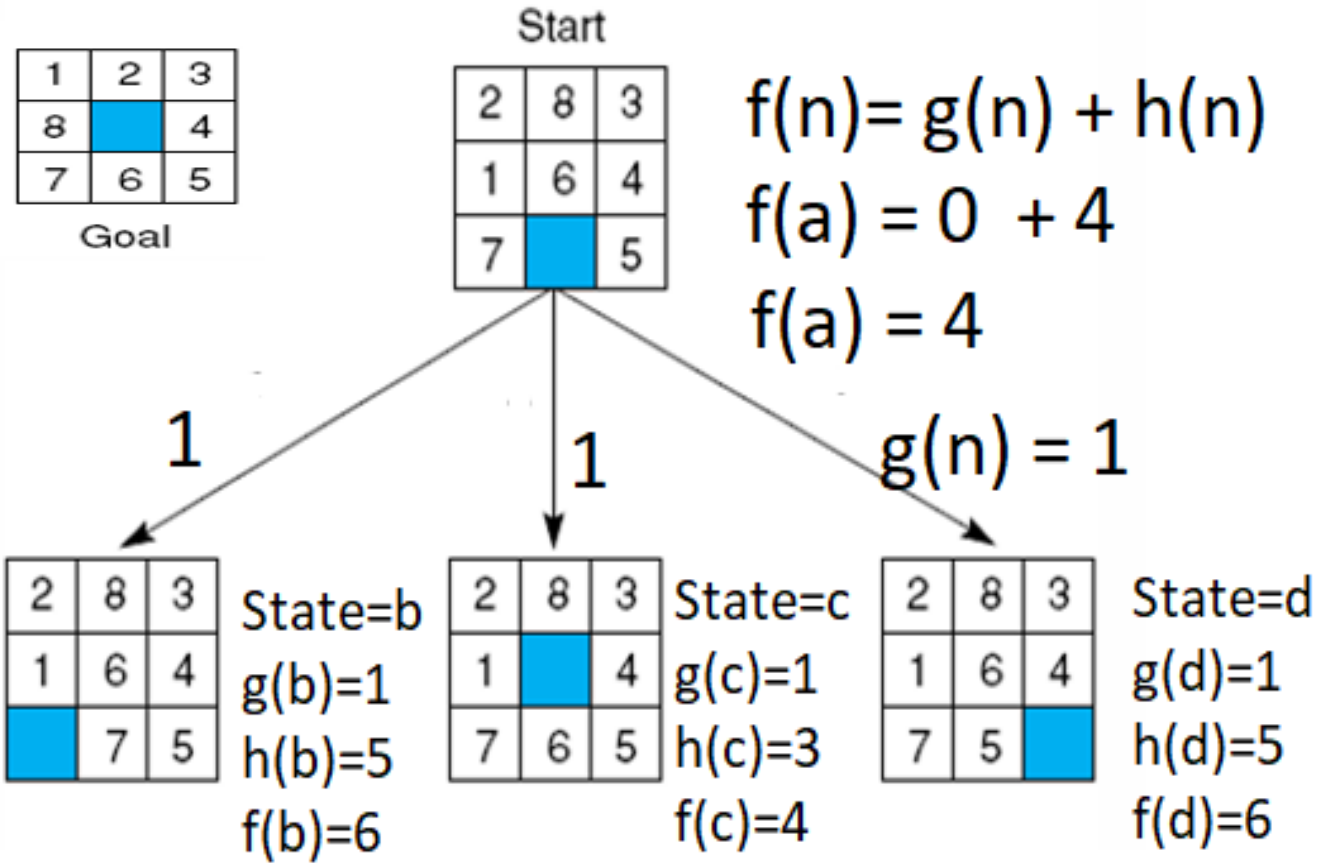
Start

2	8	3
1	6	4
7		5

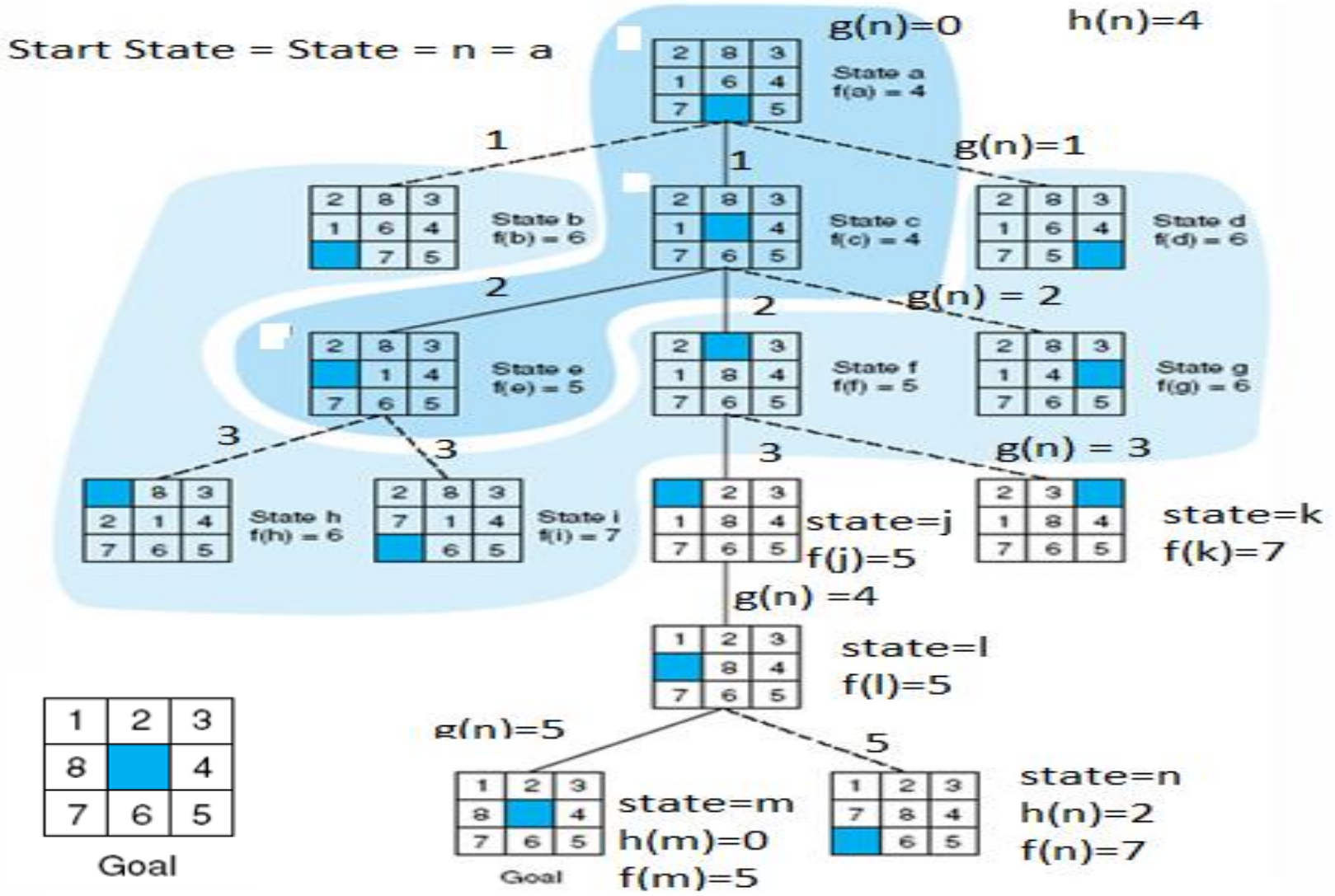
1	2	3
8		4
7	6	5

Goal

Example Heuristic Search



Example Heuristic Search



Hill Climbing Methods

- **Hill climbing is searching where the most promising child is selected for expansion.**
- When the children have been generated, alternative choices are evaluated using some type of heuristic function.
- The path that appears most promising is then chosen and no further reference to the parent or other children is retained.

Hill Climbing Methods Conti…

- This process continues from node-to-node with previously expanded nodes being discarded.
- **Example**: Suppose you are in an unfamiliar city without a map and you want to get downtown. You simply aim for the tall buildings.

Hill Climbing Methods Conti…

- The heuristic function is just distance between the current location and the location of the tall buildings and the desirable states are those in which this distance is minimized.
- Hill climbing can terminate whenever a goal state is reached.

Figure of Hill Climbing Searching

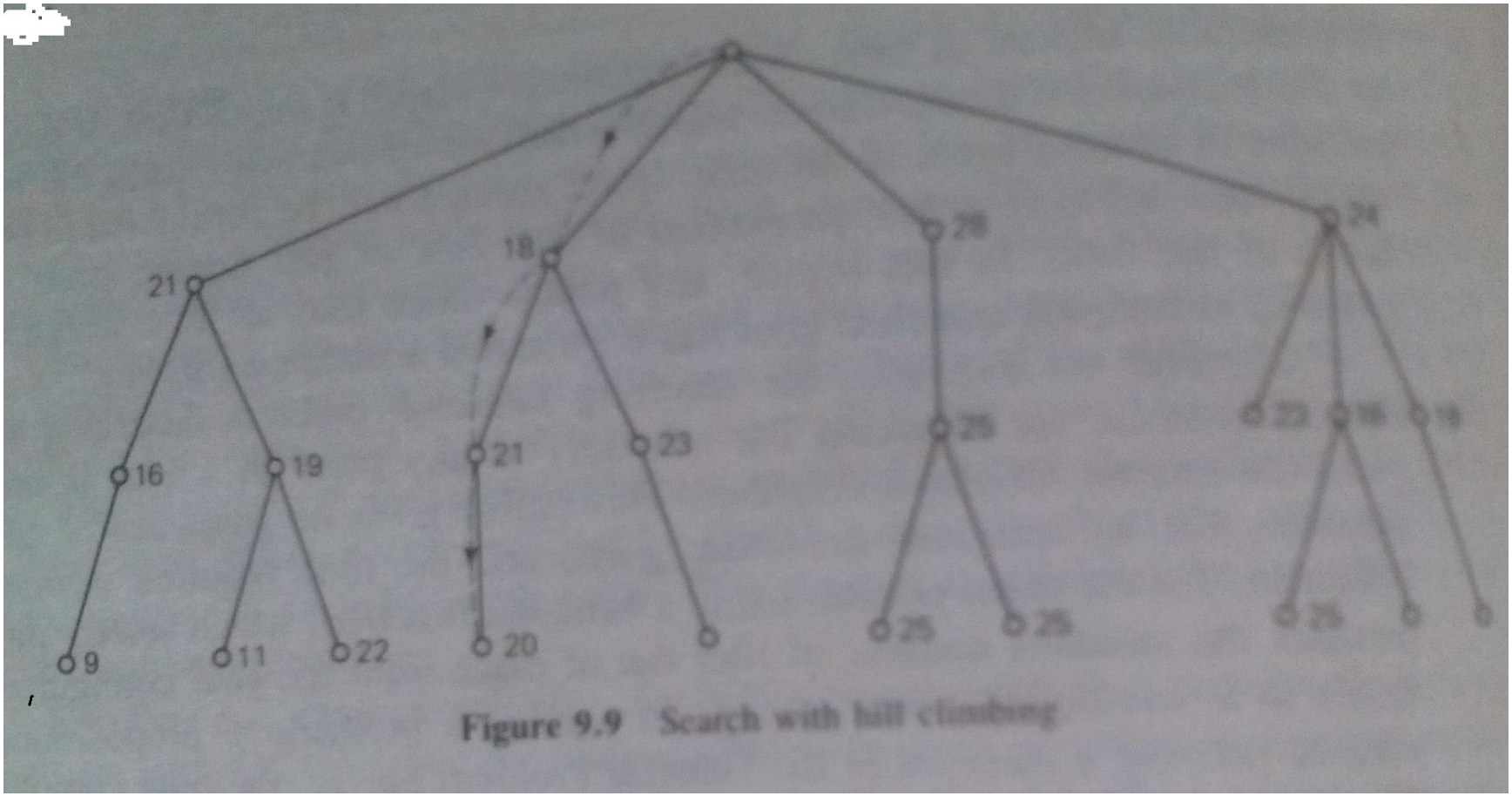


Figure 9.9 Search with hill climbing

Hill Climbing Searching(HCS)

- **Algorithm:**
- **Step 1:** Evaluate the initial state. If it is also a goal state, then return it and quit. Otherwise, continue with the initial state as the current state.
- **Step 2:** Loop until a solution is found or until there are no new operators left to be applied in the current state:

Hill Climbing Searching Algorithm Conti…

(a). Select an operator that has not yet been applied to the current state and apply it to produce a new state.

(b). Evaluate the new state.

(i). If it is a goal state, then return it and quit.

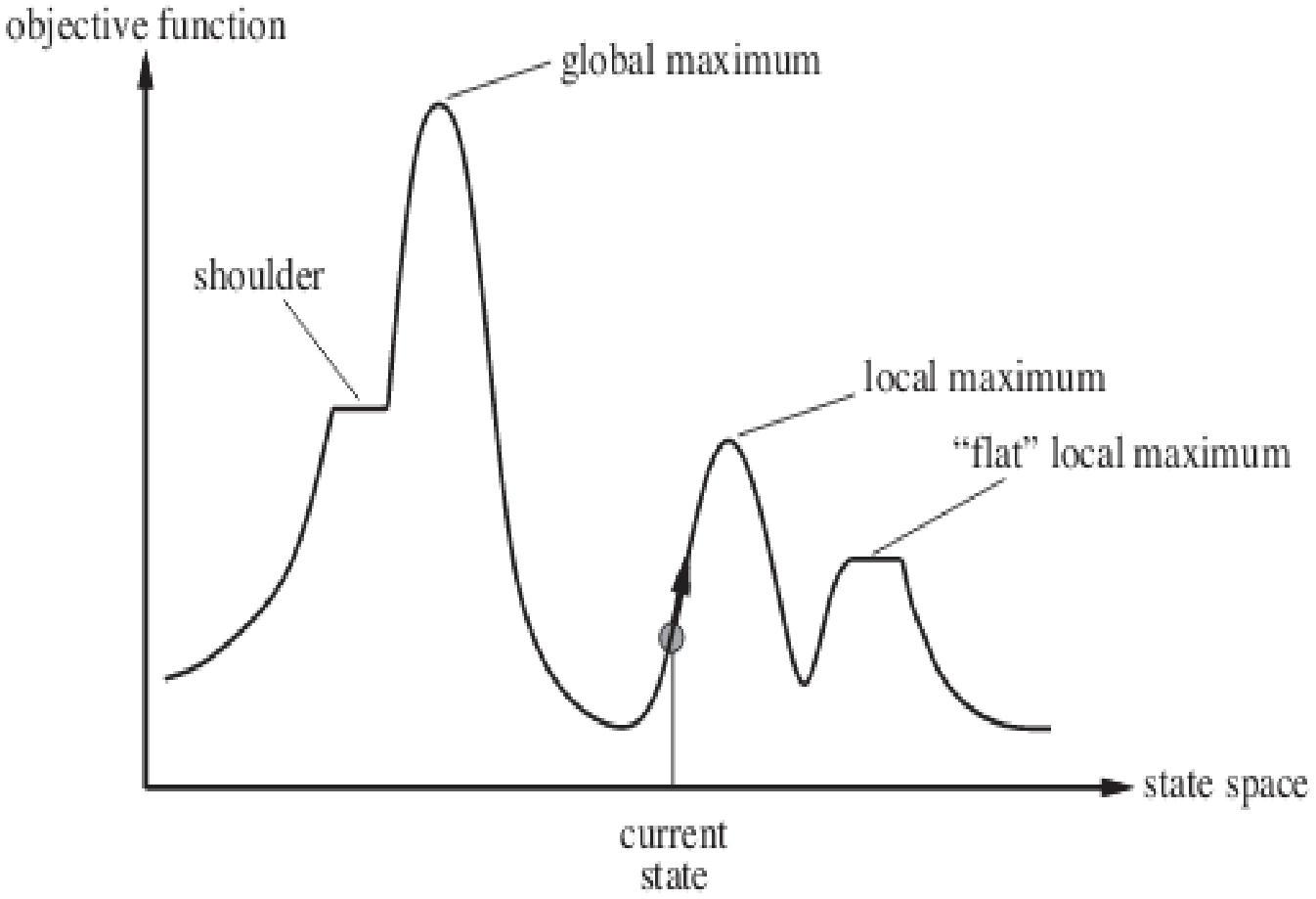
(ii). If it is a not a goal state but it is better than the current state, then make it the current state.

(iii). If it is not better than the current state, then continue is the loop.

State Space diagram for Hill Climbing

- State space diagram is a graphical representation of the set of states our search algorithm can reach vs the value of our objective function(the function which we wish to maximize).
- **X-axis :** denotes the state space is states or configuration our algorithm may reach.
- **Y-axis :** denotes the values of objective function corresponding to a particular state.
- The best solution will be that state space where objective function has maximum value(global maximum).

State Space diagram for Hill Climbing



Different regions in the State Space Diagram:

- **Local maximum:** It is a state which is better than its neighboring state however there exists a state which is better than it(global maximum). This state is better because here the value of the objective function is higher than its neighbors.
- **Global maximum :** It is the best possible state in the state space diagram. This because at this state, objective function has highest value.
- **Plateau/flat local maximum :** It is a flat region of state space where neighboring states have the same value.

Different regions in the State Space Diagram:

- **Ridge :** It is region which is higher than its neighbors but itself has a slope. It is a special kind of local maximum.
- **Current state :** The region of state space diagram where we are currently present during the search.
- **Shoulder :** It is a plateau that has an uphill edge.

Types of Hill Climbing Search

- *Simple hill climbing*
- *Steepest ascent hill climbing*
- *Simulated annealing hill climbing*

Types of Hill Climbing Search

- In *simple hill climbing*, generate and evaluate states until you find one with a higher value, then immediately move on to it
- In *steepest ascent hill climbing*, generate all successor states, evaluate them, and then move to the highest value available (as long as it is greater than the current value)
  - in both of these, you can get stuck in a local maxima but not reach a global maxima
- Another idea is *simulated annealing*
  - the idea is that early in the search, we haven't invested much yet, so we can make some downhill moves

Best-First Search (BFS)

- **Best-first search depends on the use of a heuristic value to select most promising paths to the goal node.**
- The BFS algorithm retains all estimates computed for previously generated nodes and makes its selection based moves forward from the most promising of all the nodes generated so far.

Best-First Search (BFS)

- BFS algorithm selects the path which appears best at that moment.
- BFS algorithm is the combination of BFS(Breadth-First Search) and DFS (depth-First Search).
- Best –First Search(BFS) algorithm uses the heuristics value h(n) and search the lower cost path for the goal node.

Best-First Search (BFS) Algorithm

Best First Search Algorithm

1. Create 2 empty lists: OPEN and CLOSED
2. Start from the initial node (say N) and put it in the 'ordered' OPEN list
3. Repeat the next steps until GOAL node is reached
   I. If OPEN list is empty, then EXIT from the loop returning 'False'.

© Prof. Dr. A K M Akhtar Hossain
Dept. of CSE, RU

Best-First Search (BFS) Algorithm

II. Select the first/start node (say N) in the OPEN list and move it to the CLOSED list. Also capture the information of the parent node.

III. If N is a GOAL node, then move the node to the Closed list and exit the loop returning 'True'. The solution can be found by backtracking the path.

IV. If N is not the GOAL node, expand node N to generate the 'immediate' next nodes linked to node N and add all those to the OPEN list.

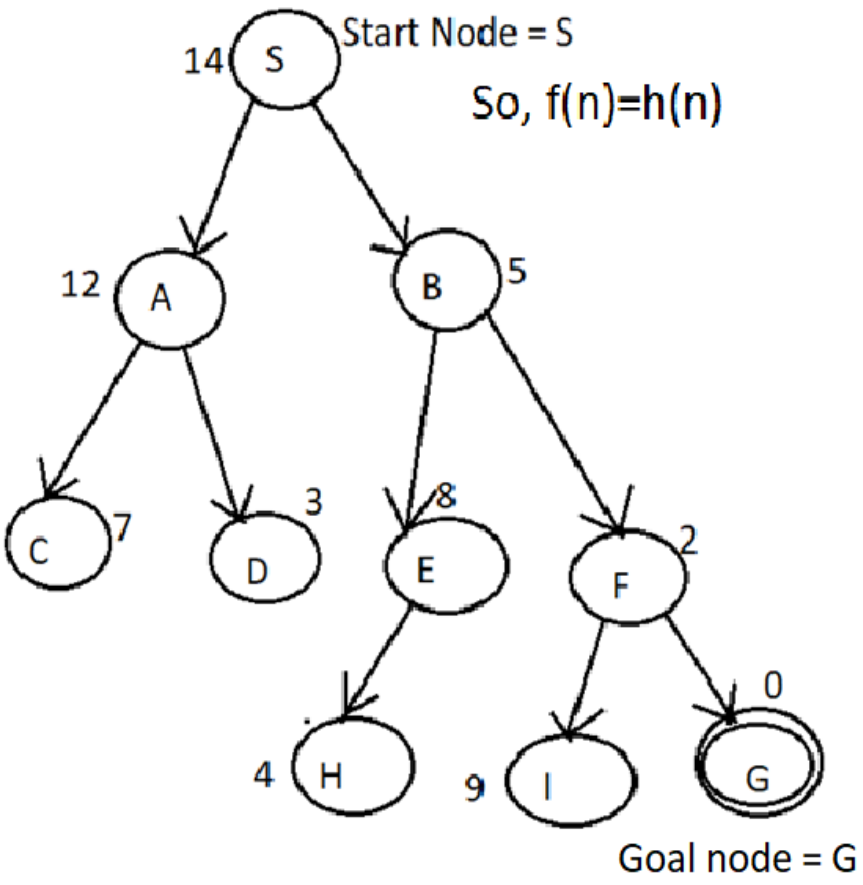
V. Reorder the nodes in the OPEN list in ascending order according to the heuristic evaluated function f(n).

Example for Best-First Search (BFS)

BFS Algorithm

State	h(n)
S	14
A	12
B	5
C	7
D	3
E	8
F	2
H	4
I	9
G	0

f(n)=[g(n)+h(n)] Here, g(n)=0



=> initialization
OPEN List, CLOSE List
Open[S], Close[ ]
Open[B,A], Close[S]
Open[A], Close[S,B]
Open[F,E,A], Close[S,B]
Open[E,A], Close[S,B,F]
Open[G,I,E,A], Close[S,B,F]
Open[I,E,A], Close[S,B,F,G]
Goal is found, So we have gotten **Final Path**.
Final Path= S → B→ F→G

Example for Best-First Search (BFS)

Selection of lower cost path.	=> initialization
Step-1: Initialization; Start State = S; Goal State= G; **Step-2:** S → A ; h(n)=h(A)=12 (Open) S → B ; h(n)= h(B)= 5, So, h(B) ≤ h(A) **Step-3:** S → B→ E ; h(E)= 8 (Open) S → B→ F ; h(F) = 2 , So, h(F) ≤ h(E) **Step-4** S → B→ F→I ; h(I)= 9 (Open) S → B→ F→G ; h(G)= 0 , So, h(G) ≤ h(I) **Final Path**: S → B→ F→ G ;	OPEN List, CLOSE List Open[S], Close[ ] Open[B,A], Close[S] Open[A], Close[S,B] Open[F,E,A], Close[S,B] Open[E,A], Close[S,B,F] Open[G,I,E,A], Close[S,B,F] Open[I,E,A], Close[S,B,F,G] Goal is found, So we have gotten Final Path. Final Path : S → B→ F→ G

Performance of BFS

- In worst case **time complexity** for **Best First Search** is O(n * Log n), where n is number of nodes. In worst case, It may has to visit all nodes before we reach goal.
- Best–First Search Space Complexity: $S(C) = O(b^m)$
- Where m is the max path length (so max number of ancestors) and b is the branching factor (number of children per ancestor).
- Performance of the algorithm depends on how well the cost or evaluation function is designed.

A* search Algorithm

- A* search algorithm finds the shortest path through the search space using the heuristic function.
- It uses heuristic value, h(n) and cost of reach the node n from the start state, g(n).
- This algorithm expands less tree and provides optimal result faster.
- A* search algorithm uses heuristic value of the node and as well as the cost to reach the node.

A* search Algorithm

- Hence we get,
- F(n) = g(n) + h(n)
- Where,
- F(n) = Estimated cost of the path.
- g(n) = Cost of reach node n from the start state.
- h(n)= The heuristic value of the n node.

A* search Algorithm

- Step-1: Consider OPEN List and CLOSED List. Place the starting node in OPEN list.
- Step-2: Check if the OPEN list is empty or not, if the list is empty then return, failure & stop.
- Step-3: Select the node from the OPEN list which has the lower cost value of the Evaluated Heuristic function [F(n)=g(n) + h(n)], if node n is goal node, then return success & stop, otherwise,
- Step-4: Expand node n & generate all of its successors & put n in the closed list:-

A* search Algorithm

i. For each successor node n check whether node n is already in the OPEN or CLOSED list.

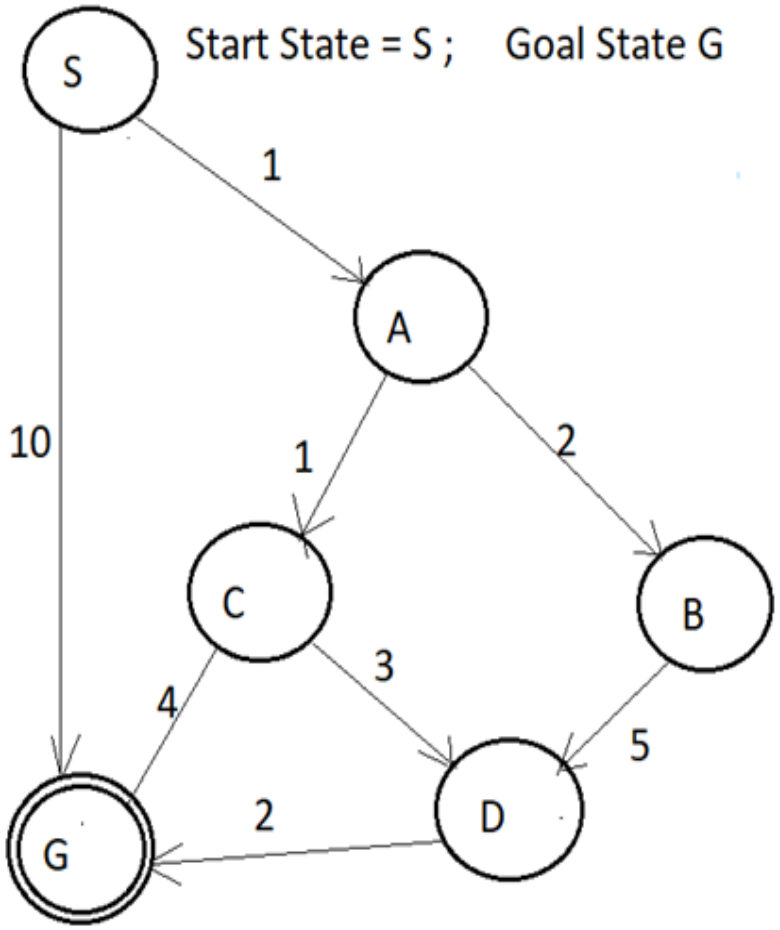
ii. If not then compute evaluation function for node n and place into OPEN list.

Step-5: Else if node n is already in OPEN & CLOSED, then it should be attached to the back pointer which rejects the lowest g (n) value.

Step-6: Return to Step-2.

Example for A* search Algorithm

State	h(n) Value
S	5
A	3
B	4
C	2
D	6
G	0



Step-1:

S → A ; F(n) = g(n) + h(n)

= 1 + 3 = 4

S → G ; F(n) = 10 + 0 = 10 (Hold)

Step-2:

S → A→ B ; F(n) = 3 + 4 = 7 (Hold)

S → A→ C ; F(n) = 2 + 2 = 4

Step-3:

S → A→C→D ; F(n) = 5 + 6 = 11 (Hold)

S → A→ C→G ; F(n) = 6 + 0 = 6

Final Path: S → A→ C→G ; F(n) = 6

© Prof. Dr. A K M Akhtar Hossain
Dept. of CSE, RU

Example for A* search Algorithm

Step-1: Initialization; Start State = S; Goal State = G;	=> initialization OPEN List, CLOSE List Open[S], Close[ ] Open[A,G], Close[S] Open[G], Close[S,A] Open[C,B,G], Close[S,A] Open[G,D,B,G], Close[S,A,C] Open[D,B,G], Close[S,A,C,G]
Step-2: S → A ; F(n) = g(n) + h(n) = 1 + 3 = 4 S → G ; F(n) = 10 + 0 = 10 (Hold)	
Step-3: S → A→ B ; F(n) = 3 + 4 = 7 (Hold) S → A→ C ; F(n) = 2 + 2 = 4	
Step-4: S → A→C →D ; F(n) = 5 + 6 = 11 (Hold) S → A→ C→**G** ; F(n) = 6 + 0 = 6	Lower cost goal is found. So, we have gotten **Final Path**. **Final Path= S → A→ C→ G**
Final Path: S → A→ C→ **G** ; F(n) = 6	

Reference Books

- 1. Dan W. Patterson, AI & Expert Systems.
- 2. Rich, Knight, Nair, AI, Third Edition.

- 3. https://www.geeksforgeeks.org/a-search-algorithm/

•THE END

Expert System

Prof. Dr. A K M Akhtar Hossain
Dept. of CSE, RU

What is Expert System?

- **Expert System** is an interactive and reliable computer-based **decision-making system** which uses both facts and heuristics to solve complex decision-making problems.
- It is considered at the highest level of human intelligence and expertise.
- The purpose of an expert system is to solve the most complex issues in a **specific domain**.

Expert Systems (ES)

- **Expert systems are knowledge based programs which provide expert quality solutions to the problems in specific domain of applications**.

- The core components of expert system are
  - **Knowledge Base** and
  - **Navigational Capability** (**Inference Engine**)

[Later on we discuss on these topics.]

Expert Systems (ES)

- Generally its knowledge is extracted from human experts in the domain of application by knowledge Engineer.

- **A process of gathering knowledge from domain expert and codifying it according to the formalism is called knowledge engineering**.

Applications of ES

- Stock market trading
- Airline scheduling & Cargo scheduling
- Information management
- Hospitals and medical facilities
- Design and Manufacturing
- Computer maintenances & Virus detection
- Planning and scheduling
- Warehouse optimization
- Financial decision making Knowledge publishing
- Monitoring & control Office or Factory
- Agricultural Plants Disease Management System
- Satellite control missile system

Phases in building Expert System

There are five basic **steps/phases** to build an expert system. These are as follows:

- **Identification Phase:**
- **Conceptualization Phase:**
- **Formalization Phase:**
- **Implementation Phase:**
- **Testing Phase:**

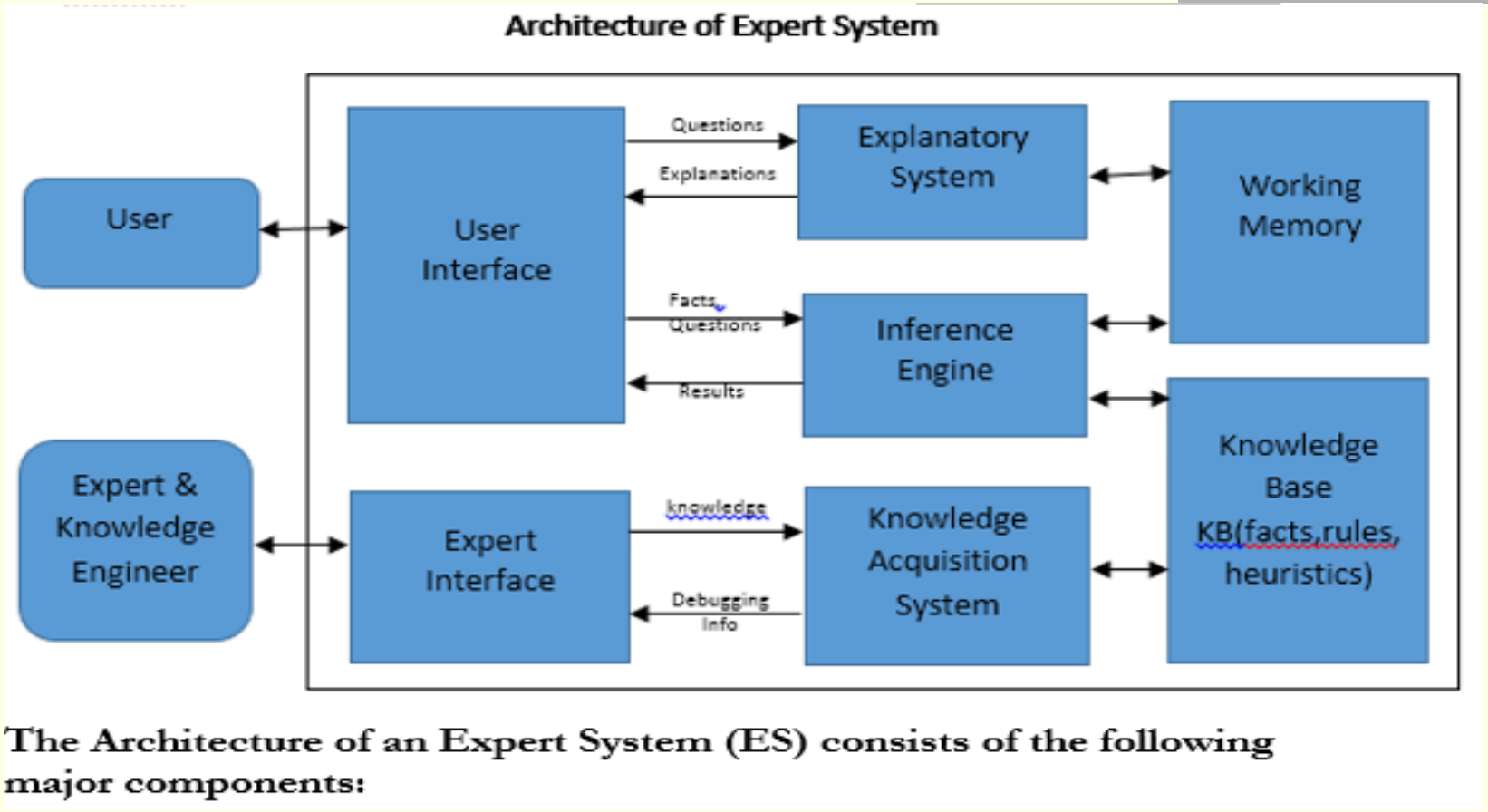
Phases in building Expert System

- ***Identification Phase***:
  - Knowledge engineer finds out important features of the problem with the help of domain expert (human).
  - He tries to determine the type and scope of the problem, the kind of resources required, goal and objective of the ES.
- **Conceptualization Phase:**
  - In this phase, knowledge engineer and domain expert decide the concepts, relations and control mechanism needed to describe a problem solving.

Phases in building Expert System

- ***Formalization Phase:***
  - It involves expressing the key concepts and relations in some framework supported by ES building tools.
  - Formalized knowledge consists of data structures, inference rules, control strategies and languages for implementation.
- ***Implementation Phase:***
  - During this phase, formalized knowledge is converted to working computer program initially called prototype of the whole system.
- ***Testing Phase:***
  - It involves evaluating the performance and utility of prototype systems and revising it, if it is needed.
  - Domain expert evaluates the prototype system and his feedback help knowledge engineer to revise it.

Expert System Architecture



The Architecture of an Expert System (ES) consists of the following major components:

Knowledge Base (KB)

- **KB consists of knowledge about problem domain in the form of static and dynamic databases.**
- Static knowledge consists of
  - rules and facts which is complied as a part of the system and does not change during execution of the system.
- **Dynamic knowledge consists of facts related to a particular consultation of the system.**
  - At the beginning of the consultation, the dynamic knowledge base often called working memory is empty.
  - **As a consultation progresses, dynamic knowledge base grows and is used along with static knowledge in decision making.**
- **Working memory is deleted at the end of consultation of the system.**

Navigational Capability /Inference Engine

- **Inference Engine consists of inference mechanism and control strategy.**
- **Inference means search through knowledge base and derive new knowledge.**
- It involve formal reasoning involving matching and unification similar to the one performed by human expert to solve problems in a specific area of knowledge.
- **Inference operates by using modus ponen rule.**
- Control strategy determines the order in which rules are applied.
- There are mainly two types of control mechanism. These are **forward chaining** and **backward chaining**.

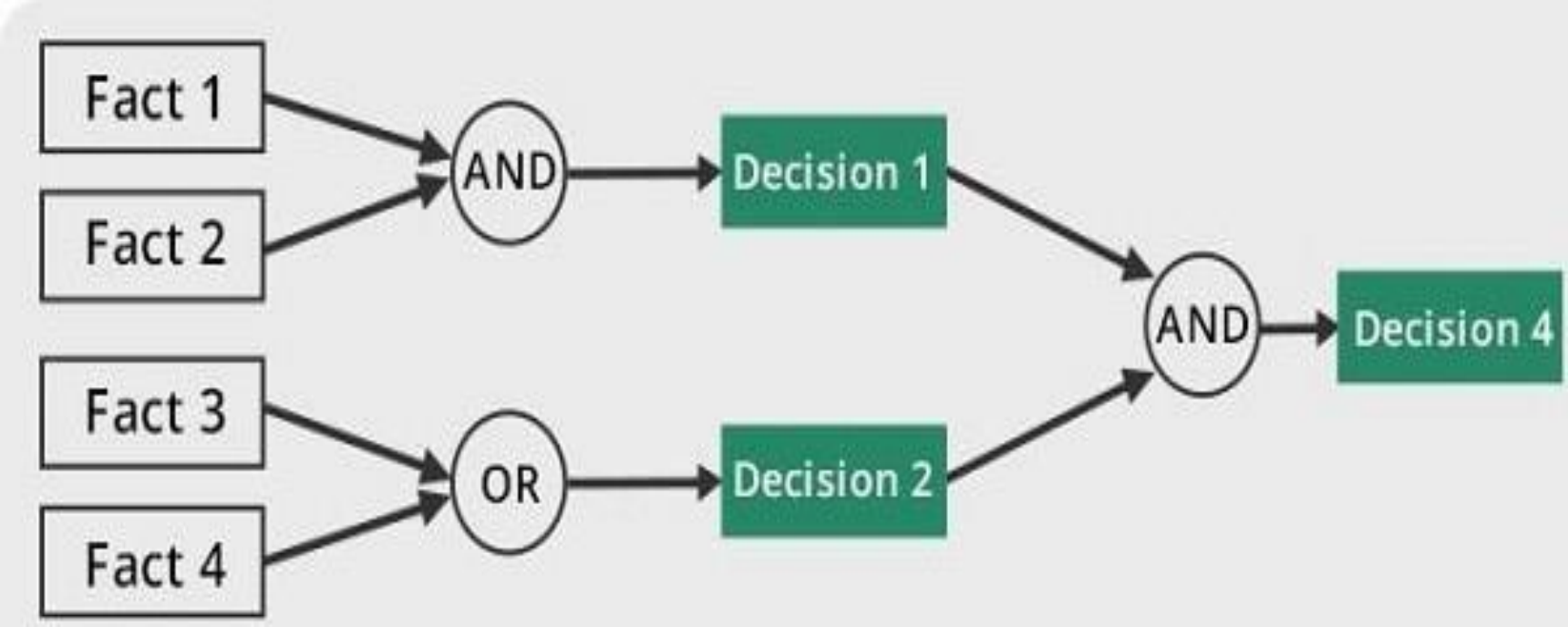
Navigational Capability /Inference Engine

- **Forward Chaining:** A reasoning process that begins with the known facts and works forward, trying to find a successful goal from the known facts.
- **Backward Chaining:** A type of reasoning process that begins at a specified goal and works backward in an attempt to prove that the goal is true.

Forward Chaining

- Forward chaining is a method of reasoning in artificial intelligence in which inference rules are applied to existing data to extract additional data until an endpoint (goal) is achieved.
- In this type of chaining, the inference engine starts by evaluating existing facts, derivations, and conditions before deducing new information. An endpoint (goal) is achieved through the manipulation of knowledge that exists in the knowledge base.

Forward Chaining



Applications of **Forward Chaining**

- Forward chaining can be used in planning, monitoring, controlling, and interpreting applications.
- **Properties of forward chaining:**
- The process uses a down-up approach (bottom to top).
- It starts from an initial state and uses facts to make a conclusion.
- This approach is data-driven.
- It’s employed in expert systems and production rule system.

Forward Chaining

- **Advantages**
- It can be used to draw multiple conclusions.
- It provides a good basis for arriving at conclusions.
- It’s more flexible than backward chaining because it does not have a limitation on the data derived from it.

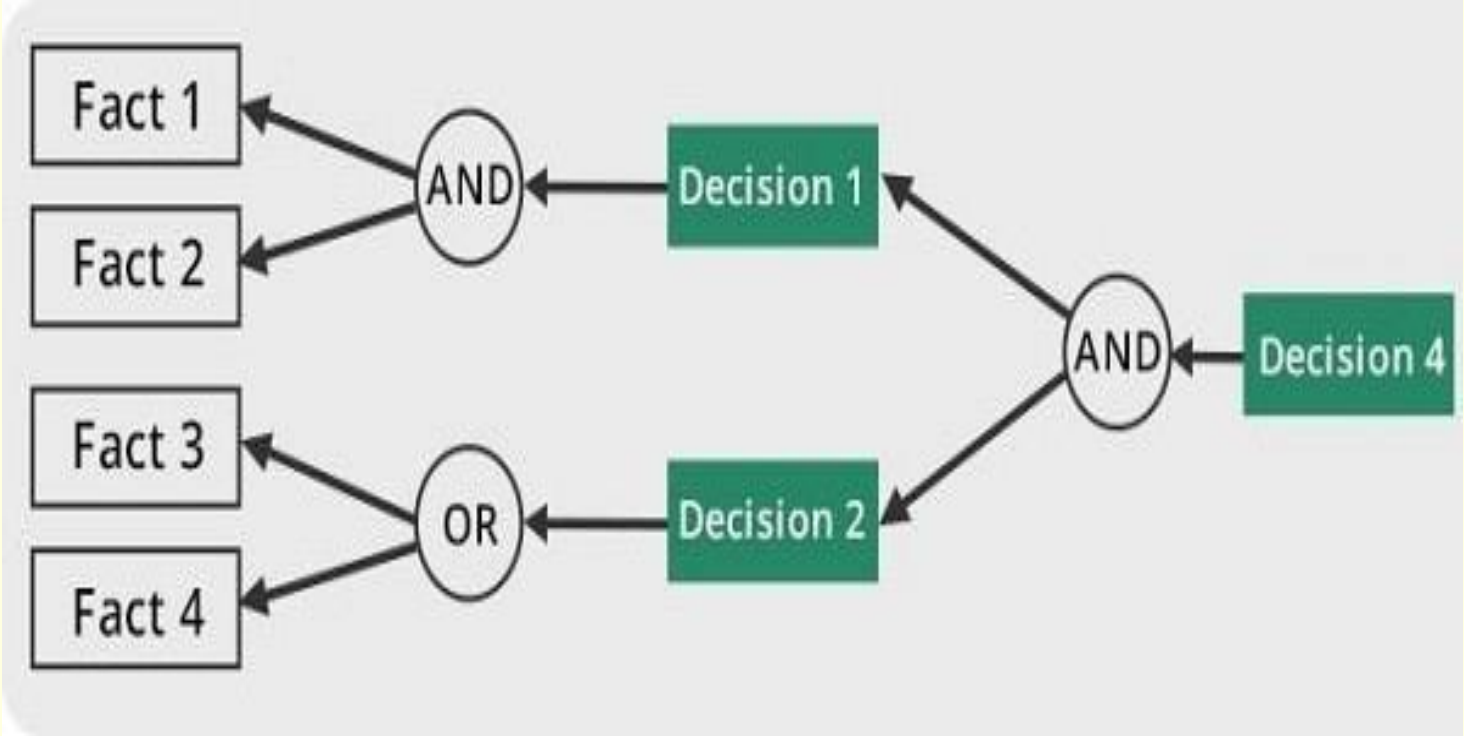
Forward Chaining

- **Disadvantages**
- The process of forward chaining may be time-consuming. It may take a lot of time to eliminate and synchronize available data.
- Unlike backward chaining, the explanation of facts or observations for this type of chaining is not very clear.
- The former uses a goal-driven method that arrives at conclusions efficiently.

Backward Chaining

- Backward chaining is a concept in artificial intelligence that involves backtracking from the endpoint or goal to steps that led to the endpoint.
- This type of chaining starts from the goal and moves backward to comprehend the steps that were taken to attain this goal.
- The backtracking process can also enable a person establish logical steps that can be used to find other important solutions.

Backward Chaining



Applications of Backward Chaining

- Backward chaining can be used in debugging, diagnostics, and prescription applications.
- **Properties of backward chaining:**
- The process uses an up-down approach (top to bottom).
- It's a goal-driven method of reasoning.
- The endpoint (goal) is subdivided into sub-goals to prove the truth of facts.
- A backward chaining algorithm is employed in inference engines, game theories, and complex database systems.

Backward Chaining

- **Advantages**
- The result is already known, which makes it easy to deduce inferences.
- It's a quicker method of reasoning than forward chaining because the endpoint is available.
- In this type of chaining, correct solutions can be derived effectively if pre-determined rules are met by the inference engine.
- **Disadvantages**
- The process of reasoning can only start if the endpoint is known.
- It doesn't deduce multiple solutions or answers.
- It only derives data that is needed, which makes it less flexible than forward chaining.

Knowledge Acquisition

- **Knowledge acquisition module allows system to acquire knowledge about the problem domain.**
- Sources of Knowledge for ES
  - text books, reports, research papers, case studies,
  - empirical (experimental)data and
  - domain expert experience.
- **Updation of Knowledge can be done using knowledge acquisition module of the system.**
  - insertion,
  - deletion and
  - updation of existing knowledge

Explanation Module

- Most expert systems have explanation facilities that allow the user to ask the system ***why*** it asked some question, and ***how*** it reached to conclusion.
- It contains 'How' and 'Why' modules attached to it.
  - **The sub-module ‘How’ tells the user about the process through which system has reached to a particular solution.**
  - **‘Why' sub-module tells that why is that particular solution offered.**
- It explains user about the reasoning behind any particular problem solution.
- Questions are answered by referring to the system goals, the rules being used, and any existing problem data.

Explanation Modules Responses

User	**Why module**
System	Do you have running nose?
User	Yes.
System	I am trying to prove that you have cold. Do you have headache?
User	Yes
System	Do you have sore throat?
User	No
System	Then you do not have cold. Do you have fever?
User	Yes
System	Now I am looking at the possibility of measles. Do you have cough?
User	Yes
System	I can infer measles using rule “If symptoms are fever, cough, running_nose, then patient has measles” measles is concluded.
User	**How Module**
System	Since you have fever, running_nose and cough and there is a rule “If symptoms are fever, cough, running_nose, then patient has measles”. So measles is concluded for you.

User Interfaces

- Allows user to communicate with system in interactive mode and helps system to create working knowledge for the problem to be solved.

Dialogue Module (User Interface)	
System	Do you have fever?
User	Yes
System	Do you have bad throat?
User	No
System	Do you have cough?
User	Yes
System	Are you suffering from running nose?
User	Yes
System	Are you suffering from headache?
User	No

Special interfaces

- It may be used for specialized activities such as handling uncertainty in knowledge.
- This is a major area of expert systems research that involves methods for reasoning with uncertain data and uncertain knowledge.
- Knowledge is generally incomplete and uncertain.
- To deal with uncertain knowledge, a rule may have associated with it a ***confidence factor*** **or a weight**.
- The set of methods for using uncertain knowledge in combination with uncertain data in the reasoning process is called ***reasoning with uncertainty***.

Rule Based Expert Systems

- **A rule based expert system is one in which knowledge base is in the form of rules and facts.**
  - Knowledge in the form of **rules and facts is most popular way** in designing expert systems.

- It is also called **production system**.

Rule Based Expert Systems

- **Example:** Suppose doctor gives a rule for measles as follows:

  "If symptoms are fever, cough, running_nose, rash and conjunctivitis then patient probably has measles".

- Turbo Prolog **(/SWI Prolog)** is most suitable for implementing such systems.

```
hypothesis(measles) :- symptom(fever), symptom(cough),
        symptom(running_nose), symptom(conjunctivitis),
symptom(rash).
```

- [**SWI is** derived from Sociaal-Wetenschappelijke Informatica ("Social Science Informatics")]

Example

```
domains
        disease, indication
        patient = string
predicates
        hypothesis(patient,disease)
        symptom(name,indication)
        response(char)
        go
clauses
        go :-
                write("What is the patient's name? "),
                readln(Patient),
                hypothesis(Patient,Disease),
                write(Patient," probably has ",Disease,"/"),nl.
```

Example Continue

```
go :-
                write("Sorry, I don't seem to be able to"),nl.

                write("diagnose the disease."),nl.
        symptom(Patient,fever) :-
                write("Does ",Patient,"have a fever (y/n) ?");
                response(Reply),
                Reply='y'.
        symptom(Patient,rash) :-
                write("Does ",Patient,"have a rash (y/n) ?");
                response(Reply),
                Reply='y'.
        symptom(Patient,headache) :-
                write("Does ",Patient,"have a headache (y/n) ?");
                response(Reply),
                Reply='y'.
```

Example Continue

```
symptom(Patient,runny_nose) :-
                write("Does ",Patient,"have a runny_nose (y/n) ?");
                response(Reply),
                Reply='y'.

        symptom(Patient,conjunctivitis) :-
                write("Does ",Patient,"have a conjunctivitis (y/n) ?");
                response(Reply),
                Reply='y'.
        symptom(Patient,cough) :-
                write("Does ",Patient,"have a cough (y/n) ?");
                response(Reply),
                Reply='y'.

        symptom(Patient,body_ache) :-
                write("Does ",Patient,"have a body_ache (y/n) ?");
                response(Reply),
                Reply='y'.
```

Example Continue

```
symptom(Patient,chills) :-
                write("Does ",Patient,"have a chills (y/n) ?");
                response(Reply),
                Reply='y'.

        symptom(Patient,sore_throat) :-
                write("Does ",Patient,"have a sore_throat (y/n) ?");
                response(Reply),
                Reply='y'.

        symptom(Patient,sneezing) :-
                write("Does ",Patient,"have a sneezing (y/n) ?");
                response(Reply),
                Reply='y'.

        symptom(Patient,swollen_glands) :-
                write("Does ",Patient,"have a swollen_glands (y/n) ?");
                response(Reply),
                Reply='y'.
```

Example Continue

```
hypothesis(Patient,measles) :-
                    symptom(Patient,fever),
                    symptom(Patient,cough),
                    symptom(Patient,conjunctivitis),
                    symptom(Patient,runny_nose),
                    symptom(Patient,rash).
          hypothesis(Patient,german_measles)
                    symptom(Patient,fever),
                    symptom(Patient,headache),
                    symptom(Patient,runny_nose),
                    symptom(Patient,rash).
          hypothesis(Patient,flu) :-
                    symptom(Patient,fever),
                    symptom(Patient,headache),
                    symptom(Patient,body_ache),
                    symptom(Patient,conjunctivitis),
                    symptom(Patient,chills),
                    symptom(Patient,sore_throat),
                    symptom(Patient,cough),
                    symptom(Patient,runny_nose).
```

Example Continue

```
hypothesis(Patient,common_cold) :-
                    symptom(Patient,headache),
                    symptom(Patient,sneezing),
                    symptom(Patient,sore_throat),
                    symptom(Patient,chills),
                    symptom(Patient,runny_nose).

          hypothesis(Patient,mumps) :-
                    symptom(Patient,fever),
                    symptom(Patient,swollen_glands),

          hypothesis(Patient,chicken_pox) :-
                    symptom(Patient,fever),
                    symptom(Patient,rash),
                    symptom(Patient,body_ache),
                    symptom(Patient,chills).
```

Example Continue

```
hypothesis(Patient,whooping_cough) :-
                    symptom(Patient,cough),
                    symptom(Patient,sneezing),
                    symptom(Patient,runny_nose).

        response(Reply) :-
                    readchar(Reply),
                    write(Reply),nl.
```

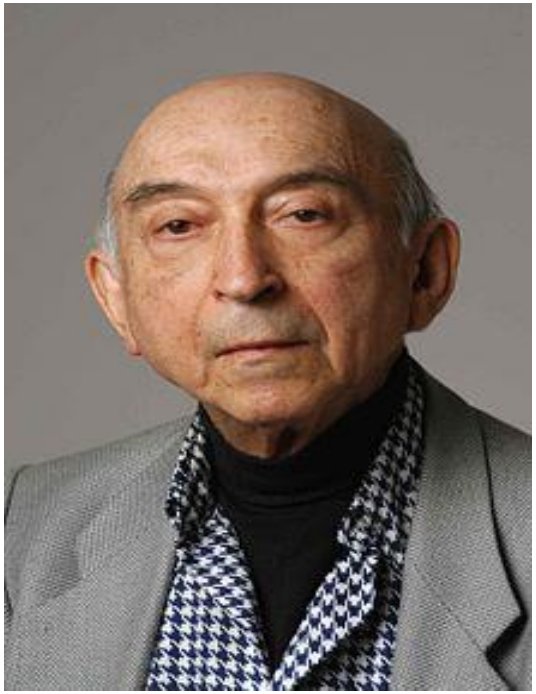
- THANKS
- &
- THE END

Fuzzy Logic In AI

Prof. Dr. A K M Akhtar Hossain
Dept. of CSE, RU

Fuzzy Logic History

- The term *fuzzy logic* was introduced by **Lotfi A. Zadeh** in the year **1965.**



Lotfi Aliasker Zadeh was born in 4 February 1921 in Baku, Azerbaijan, SSR.

What is Fuzzy Logic?

- **Fuzzy logic** is a form of **many-valued logic** in which the truth values of variables may be any real number between 0 and 1, considered to be "fuzzy".
- In Boolean logic, the truth values of variables may only be 0 or 1, often called "crisp" values.
- **Fuzzy logic has been employed to handle the concept of partial truth, where the truth value may range between completely true and completely false.**
- Furthermore, when linguistic variables are used, these degrees may be managed by specific (membership) functions.

Fuzzy logic

- Fuzzy logic is a branch of fuzzy set theory, which deals with the representation and inference from knowledge. Fuzzy logic, unlike other logical systems, deals with imprecise or uncertain knowledge.
- In this narrow, and perhaps correct sense, fuzzy logic is just one of the branches of fuzzy set theory.

Fuzzy Applications

- Pattern recognition and classification
- Fuzzy clustering
- Image and speech processing
- Fuzzy systems for prediction
- Fuzzy control
- Monitoring
- Diagnosis
- Optimization and decision making
- Group decision making
- **Washing machines**
- **Electric Fan**; etc.

Applications

- **Washing machines** now use fuzzy logic. They can sense how heavy a load is and determine the correct amount of water and detergent, speed of agitation, and length of the wash cycles.
- There is no single standard for the dirty laundry. Fuzzy logic enables the machine's computer to make “in between” decisions.

Example:

- **Words like young, tall, good, or high are fuzzy.**

❑ There is no single quantitative value which defines the term **young**.

❑ For some people, age **25 is young**, and for others, **age 35 is young.**

❑ The concept young has no clean boundary.

❑ Age 1 is definitely **child** and age 100 is definitely **not child**;

❑ Age 35 has some possibility of being young and usually depends.

- **On the context in which it is being considered.**

Examples: Fuzzy Linguistic variables

- Fuzzy sets and concepts are commonly used in natural language :-
- John is tall.
- Dan is smart.
- Alex is happy .
- Today is hot .
- Car runs faster.

Fuzzy set theory

- Fuzzy set theory is an extension of classical set theory where elements have varying degrees of membership.
- A logic based on the two truth values, *True and False, is sometimes inadequate when* describing human reasoning.
- Fuzzy logic uses the whole interval between **0 (false) and 1 (true) to describe human reasoning.**

A **Fuzzy Set**

- A **Fuzzy Set is any set that allows its members to have different** degree of membership, called **membership function*, in the interval***

  [0 , 1].

- The **degree of membership or truth is not same as probability;**

- fuzzy truth is not likelihood of some event or condition.
- fuzzy truth represents membership in vaguely defined sets;

Fuzzy logic

- **Fuzzy logic is derived from fuzzy set theory dealing with reasoning** that is approximate rather than precisely deduced from classical predicate logic.
- Fuzzy logic is capable of handling inherently imprecise concepts.
- Fuzzy logic allows in linguistic form the set membership values to imprecise concepts like **"slightly", "quite" and "very".**
- Fuzzy set theory defines Fuzzy Operators on Fuzzy Sets.

Crisp and Non-Crisp Set

The notations used to express these mathematically are

$$A : X \rightarrow [0, 1]$$

$$A(x) = 1, \; x \text{ is a member of } A$$

$$A(x) = 0, \; x \text{ is not a member of } A$$

Eq.(1)

Crisp and Non-Crisp Set

Alternatively, the set **A** can be represented for all elements $x \in X$ by its characteristic function $\mu_A(x)$ defined as

$$\mu_A(x) = \begin{cases} 1 & \text{if } x \in X \\ 0 & \text{otherwise} \end{cases} \qquad \text{Eq.(2)}$$

Thus in classical set theory $\mu_A(x)$ has only the values **0** ('false') and **1** ('true''). Such sets are called **crisp sets.**

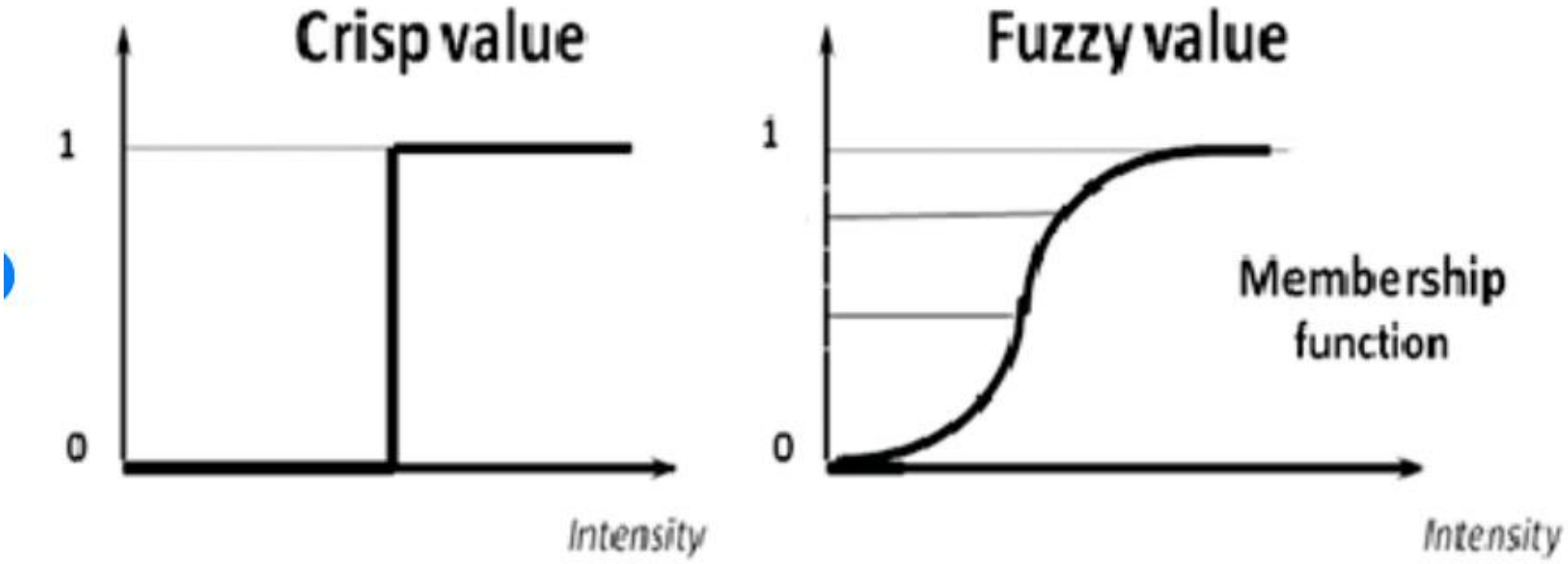
Non-Crisp Set

- For Non-crisp sets the characteristic function $\mu_A(x)$ can be defined.
  - The characteristic function $\mu_A(x)$ of Eq. (2) for the crisp set is generalized for the Non-crisp sets.
  - This generalized characteristic function $\mu_A(x)$ of Eq.(2) is called **membership function**.

  Such Non-crisp sets are called **Fuzzy Sets**.

- Crisp set theory is not capable of representing descriptions and classifications in many cases; In fact, Crisp set does not provide adequate representation for most cases.

Crisp and Non-Crisp Value



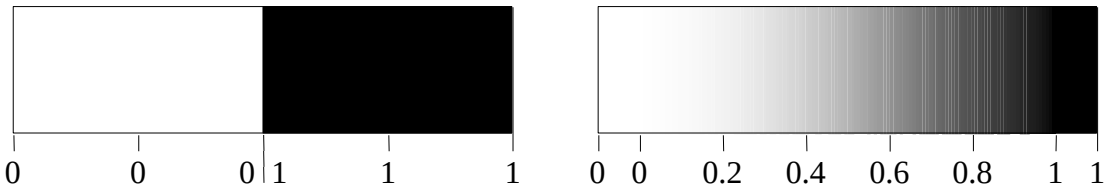
Difference between Crisp and Fuzzy Sets.

More Definitions (On Fuzzy Logic)

- Fuzzy logic is a set of mathematical principles for knowledge representation based on **degrees of membership**.

- Unlike two-valued Boolean logic, fuzzy logic is **multi-valued**. It deals with **degrees of membership** and **degrees of truth**.

- Fuzzy logic uses the continuum of logical values between 0 (completely false) and 1 (completely true). Instead of just black and white, it employs the spectrum of colours, accepting that things can be partly true and partly false at the same time.

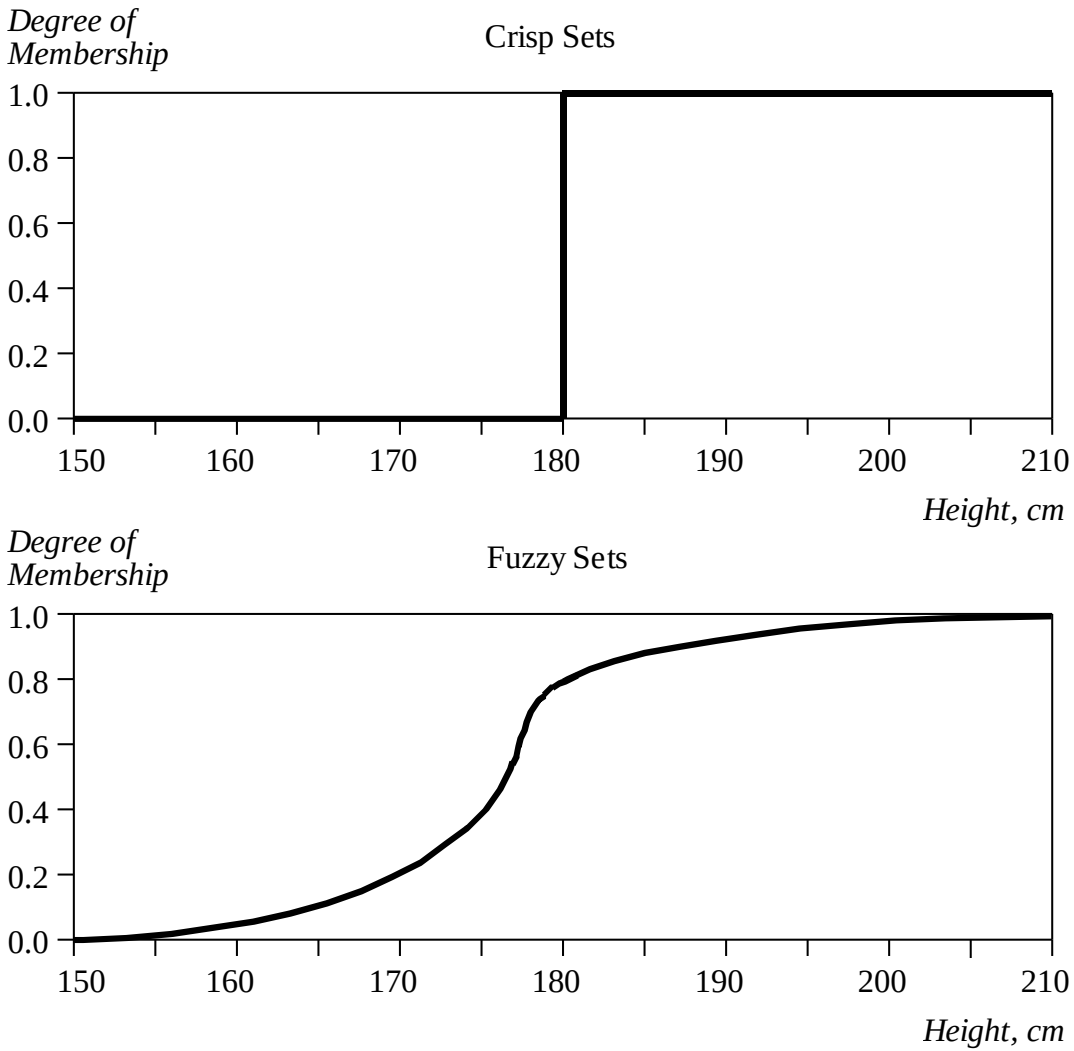


(*a*) Boolean Logic. (*b*) Multi-valued Logic.

Crisp Vs Fuzzy Sets

The x-axis represents the **universe of discourse** – the range of all possible values applicable to a chosen variable. In our case, the variable is the man height. According to this representation, the universe of men's heights consists of all tall men.

The y-axis represents the **membership value of the fuzzy set**. In our case, the fuzzy set of "tall men" maps height values into corresponding membership values.



Example: Fuzzy Sets

- **Hard science with IF-THEN rules:-**
- Example:- The logic for a simple temperature regulator that uses a fan might look like this:-

Example:

- IF temperature is **very cold** THEN **stop fan**
- IF temperature is **cold** THEN fan speed is **zero**
- IF temperature is **warm** THEN fan speed is **moderate**
- IF temperature is **hot** THEN fan speed is **high**

© Prof. Dr. A K M Akhtar Hossain
Dept. of CSE, RU

- **THE END**
- **Thanks**